UNIT-1

Introduction, Toolboxes: Python, fundamental libraries for data Scientists. Integrated development environment (IDE). Data operations: Reading, selecting, filtering, manipulating, sorting, grouping, rearranging, ranking, and plotting.

Introduction to Data Science

1.1 What is Data Science?

You have, no doubt, already experienced data science in several forms. When you are looking for information on the web by using a search engine or asking your mobile phone for directions, you are interacting with data science products. Data science has been behind resolving some of our most common daily tasks for several years.

Most of the scientific methods that power data science are not new and they havebeen out there, waiting for applications to be developed, for a long time. Statistics is an old science that stands on the shoulders of eighteenth-century giants such as Pierre Simon Laplace (1749–1827) and Thomas Bayes (1701–1761). Machine learning isyounger, but it has already moved beyond its infancy and can be considered a well-established discipline. Computer science changed our lives several decades ago andcontinues to do so; but it cannot be considered new.

So, why is data science seen as a novel trend within business reviews, in technologyblogs, and at academic conferences?

The novelty of data science is not rooted in the latest scientific knowledge, but in a disruptive change in our society that has been caused by the evolution of technology: datification. Datification is the process of rendering into data aspects of the world that have never been quantified before. At the personal level, the list of datified conceptsis very long and still growing: business networks, the lists of books we are reading,the films we enjoy, the food we eat, our physical activity, our purchases, our drivingbehavior, and so on. Even our thoughts are datified when we publish them on our favorite social network; and in a not so distant future, your gaze could be datified by wearable vision registering devices. At the business level, companies are datifying semi-structured data that were previously discarded: web activity logs, computer network activity, machinery signals, etc. Nonstructured data, such as written reports,e-mails, or voice recordings, are now being stored not only for archive purposes butalso to be analyzed.

However, datification is not the only ingredient of the data science revolution. The other ingredient is the democratization of data analysis. Large companies such as Google, Yahoo, IBM, or SAS were the only players in this field when data science had no name. At the beginning of the century, the huge computational resources of those companies allowed them to take advantage of datification by using analytical techniques to develop innovative products and even to take decisions about their own business. Today, the analytical gap between those companies and the rest of the world (companies and people) is shrinking. Access to cloud computing allows any individual to analyze huge amounts of data in short periods of time. Analytical knowledge is free and most of the crucial algorithms that are needed to create a solution can be found, because open-source development is the norm in this field. As a result, the possibility of using rich data to take evidence-based decisions is open to virtually any person or company.

Data science is commonly defined as a methodology by which actionable insights can be inferred from data. This is a subtle but important difference with respect to previous approaches to data analysis, such as business intelligence or exploratory statistics. Performing data science is a task with an ambitious objective: the produc-tion of beliefs informed by data and to be used as the basis of decision-making. In the absence of data, beliefs are uninformed and decisions, in the best of cases, are based on best practices or intuition. The representation of complex environments by rich data opens up the possibility of applying all the scientific knowledge we have regarding how to infer knowledge from data.

In general, data science allows us to adopt four different strategies to explore theworld using data:

1. *Probing reality*. Data can be gathered by passive or by active methods. In the latter case, data represents the response of the world to our actions. Analysis of those responses can be extremely valuable when it comes to taking decisions about our subsequent actions. One of the best examples of this strategy is the use of A/B testing for web development: What is the best button size and color? The best answer can only be found by probing the world.
2. *Pattern discovery*. Divide and conquer is an old heuristic used to solve complex problems; but it is not always easy to decide how to apply this common sense to problems. Datified problems can be analyzed automatically to discover useful patterns and natural clusters that can greatly simplify their solutions. The use of this technique to profile users is a critical ingredient today in such importantfields as programmatic advertising or digital marketing.
3. *Predicting future events*. Since the early days of statistics, one of the most important scientific questions has been how to build robust data models that are capa- ble of predicting future data samples. Predictive analytics allows decisions to be taken in response to future events, not only reactively. Of course, it is not possible to predict the future in any environment and there will always be unpre- dictable events; but the identification of predictable events represents valuableknowledge. For example, predictive analytics can be used

to optimize the tasks

planned for retail store staff during the following week, by analyzing data suchas weather, historic sales, traffic conditions, etc.

4. *Understanding people and the world*. This is an objective that at the moment is beyond the scope of most companies and people, but large companies and governments are investing considerable amounts of money in research areas such as understanding natural language, computer vision, psychology and neu- roscience. Scientific understanding of these areas is important for data science because in the end, in order to take optimal decisions, it is necessary to know the real processes that drive people's decisions and behavior. The development of deep learning methods for natural language understanding and for visual object recognition is a good example of this kind of research.

Introduction

In this chapter, first we introduce some of the tools that data scientists use. The toolbox of any data scientist, as for any kind of programmer, is an essential ingredient for success and enhanced performance. Choosing the right tools can save a lot of time and thereby allow us to focus on data analysis.

The most basic tool to decide on is which programming language we will use. Many people use only one programming language in their entire life: the first and only one they learn. For many, learning a new language is an enormous task that, if at all possible, should be undertaken only once. The problem is that some languages are intended for developing high-performance or production code, such as C, C++, or Java, while others are more focused on prototyping code, among these the best known are the so-called scripting languages: Ruby, Perl, and Python. So, depending on the first language you learned, certain tasks will, at the very least, be rather tedious. The main problem of being stuck with a single language is that many basic tools simply will not be available in it, and eventually you will have either to reimplementthem or to create a bridge to use some other language just for a specific task.

In conclusion, you either have to be ready to change to the best language for each task and then glue the results together, or choose a very flexible language with a rich ecosystem (e.g., third-party open-source libraries). In this book we have selected Python as the programming language.

Why Python?

Python[1] is a mature programming language but it also has excellent properties for newbie programmers, making it ideal for people who have never programmed before. Some of the most remarkable of those properties are easy to read code, suppression of non-mandatory delimiters, dynamic typing, and dynamic memory usage. Pythonis an interpreted language, so the code is executed immediately in the Python con- sole without needing the compilation step to machine language. Besides the Python console (which comes included with any Python installation) you can find other in-teractive consoles, such as IPython,[2] which give you a richer environment in whichto execute your Python code.

Currently, Python is one of the most flexible programming languages. One of its main characteristics that makes it so flexible is that it can be seen as a multiparadigm language. This is especially useful for people who already know how to program with other languages, as they can rapidly start programming with Python in the same way. For example, Java programmers will feel comfortable using Python as it supports the object-oriented paradigm, or C programmers could mix Python and C code using *cython*. Furthermore, for anyone who is used to programming in functional languages such as Haskell or Lisp, Python also has basic statements for functional programming in its own core library.

In this book, we have decided to use Python language because, as explained before, it is a mature language programming, easy for the newbies, and can be used as a specific platform for data scientists, thanks to its large ecosystem of scientific libraries and its high and vibrant community. Other popular alternatives to Python for data scientists are R and MATLAB/Octave.

Fundamental Python Libraries for Data Scientists

The Python community is one of the most active programming communities with a huge number of developed toolboxes. The most popular Python toolboxes for any data scientist are NumPy, SciPy, Pandas, and Scikit-Learn.

Numeric and Scientific Computation: NumPy and SciPy

NumPy[3] is the cornerstone toolbox for scientific computing with Python. NumPy provides, among other things, support for multidimensional arrays with basic oper-ations on them and useful linear algebra functions. Many toolboxes use the NumPyarray representations as an efficient basic data structure. Meanwhile, *SciPy* provides a collection of numerical algorithms and domain-specific toolboxes, including signal processing, optimization, statistics, and much more. Another core toolbox in SciPyis the plotting library *Matplotlib*. This toolbox has many tools for data visualization.

SCIKIT-Learn: Machine Learning in Python

Scikit-learn[4] is a machine learning library built from NumPy, SciPy, and Matplotlib. Scikit-learn offers simple and efficient tools for common tasks in data analysis such as classification, regression, clustering, dimensionality reduction, model selection, and preprocessing.

PANDAS: Python Data Analysis Library

Pandas[5] provides high-performance data structures and data analysis tools. The keyfeature of Pandas is a fast and efficient DataFrame object for data manipulation withintegrated indexing. The DataFrame structure can be seen as a spreadsheet which offers very flexible ways of working with it. You can easily transform any dataset in the way you want, by reshaping it and adding or removing columns or rows. It also provides high-performance functions for aggregating, merging, and joining dataset-
s. Pandas also has tools for importing and exporting data from different formats: comma-separated value (CSV), text files, Microsoft Excel, SQL databases, and the fast HDF5 format. In many situations, the data you have in such formats will not be complete or totally structured. For such cases, Pandas offers handling of miss-ing data and intelligent data alignment. Furthermore, Pandas provides a convenientMatplotlib interface.

Data Science Ecosystem Installation

Before we can get started on solving our own data-oriented problems, we will need toset up our programming environment. The first question we need to answer concerns

Python language itself. There are currently two different versions of Python: Python 2.X and Python 3.X. The differences between the versions are important, so there isno compatibility between the codes, i.e., code written in Python 2.X does not workin Python 3.X and vice versa. Python 3.X was introduced in late 2008; by then, a lotof code and many toolboxes were already deployed using Python 2.X (Python 2.0 was initially introduced in 2000). Therefore, much of the scientific community didnot change to Python 3.0 immediately and they were stuck with Python 2.7. By now, almost all libraries have been ported to Python 3.0; but Python 2.7 is still maintained, so one or another version can be chosen. However, those who already have a large amount of code in 2.X rarely change to Python 3.X. In our examples throughout thisbook we will use Python 2.7.

Once we have chosen one of the Python versions, the next thing to decide is

whether we want to install the data scientist Python ecosystem by individual tool- boxes, or to perform a bundle installation with all the needed toolboxes (and a lot more). For newbies, the second option is recommended. If the first option is chosen,then it is only necessary to install all the mentioned toolboxes in the previous section, in exactly that order.

However, if a bundle installation is chosen, the Anaconda Python distribution[6] is then a good option. The Anaconda distribution provides integration of all the Python toolboxes and applications needed for data scientists into a single directory without mixing it with other Python toolboxes installed on the machine. It contain- s, of course, the core toolboxes and applications such as NumPy, Pandas, SciPy, Matplotlib, Scikit-learn, IPython, Spyder, etc., but also more specific tools for other related tasks such as data visualization, code optimization, and big data processing.

Integrated Development Environments (IDE)

For any programmer, and by extension, for any data scientist, the integrated development environment (IDE) is an essential tool. IDEs are designed to maximize programmer productivity. Thus, over the years this software has evolved in order tomake the coding task less complicated. Choosing the right IDE for each person is crucial and, unfortunately, there is no “one-size-fits-all” programming environment. The best solution is to try the most popular IDEs among the community and keep whichever fits better in each case.

In general, the basic pieces of any IDE are three: the editor, the compiler, (or interpreter) and the debugger. Some IDEs can be used in multiple programming languages, provided by language-specific plugins, such as Netbeans[7] or Eclipse.[8] Others are only specific for one language or even a specific programming task. In

the case of Python, there are a large number of specific IDEs, both commercial (PyCharm,[9] WingIDE[10] ...) and open-source. The open-source community helps IDEs to spring up, thus anyone can customize their own environment and share it with the rest of the community. For example, Spyder[11] (Scientific Python Development EnviRonment) is an IDE customized with the task of the data scientist in mind.

Web Integrated Development Environment (WIDE): Jupyter

With the advent of web applications, a new generation of IDEs for interactive languages such as Python has been developed. Starting in the academia and e-learning communities, web-based IDEs were developed considering how not only your codebut also all your environment and executions can be stored in a server. One of the first applications of this kind of WIDE was developed by William Stein in early 2005 using Python 2.3 as part of his SageMath mathematical software. In

SageMath, a server can be set up in a center, such as a university or school, and then students can work on their homework either in the classroom or at home, starting from exactly the same point they left off. Moreover, students can execute all the previous steps overand over again, and then change some particular *code cell* (a segment of the docu- ment that may content source code that can be executed) and execute the operation again. Teachers can also have access to student sessions and review the progress orresults of their pupils.

Nowadays, such sessions are called notebooks and they are not only used in classrooms but also used to show results in presentations or on business dashboards. The recent spread of such notebooks is mainly due to IPython. Since December 2011, IPython has been issued as a browser version of its interactive console, called IPython notebook, which shows the Python execution results very clearly and concisely by means of cells. Cells can contain content other than code. For example, markdown (a wiki text language) cells can be added to introduce algorithms. It is also possible toinsert Matplotlib graphics to illustrate examples or even web pages. Recently, some scientific journals have started to accept notebooks in order to show experimental results, complete with their code and data sources. In this way, experiments can become completely and absolutely replicable.

Since the project has grown so much, IPython notebook has been separated fromIPython software and now it has become a part of a larger project: Jupyter[12]. Jupyter (for Julia, Python and R) aims to reuse the same WIDE for all these interpreted languages and not just Python. All old IPython notebooks are automatically imported to the new version when they are opened with the Jupyter platform; but once they

Get Started with Python for Data Scientists

Throughout this book, we will come across many practical examples. In this chapter, we will see a very basic example to help get started with a data science ecosystem from scratch. To execute our examples, we will use Jupyter notebook, although anyother console or IDE can be used.

The Jupyter Notebook Environment

Once all the ecosystem is fully installed, we can start by launching the Jupyter notebook platform. This can be done directly by typing the following command onyour terminal or command line: $ jupyter notebook

If we chose the bundle installation, we can start the Jupyter notebook platform by clicking on the Jupyter Notebook icon installed by Anaconda in the start menu or onthe desktop.

The browser will immediately be launched displaying the Jupyter notebook home-page, whose URL is http://localhost:8888/tree. Note that a special port is used; by default it is 8888. As can be seen in Fig. 2.1, this initial page displays a tree view of a directory. If we use the command line, the root directory is the same directory where we launched the Jupyter notebook. Otherwise, if we use the Anaconda launcher, the root directory is the current user directory. Now, to start a new

New Notebooks Python 2

notebook, we only need to press button at the top on the right of the thehome page.

As can be seen in Fig. 2.2, a blank notebook is created called Untitled. First of all, we are going to change the name of the notebook to something more appropriate. To do this, just click on the notebook name and rename it: DataScience-GetStartedExample.

Let us begin by importing those toolboxes that we will need for our program. In the first cell we put the code to import the *Pandas* library as pd. This is for convenience; every time we need to use some functionality from the Pandas library, we will write pd instead of pandas. We will also import the two core libraries mentioned above: the numpy library as npand the matplotlib library as plt.

In []:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

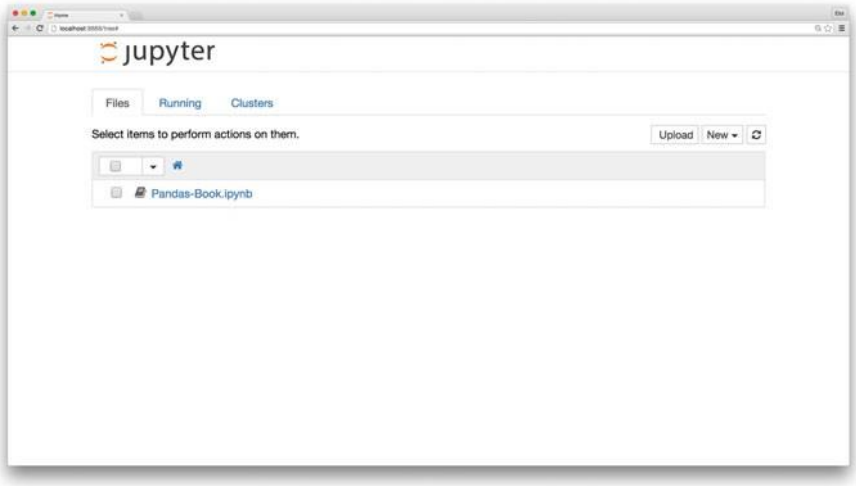


Fig. 2.1 IPython notebook home page, displaying a home tree directory

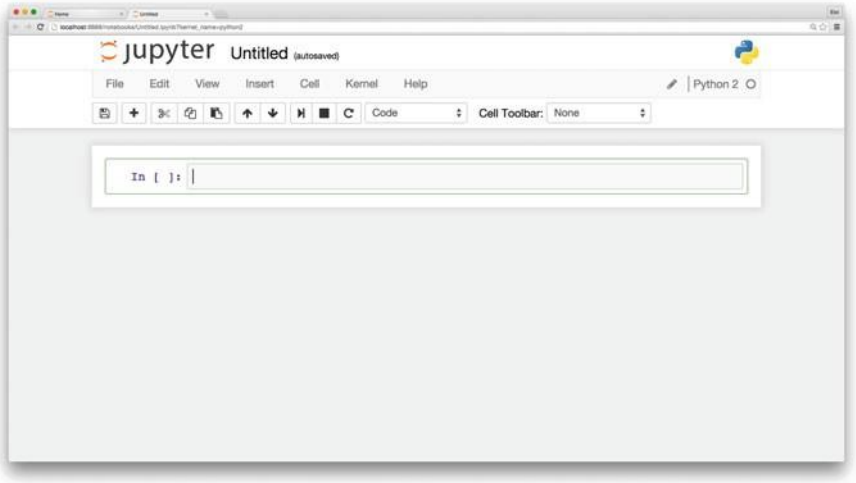


Fig. 2.2 An empty new notebook

To execute just one cell, we press the ▶ button or click on Cell > Run or press the keys Ctrl + Enter. While execution is underway, the header of the cell shows the * mark:

In [*]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

While a cell is being executed, no other cell can be executed. If you try to executeanother cell, its execution will not start until the first cell has finished its execution.Once the execution is finished, the header of the cell will be replaced by the next number of execution. Since this will be the first cell executed, the number shown will be 1. If the process of importing the libraries is correct, no output cell is produced.

In[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

For simplicity, other chapters in this book will avoid writing these imports.

The DataFrame Data Structure

The key data structure in Pandas is the DataFrame object. A DataFrame is basically a tabular data structure, with rows and columns. Rows have a specific index to access them, which can be any name or value. In Pandas, the columns are called Series,a special type of data, which in essence consists of a list of several values, where each value has an index. Therefore, the DataFrame data structure can be seen as a spreadsheet, but it is much more flexible. To understand how it works, let us see how to create a DataFrame from a common Python dictionary of lists. First, we will create a new cell by clicking Insert > Insert Cell Below or pressing the keys Ctrl + B. Then, we write in the following code:

In[2]:
```
data = {'year': [
          2010, 2011, 2012,
          2010, 2011, 2012,
          2010, 2011, 2012
     ],
     'team': [
          'FCBarcelona', 'FCBarcelona',
          'FCBarcelona', 'RMadrid',
          'RMadrid', 'RMadrid',
          'ValenciaCF', 'ValenciaCF',
          'ValenciaCF'
     ],
     'wins':   [30, 28, 32, 29, 32, 26, 21, 17, 19],
     'draws':  [6, 7, 4, 5, 4, 7, 8, 10, 8],
     'losses': [2, 3, 2, 4, 2, 5, 9, 11, 11]
     }
football = pd.DataFrame(data, columns = [
     'year', 'team', 'wins', 'draws', 'losses'
     ]
)
```

In this example, we use the pandas DataFrame object constructor with a dictionary of lists as argument. The value of each entry in the dictionary is the name of the column, and the lists are their values.

The DataFrame columns can be arranged at construction time by entering a key-word columnswith a list of the names of the columns ordered as we want. If the

column keyword is not present in the constructor, the columns will be arranged in alphabetical order. Now, if we execute this cell, the result will be a table like this:

Out[2]:

	year	team	wins	draws	losses
0	2010	FCBarcelona	30	6	2
1	2011	FCBarcelona	28	7	3
2	2012	FCBarcelona	32	4	2
3	2010	RMadrid	29	5	4
4	2011	RMadrid	32	4	2
5	2012	RMadrid	26	7	5
6	2010	ValenciaCF	21	8	9
7	2011	ValenciaCF	17	10	11
8	2012	ValenciaCF	19	8	11

where each entry in the dictionary is a column. The index of each row is created automatically taking the position of its elements inside the entry lists, starting from 0. Although it is very easy to create DataFrames from scratch, most of the time what we will need to do is import chunks of data into a DataFrame structure, and we willsee how to do this in later examples.

Apart from DataFrame data structure creation, Panda offers a lot of functions to manipulate them. Among other things, it offers us functions for aggregation, manipulation, and transformation of the data. In the following sections, we will introduce some of these functions.

Open Government Data Analysis Example Using Pandas

To illustrate how we can use Pandas in a simple real problem, we will start doing some basic analysis of government data. For the sake of transparency, data producedby government entities must be open, meaning that they can be freely used, reused,and distributed by anyone. An example of this is the Eurostat, which is the home ofEuropean Commission data. Eurostat's main role is to process and publish compa- rable statistical information at the European level. The data in Eurostat are providedby each member state and it is free to reuse them, for both noncommercial and commercial purposes (with some minor exceptions).

Since the amount of data in the Eurostat database is huge, in our first study we are only going to focus on data relative to indicators of educational funding by the member states. Thus, the first thing to do is to retrieve such data from Eurostat. Since open data have to be delivered in a plain text format, CSV (or any other delimiter-separated value) formats are commonly used to store tabular data. In a delimiter-separated value file, each line is a data record and each record consist- s of one or more fields, separated by the delimiter character (usually a comma). Therefore, the data we will use can be found already processed at book's Github repository as educ_figdp_1_Data.csv file. Of course, it can also be download- ed as unprocessed tabular data from the Eurostat database site[13] following the path:

Tables by themes > Population and social conditions > Education and training > Education Indicators on education finance > Public expenditure on education ,

Reading

Let us start reading the data we downloaded. First of all, we have to create a new notebook called Open Government Data Analysis and open it. Then, after ensuring that the educ_figdp_1_Data.csvfile is stored in the same directoryas our notebook directory, we will write the following code to read and show the content:

In[1]:

```
edu = pd.read_csv('files/ch02/educ_figdp_1_Data.csv',
                  na_values = ':',
                  usecols = ["TIME","GEO","Value"])
edu
```

Out[1]:

	TIME	GEO	Value
0	2000	European Union ...	NaN
1	2001	European Union ...	NaN
2	2002	European Union ...	5.00
3	2003	European Union ...	5.03
...	...	...	...
382	2010	Finland	6.85
383	2011	Finland	6.76

384 rows × 5 columns

The way to read CSV (or any other separated value, providing the separator character) files in Pandas is by calling the read_csv method. Besides the nameof the file, we add the na_values key argument to this method along with the character that represents “non available data” in the file. Normally, CSV files have aheader with the names of the columns. If this is the case, we can use the usecols parameter to select which columns in the file will be used.

In this case, the DataFrame resulting from reading our data is stored in edu. The output of the execution shows that the edu DataFrame size is 384 rows × 3 columns. Since the DataFrame is too large to be fully displayed, three dots appear in the middle of each row.

Beside this, Pandas also has functions for reading files with formats such as Excel, HDF5, tabulated files, or even the content from the clipboard (read_excel(), read_hdf(), read_table(), read_clipboard()). Whichever function we use, the result of reading a file is stored as a DataFrame structure.

To see how the data looks, we can use the head() method, which shows just the first five rows. If we use a number as an argument to this method, this will be the number of rows that will be listed:

.

In [2]:
```
edu.head()
```

Out[2]:

	TIME	GEO	Value
0	2000	European Union ...	NaN
1	2001	European Union ...	NaN
2	2002	European Union ...	5.00
3	2003	European Union ...	5.03
4	2004	European Union ...	4.95

Similarly, it exists the tail()method, which returns the last five rows by default.

In [3]:
```
edu.tail()
```

Out[3]:

379	2007	Finland	5.90
380	2008	Finland	6.10
381	2009	Finland	6.81
382	2010	Finland	6.85
383	2011	Finland	6.76

If we want to know the names of the columns or the names of the indexes, we can use the DataFrame attributes columns and index respectively. The names of the columns or indexes can be changed by assigning a new list of the same length to these attributes. The values of any DataFrame can be retrieved as a Python array bycalling its valuesattribute.

If we just want quick statistical information on all the numeric columns in a DataFrame, we can use the function describe(). The result shows the count, the mean, the standard deviation, the minimum and maximum, and the percentiles, by default, the 25th, 50th, and 75th, for all the values in each column or series.

In [4]:
```
edu.describe()
```

Out[4]:

	TIME	Value
count	384.000000	361.000000
mean	2005.500000	5.203989
std	3.456556	1.021694
min	2000.000000	2.880000
25%	2002.750000	4.620000
50%	2005.500000	5.060000
75%	2008.250000	5.660000
max	2011.000000	8.810000

Name: Value, dtype: float64

Selecting Data

If we want to select a subset of data from a DataFrame, it is necessary to indicate this subset using square brackets ([]) after the DataFrame. The subset can be specified in several ways. If we want to select only one column from a DataFrame, we only need to put its name between the square brackets. The result will be a Series data structure, not a DataFrame, because only one column is retrieved.

In[5]:
```
edu['Value']
```

Out[5]:
```
0      NaN
1      NaN
2     5.00
3     5.03
4     4.95
...    ...
380   6.10
381   6.81
382   6.85
383   6.76
Name: Value, dtype: float64
```

If we want to select a subset of rows from a DataFrame, we can do so by indicating a range of rows separated by a colon (:) inside the square brackets. This is commonly known as a *slice* of rows:

In [6]:
```
edu[10:14]
```

Out[6]:

	TIME	GEO	Value
10	2010	European Union (28 countries)	5.41
11	2011	European Union (28 countries)	5.25
12	2000	European Union (27 countries)	4.91
13	2001	European Union (27 countries)	4.99

This instruction returns the slice of rows from the 10th to the 13th position. Note that the slice does not use the index labels as references, but the position. In this case, the labels of the rows simply coincide with the position of the rows.

If we want to select a subset of columns and rows using the labels as our references instead of the positions, we can use ixindexing:

In[7]:
```
edu.ix[90:94, ['TIME','GEO']]
```

Out[7]:

	TIME	GEO
90	2006	Belgium
91	2007	Belgium
92	2008	Belgium
93	2009	Belgium
94	2010	Belgium

This returns all the rows between the indexes specified in the slice before the comma, and the columns specified as a list after the comma. In this case, ixreferences the index labels, which means that ix does not return the 90th to 94th rows, but it returns all the rows between the row labeled 90 and the row labeled 94; thus if the index 100 is placed between the rows labeled as 90 and 94, this row would also bereturned.

Filtering Data

Another way to select a subset of data is by applying Boolean indexing. This indexing is commonly known as a *filter*. For instance, if we want to filter those values less than or equal to 6.5, we can do it like this:

In [8]:
```
edu[edu['Value'] > 6.5].tail()
```

Out[8]:

	TIME	GEO	Value
218	2002	Cyprus	6.60
281	2005	Malta	6.58
94	2010	Belgium	6.58
93	2009	Belgium	6.57
95	2011	Belgium	6.55

Boolean indexing uses the result of a Boolean operation over the data, returninga mask with True or False for each row. The rows marked True in the mask will be selected. In the previous example, the Boolean operation edu['Value'] >produces a Boolean mask. When an element in the "Value" column is greaterthan 6.5, the corresponding value in the mask is set to True, otherwise it is set to False. Then, when this mask is applied as an index in edu[edu['Value'] > 6.5], the result is a filtered DataFrame containing only rows with values higher than 6.5. Of course, any of the usual Boolean operators can be used for filtering: <(less than),<= (less than or equal to), > (greater than), >= (greater than or equal to), = (equal to), and != (not equal to).

Filtering Missing Values

Pandasuses the special value NaN(not a number) to represent missing values. InPython, NaNis

Table 2.1 List of most common aggregation functions

Function	Description
count()	Number of non-null observations
sum()	Sum of values
mean()	Mean of values
median()	Arithmetic median of values
min()	Minimum
max()	Maximum
prod()	Product of values
std()	Unbiased standard deviation
var()	Unbiased variance

one of their results ends in an undefined value. A subtle feature of NaN values is that two NaN are never equal. Because of this, the only safe way to tell whether a value is missing in a DataFrame is by using the isnull() function. Indeed, this function can be used to filter rows with missing values:

In [9]:
```
edu[edu["Value"].isnull()].head()
```

Out[9]:

	TIME	GEO	Value
0	2000	European Union (28 countries)	NaN
1	2001	European Union (28 countries)	NaN
36	2000	Euro area (18 countries)	NaN
37	2001	Euro area (18 countries)	NaN
48	2000	Euro area (17 countries)	NaN

Manipulating Data

Once we know how to select the desired data, the next thing we need to know is howto manipulate data. One of the most straightforward things we can do is to operate with columns or rows using aggregation functions. Table 2.1 shows a list of the most common aggregation functions. The result of all these functions applied to a row orcolumn is always a number. Meanwhile, if a function is applied to a DataFrame or aselection of rows and columns, then you can specify if the function should be appliedto the rows for each column (setting the axis=0keyword on the invocation of thefunction), or it should be applied on the columns for each row (setting the axis=1keyword on the invocation of the function).

In [10]:
```
edu.max(axis = 0)
```

Out[10]:
```
TIME      2011
GEO      Spain
Value     8.81
dtype: object
```

Note that these are functions specific to Pandas, not the generic Python functions. There are differences in their implementation. In Python, NaN values propagate through all operations without raising an exception. In contrast, Pandas operations exclude NaN values representing missing data. For example, the pandas max function excludes NaN values, thus they are interpreted as missing values, while the standard Python max function will take the mathematical interpretation of NaN and return it as the maximum:

In [11]:
```
print "Pandas max function:", edu['Value'].max()
print "Python max function:", max(edu['Value'])
```

Out[11]:
```
Pandas max function: 8.81
Python max function:
nan
```

Beside these aggregation functions, we can apply operations over all the values in rows, columns or a selection of both. The rule of thumb is that an operation between columns means that it is applied to each row in that column and an operation between rows means that it is applied to each column in that row. For example we can apply any binary arithmetical operation (+,-,*,/) to an entire row:

In [12]:
```
s = edu["Value"]/100
s.head()
```

Out[12]:
```
0      NaN
1      NaN
2    0.0500
3    0.0503
4    0.0495
Name: Value, dtype: float64
```

However, we can apply any function to a DataFrame or Series just setting its name as argument of the apply method. For example, in the following code, we apply the sqrt function from the NumPy library to perform the square root of each value in the Value column.

In [13]:
```
s = edu["Value"].apply(np.sqrt)
s.head()
```

Out[13]:
```
0      NaN
1      NaN
2    2.236068
3    2.242766
4    2.224860
Name: Value, dtype: float64
```

If we need to design a specific function to apply it, we can write an in-line function, commonly known as a λ-function. A λ-function is a function without a name. It is only necessary to specify the parameters it receives, between the lambda keyword and the colon (:). In the next example, only one parameter is needed, which will bethe value of each element in the Value column. The value the function returns will be the square of that value.

In [14]:
```
s = edu["Value"].apply(lambda d: d**2)
s.head()
```

Out[14]:
```
0      NaN
1      NaN
2    25.0000
3    25.3009
4    24.5025
Name: Value, dtype: float64
```

Another basic manipulation operation is to set new values in our DataFrame. This can be done directly using the assign operator (=) over a DataFrame. For example, to add a new column to a DataFrame, we can assign a Series to a selection of a column that does not exist. This will produce a new column in the DataFrame after all the others. You must be aware that if a column with the same name already exists, the previous values will be overwritten. In the following example, we assign the Seriesthat results from dividing the Value column by the maximum value in the same column to a new column named ValueNorm.

In [15]:
```
edu['ValueNorm'] = edu['Value']/edu['Value'].max()
edu.tail()
```

Out[15]:

	TIME	GEO	Value	ValueNorm
379	2007	Finland	5.90	0.669694
380	2008	Finland	6.10	0.692395
381	2009	Finland	6.81	0.772985
382	2010	Finland	6.85	0.777526
383	2011	Finland	6.76	0.767310

Now, if we want to remove this column from the DataFrame, we can use the drop function; this removes the indicated rows if axis=0, or the indicated columns if axis=1. In Pandas, all the functions that change the contents of a DataFrame, such as the drop function, will normally return a copy of the modified data, instead of overwriting the DataFrame. Therefore, the original DataFrame is kept. If you do notwant to keep the old values, you can set the keyword inplaceto True. By default, this keyword is set to False, meaning that a copy of the data is returned.

In [16]:
```
edu.drop('ValueNorm', axis = 1, inplace = True)
edu.head()
```

Out[16]:

	TIME	GEO	Value
0	2000	European Union (28 countries)	NaN
1	2001	European Union (28 countries)	NaN
2	2002	European Union (28 countries)	5
3	2003	European Union (28 countries)	5.03
4	2004	European Union (28 countries)	4.95

Instead, if what we want to do is to insert a new row at the bottom of the DataFrame,we can use the Pandas appendfunction. This function receives as argumentthe new row, which is represented as a dictionary where the keys are the nameof the columns and the values are the associated value. You must be aware to setting the ignore_index flag in the append method to True, otherwise the index 0is given to this new row, which will produce an error if it already

In[17]:
```
edu = edu.append({"TIME": 2000,"Value": 5.00,"GEO": 'a'},
                 ignore_index = True)
edu.tail()
```

Out[17]:

	TIME	GEO	Value
380	2008	Finland	6.1
381	2009	Finland	6.81
382	2010	Finland	6.85
383	2011	Finland	6.76
384	2000	a	5

Finally, if we want to remove this row, we need to use the drop function again. Now we have to set the axis to 0, and specify the index of the row we want to remove. Since we want to remove the last row, we can use the max function over the indexesto determine which row is.

In[18]:
```
edu.drop(max(edu.index), axis = 0, inplace = True)
edu.tail()
```

Out[18]:

	TIME	GEO	Value
379	2007	Finland	5.9
380	2008	Finland	6.1
381	2009	Finland	6.81
382	2010	Finland	6.85
383	2011	Finland	6.76

The drop()function is also used to remove missing values by applying it overthe result of the isnull() function. This has a similar effect to filtering the NaN values, as we explained above, but here the difference is that a copy of the DataFramewithout the NaNvalues is returned, instead of a view.

In[19]:
```
eduDrop = edu.drop(edu["Value"].isnull(), axis = 0)
eduDrop.head()
```

Out[19]:

	TIME	GEO	Value
2	2002	European Union (28 countries)	5.00
3	2003	European Union (28 countries)	5.03
4	2004	European Union (28 countries)	4.95
5	2005	European Union (28 countries)	4.92
6	2006	European Union (28 countries)	4.91

To remove NaN values, instead of the generic drop function, we can use the specificdropna() function. If we want to erase any row that contains an NaN value, we have to set the how keyword to any. To restrict it to a subset of columns, we can specify it using the subset keyword. As we can see below, the result will be the same as using the dropfunction:

In[20]:

```
eduDrop = edu.dropna(how = 'any', subset = ["Value"])
eduDrop.head()
```

Out[20]:

	TIME	GEO	Value
2	2002	European Union (28 countries)	5.00
3	2003	European Union (28 countries)	5.03
4	2004	European Union (28 countries)	4.95
5	2005	European Union (28 countries)	4.92
6	2006	European Union (28 countries)	4.91

If, instead of removing the rows containing NaN, we want to fill them with another value, then we can use the fillna() method, specifying which value has to be used. If we want to fill only some specific columns, we have to set as argument to the fillna() function a dictionary with the name of the columns as the key and which character to be used for filling as the value.

In[21]:

```
eduFilled = edu.fillna(value = {"Value": 0})
eduFilled.head()
```

Out[21]:

	TIME	GEO	Value
0	2000	European Union (28 countries)	0.00
1	2001	European Union (28 countries)	0.00
2	2002	European Union (28 countries)	5.00
3	2003	European Union (28 countries)	4.95
4	2004	European Union (28 countries)	4.95

Sorting

Another important functionality we will need when inspecting our data is to sort bycolumns. We can sort a DataFrame using any column, using the sortfunction. If we want to see the first five rows of data sorted in descending order (i.e., from the largest to the smallest values) and using the Value column, then we just need to do this:

In [22]:
```
edu.sort_values(by = 'Value', ascending = False,
                inplace = True)
edu.head()
```

Out[22]:

	TIME	GEO	Value
130	2010	Denmark	8.81
131	2011	Denmark	8.75
129	2009	Denmark	8.74
121	2001	Denmark	8.44
122	2002	Denmark	8.44

Note that the inplace keyword means that the DataFrame will be overwritten, and hence no new DataFrame is returned. If instead of ascending = False we use ascending = True, the values are sorted in ascending order (i.e., from thesmallest to the largest values).

If we want to return to the original order, we can sort by an index using the sort_indexfunction and specifying axis=0:

In [23]:
```
edu.sort_index(axis = 0, ascending = True, inplace = True)
edu.head()
```

Out[23]:

	TIME	GEO	Value
0	2000	European Union ...	NaN
1	2001	European Union ...	NaN
2	2002	European Union ...	5.00
3	2003	European Union ...	5.03
4	2004	European Union ...	4.95

Grouping Data

Another very useful way to inspect data is to group it according to some criteria. For instance, in our example it would be nice to group all the data by country, regardlessof the year. Pandas has the groupby function that allows us to do exactly this. The value returned by this function is a special grouped DataFrame. To have a proper DataFrame as a result, it is necessary to apply an aggregation function. Thus, this function will be applied to all the values in the same group.

For example, in our case, if we want a DataFrame showing the mean of the valuesfor each country over all the years, we can obtain it by grouping according to country and using the mean function as the aggregation method for each group. The result would be a DataFrame with countries as indexes and the mean values as

In [24]:
```
group = edu[["GEO", "Value"]].groupby('GEO').mean()
group.head()
```

Out[24]:

	Value
GEO	
Austria	5.618333
Belgium	6.189091
Bulgaria	4.093333
Cyprus	7.023333
Czech Republic	4.16833

Rearranging Data

Up until now, our indexes have been just a numeration of rows without much meaning. We can transform the arrangement of our data, redistributing the indexes and columns for better manipulation of our data, which normally leads to better performance. We can rearrange our data using the pivot_table function. Here, we can specify which columns will be the new indexes, the new values, and the new columns.

For example, imagine that we want to transform our DataFrame to a spreadsheet-like structure with the country names as the index, while the columns will be the years starting from 2006 and the values will be the previous Value column. To do this, first we need to filter out the data and then pivot it in

In [25]:
```
filtered_data = edu[edu["TIME"] > 2005]
pivedu = pd.pivot_table(filtered_data, values = 'Value',
                        index = ['GEO'],
                        columns = ['TIME'])
pivedu.head()
```

Out[25]:

TIME	2006	2007	2008	2009	2010	2011
GEO						
Austria	5.40	5.33	5.47	5.98	5.91	5.80
Belgium	5.98	6.00	6.43	6.57	6.58	6.55
Bulgaria	4.04	3.88	4.44	4.58	4.10	3.82
Cyprus	7.02	6.95	7.45	7.98	7.92	7.87
Czech Republic	4.42	4.05	3.92	4.36	4.25	4.51

Now we can use the new index to select specific rows by label, using the ix operator:

In [26]:
```
pivedu.ix[['Spain','Portugal'], [2006,2011]]
```

Out[26]:

TIME	2006	2011
GEO		
Spain	4.26	4.82
Portugal	5.07	5.27

Pivot also offers the option of providing an argument aggr_function that allows us to perform an aggregation function between the values if there is more

than one value for the given row and column after the transformation. As usual, you can design any custom function you want, just giving its name or using a λ-function.

Ranking Data

Another useful visualization feature is to rank data. For example, we would like to know how each country is ranked by year. To see this, we will use the pandas rank function. But first, we need to clean up our previous pivoted table a bit so that it only has real countries with real data. To do this, first we drop the Euro area entries andshorten the Germany name entry, using the renamefunction and then we drop allthe rows containing any NaN, using the dropnafunction.

Now we can perform the ranking using the rank function. Note here that the parameter ascending=False makes the ranking go from the highest values tothe lowest values. The Pandas rank function supports different tie-breaking methods, specified with the method parameter. In our case, we use the first method, in which ranks are assigned in the order they appear in the array, avoiding gaps between ranking.

In [27]:
```
pivedu = pivedu.drop([
                        'Euro area (13 countries)','Euro area (
                        15 countries)','Euro area (17 countries
                        )','Euro area (18 countries)',
                        'European Union (25 countries)','European
                        Union (27 countries)','European Union (28
                        countries)'
                       ],
                       axis = 0)
pivedu = pivedu.rename(index ={'Germany (until 1990 former territoryof the FRG)': 'Germany'})
pivedu = pivedu.dropna()
pivedu.rank(ascending = False, method = 'first').head()
```

Out[27]:

TIME	2006	2007	2008	2009	2010	2011
GEO						
Austria	10	7	11	7	8	8
Belgium	5	4	3	4	5	5
Bulgaria	21	21	20	20	22	21
Cyprus	2	2	2	2	2	3
Czech Republic	19	20	21	21	20	18

If we want to make a global ranking taking into account all the years, we can sum up all the columns and rank the result. Then we can sort the resulting values toretrieve the top five countries for the last 6 years, in this way:

In [28]:
```
totalSum = pivedu.sum(axis = 1)
totalSum.rank(ascending = False, method = 'dense')
         .sort_values().head()
```

Out[28]:

GEO	
Denmark	1
Cyprus	2
Finland	3
Malta	4
Belgium	5

dtype: float64

Notice that the method keyword argument in the in the rank function specifies how items that compare equals receive ranking. In the case of dense, items that compare equals receive the same ranking number, and the next not equal item receives the immediately following ranking number.

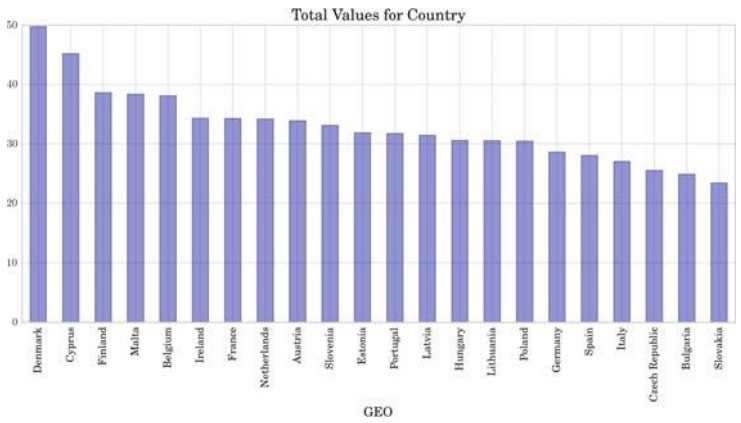
Plotting

Pandas DataFrames and Series can be plotted using the plot function, which uses the library for graphics Matplotlib. For example, if we want to plot the accumulated values for each country over the last 6 years, we can take the Series obtained in theprevious example and plot it directly by calling the plot function as shown in the next cell:

In [29]:

```
totalSum = pivedu.sum(axis = 1)
                 .sort_values(ascending = False)
totalSum.plot(kind = 'bar', style = 'b', alpha = 0.4,
              title = "Total Values for Country")
```

Out[29]:



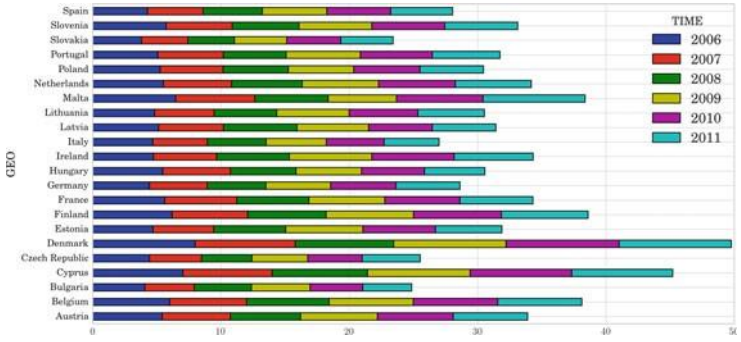
Note that if we want the bars ordered from the highest to the lowest value, we need to sort the values in the Series first. The parameter kind used in the plot function defines which kind of graphic will be used. In our case, a bar graph. The parameter stylerefers to the style properties of the graphic, in our case, the color

of bars is set to b (blue). The alpha channel can be modified adding a keyword parameter alpha with a percentage, producing a more translucent plot. Finally, using the titlekeyword the name of the graphic can be set.

It is also possible to plot a DataFrame directly. In this case, each column is treated as a separated Series. For example, instead of printing the accumulated value over the years, we can plot the value for each year.

In [30]:
```
my_colors = ['b', 'r', 'g', 'y', 'm', 'c']
ax = pivedu.plot(kind = 'barh',
                 stacked = True,
                 color = my_colors)
ax.legend(loc = 'center left', bbox_to_anchor = (1, .5))
```

Out[30]:



In this case, we have used a horizontal bar graph (kind='barh') stacking all the years in the same country bar. This can be done by setting the parameter stacked to True. The number of default colors in a plot is only 5, thus if you have more than 5 Series to show, you need to specify more colors or otherwise the same set ofcolors will be used again. We can set a new set of colors using the keyword color with a list of colors. Basic colors have a single-character code assigned to each, for example, "b" is for blue, "r" for red, "g" for green, "y" for yellow, "m" for magenta, and "c" for cyan. When several Series are shown in a plot, a legend is created for identifying each one. The name for each Series is the name of the column in the DataFrame. By default, the legend goes inside the plot area. If we want to change this, we can use the legend function of the axis object (this is the object returned when the plot function is called). By using the loc keyword, we can set the relative position of the legend with respect to the plot. It can be a combination of right or left and upper, lower, or center. With bbox_to_anchor we can set an absolute position with respect to the plot, allowing us to put the legend outside the graph.

UNIT-2

Descriptive statistics, data preparation. Exploratory Data Analysis data summarization, data distribution, measuring asymmetry. Sample and estimated mean, variance and standard score. Statistical Inference frequency approach, variability of estimates, hypothesis testing using confidence intervals, using p-values

DescriptiveStatistics

Descriptive statistics helps to simplify large amounts of data in a sensible way. In contrast to inferential statistics, which will be introduced in a later chapter, in descriptive statistics we do not draw conclusions beyond the data we are analyzing; neither do we reach any conclusions regarding hypotheses we may make. We do nottry to infer characteristics of the "population" (see below) of the data, but claim to present quantitative descriptions of it in a manageable form. It is simply a way to describe the data.

Statistics, and in particular descriptive statistics, is based on two main concepts:

- a *population* is a collection of objects, items ("units") about which information issought;
- a *sample* is a part of the population that is observed.

Descriptive statistics applies the concepts, measures, and terms that are used to describe the basic features of the samples in a study. These procedures are essential to provide summaries about the samples as an approximation of the population. Together with simple graphics, they form the basis of every quantitative analysis ofdata. In order to describe the sample data and to be able to infer any conclusion, weshould go through several steps:

1. *Data preparation*: Given a specific example, we need to prepare the data forgenerating statistically valid descriptions.
2. *Descriptive statistics*: This generates different statistics to describe and summa-rize the data concisely and evaluate different ways to visualize them.

Data Preparation

One of the first tasks when analyzing data is to collect and prepare the data in a format appropriate for analysis of the samples. The most common steps for data preparationinvolve the following operations.

1. *Obtaining the data:* Data can be read directly from a file or they might be obtained by scraping the web.
2. *Parsing the data:* The right parsing procedure depends on what format the dataare in: plain text, fixed columns, CSV, XML, HTML, etc.
3. *Cleaning the data:* Survey responses and other data files are almost always incomplete. Sometimes, there are multiple codes for things such as, not asked, didnot know, and declined to answer. And there are almost always errors. A simplestrategy is to remove or ignore incomplete records.
4. *Building data structures:* Once you read the data, it is necessary to store them ina data structure that lends itself to the analysis we are interested in. If the data fitinto the memory, building a data structure is usually the way to go. If not, usually a database is built, which is an out-of-memory data structure. Most databases provide a mapping from keys to values, so they serve as dictionaries.

The Adult Example

Let us consider a public database called the "Adult" dataset, hosted on the UCI's Machine Learning Repository.[1] It contains approximately 32,000 observations concerning different financial parameters related to the US population: age, sex, marital(marital status of the individual), country, income (Boolean variable: whether the per- son makes more than $50,000 per annum), education (the highest level of educationachieved by the individual), occupation, capital gain, etc.

We will show that we can explore the data by asking questions like: "Are men more likely to become high-income professionals than women, i.e., to receive an income of over $50,000 per annum?"

First, let us read the data:

In[1]:

```
file = open('files/ch03/adult.data', 'r')
def chr_int(a):
    if a.isdigit(): return int(a)
    else: return 0

data = []
for line in file:
     data1 = line.split(', ')
     if len(data1) == 15:
         data.append([chr_int(data1[0]), data1[1],
                      chr_int(data1[2]), data1[3],
                      chr_int(data1[4]), data1[5],
                      data1[6], data1[7], data1[8],
                      data1[9], chr_int(data1[10]),
                      chr_int(data1[11]),
                      chr_int(data1[12]),
                      data1[13], data1[14]
                     ])
```

Checking the data, we obtain:

In[2]:

```
print data[1:2]
```

Out[2]: [[50, 'Self-emp-not-inc', 83311, 'Bachelors', 13,
'Married-civ-spouse', 'Exec-managerial', 'Husband', 'White','Male', 0, 0, 13, 'United-States', ' <= 50K']]

One of the easiest ways to manage data in Python is by using the DataFrame structure, defined in the *Pandas* library, which is a two-dimensional, size-mutable,potentially heterogeneous tabular data structure with labeled axes:

In[3]:

```
df = pd.DataFrame(data)
df.columns = [
    'age', 'type_employer', 'fnlwgt',
    'education', 'education_num', 'marital',
    'occupation',' relationship', 'race',
    'sex', 'capital_gain', 'capital_loss',
    'hr_per_week', 'country', 'income'
    ]
```

The command shapegives exactly the number of data samples (in rows, in this case) and features (in columns):

In[4]:

```
df.shape
```

Out[4]: (32561, 15)

Thus, we can see that our dataset contains 32,561 data records with 15 featureseach. Let us count the number of items per country:

In[5]:
```
counts = df.groupby('country').size()
print counts.head()
```

Out[5]: country
? 583
Cambodia 19
Vietnam 67
Yugoslavia 16

The first row shows the number of samples with unknown country, followed bythe number of samples corresponding to the first countries in the dataset.

Let us split people according to their gender into two groups: men and women.

In[6]:
```
ml = df[(df.sex == 'Male')]
```

If we focus on high-income professionals separated by sex, we can do:

In[7]:
```
ml1 = df[(df.sex == 'Male') & (df.income =='>50K\n')
    ]
fm = df[(df.sex == 'Female')]
fml = df[(df.sex == 'Female') & (df.income =='>50K\n
    ')]
```

Exploratory Data Analysis

The data that come from performing a particular measurement on all the subjects in a sample represent our observations for a single characteristic like country, age, education, etc. These measurements and categories represent a *sample distribution* of the variable, which in turn approximately represents the *population distribution* of the variable. One of the main goals of exploratory data analysis is to visualize and summarize the sample distribution, thereby allowing us to make tentative assumptions about the population distribution.

Summarizing the Data

The data in general can be categorical or quantitative. For categorical data, a simple tabulation of the frequency of each category is the best non-graphical exploration for data analysis. For example, we can ask ourselves what is the proportion of high-income professionals in our database:

In[8]:

```
df1 = df[(df.income=='>50K\n')]
print 'The rate of people with high income is: ',
      int(len(df1)/float(len(df))*100), '%.'
print 'The rate of men with high income is: ',
      int(len(ml1)/float(len(ml))*100), '%.'
print 'The rate of women with high income is: ',
      int(len(fm1)/float(len(fm))*100), '%.'
```

Out[8]: The rate of people with high income is: 24 %.
The rate of men with high income is: 30 %. The rate of women with high income is: 10 %.

Given a quantitative variable, exploratory data analysis is a way to make prelim-inary assessments about the population distribution of the variable using the data of the observed samples. The characteristics of the population distribution of a quanti- tative variable are its *mean*, *deviation*, *histograms*, *outliers*, etc. Our observed data represent just a finite set of samples of an often infinite number of possible samples. The characteristics of our randomly observed samples are interesting only to the degree that they represent the population of the data they came from.

Mean

One of the first measurements we use to have a look at the data is to obtain *samplestatistics* from the data, such as the sample mean [1]. Given a sample of n values, $\{x_i, i = 1, \ldots, n\}$, the *mean*, μ, is the sum of the values divided by the number of values,[2] in other words:

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i . \tag{3.1}$$

The terms mean and *average* are often used interchangeably. In fact, the maindistinction between them is that the mean of a sample is the summary statistic com-puted by Eq. (3.1), while an average is not strictly defined and could be one of manysummary statistics that can be chosen to describe the central tendency of a sample.

In our case, we can consider what the average age of men and women samples inour dataset would be in terms of their mean:

Descriptive Statistics

In[9]:

```
print 'The average age of men is: ',
      ml['age'].mean()
print 'The average age of women is: ',
      fm['age'].mean()

print 'The average age of high-income men is: ',
      ml1['age'].mean()
print 'The average age of high-income women is: ',
      fm1['age'].mean()
```

Out[9]:

```
The average age of men is: 39.4335474989 The average age of
women is: 36.8582304336
The average age of high-income men is: 44.6257880516
The average age of high-income women is: 42.1255301103
```

This difference in the sample means can be considered initial evidence that thereare differences between men and women with high income!

Comment: Later, we will work with both concepts: the population mean and thesample mean. We should not confuse them! The first is the mean of samples takenfrom the population; the second, the mean of the whole population.

Sample Variance

The mean is not usually a sufficient descriptor of the data. We can go further by knowing two numbers: mean and *variance*. The variance σ^2 describes the spread ofthe data and it is defined as follows:

$$\sigma^2 = \frac{1}{n} \sum_{i} (x_i - \mu)^2. \tag{3.2}$$

The term $(x_i - \mu)$ is called the *deviation* from the mean, so the variance is the mean squared deviation. The square root of the variance, σ, is called the *standard deviation*. We consider the standard deviation, because the variance is hard to interpret (e.g., ifthe units are grams, the variance is in grams squared).

Let us compute the mean and the variance of hours per week men and women inour dataset work:

In[10]:

```
ml_mu = ml['age'].mean()
fm_mu = fm['age'].mean()
ml_var = ml['age'].var()
fm_var = fm['age'].var()
ml_std = ml['age'].std()
fm_std = fm['age'].std()
print 'Statistics of age for men: mu:',
```

Out[10]: Statistics of age for men: mu: 39.4335474989 var: 178.773751745std: 13.3706301925
Statistics of age for women: mu: 36.8582304336 var:196.383706395 std: 14.0136970994

We can see that the mean number of hours worked per week by women is significantly lesser than that worked by men, but with much higher variance and standarddeviation.

Sample Median

The mean of the samples is a good descriptor, but it has an important drawback: what will happen if in the sample set there is an error with a value very different from the rest? For example, considering hours worked per week, it would normally be in a range between 20 and 80; but what would happen if by mistake there was a value of 1000? An item of data that is significantly different from the rest of the data is called an *outlier*. In this case, the mean, μ, will be drastically changed towards the outlier. One solution to this drawback is offered by the statistical *median*, μ_{12}, which is an order statistic giving the middle value of a sample. In this case, all the values are ordered by their magnitude and the median is defined as the value that is in themiddle of the ordered list. Hence, it is a value that is much more robust in the face of outliers.

Let us see, the median age of working men and women in our dataset and the median age of high-income men and women:

In[11]:
```
ml_median = ml['age'].median()
fm_median = fm['age'].median()
print "Median age per men and women: ",
      ml_median, fm_median

ml_median_age = ml1['age'].median()
fm_median_age = fm1['age'].median()
print "Median age per men and women with high-
    income: ",
      ml_median_age, fm_median_age
```

Out[11]: Median age per men and women: 38.0 35.0
Median age per men and women with high-income: 44.0 41.0

As expected, the median age of high-income people is higher than the whole setof working people, although the difference between men and women in both sets isthe same.

Quantiles and Percentiles

Sometimes we are interested in observing how sample data are distributed in general. In this case, we can order the samples x_i , then find the x_p so that it divides the datainto two parts, where:

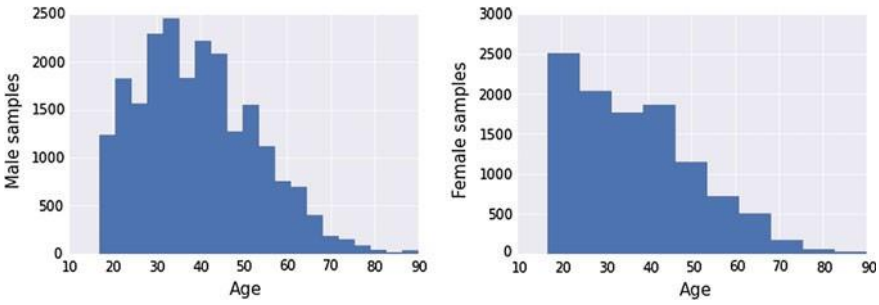


Fig. 3.1 Histogram of the age of working men (*left*) and women (*right*)

- a fraction p of the data values is less than or equal to x_p and
- the remaining fraction $(1 - p)$ is greater than x_p.

That value, x_p, is the *p-th* quantile, or the 100 p-*th* percentile. For example, a 5-number summary is defined by the values x_{min}, Q_1, Q_2, Q_3, x_{max}, where Q_1 is the 25 *p-th* percentile, Q_2 is the 50 *p-th* percentile and Q_3 is the 75 *p-th* percentile.

Data Distributions

Summarizing data by just looking at their mean, median, and variance can be dangerous: very different data can be described by the same statistics. The best thing to do is to validate the data by inspecting them. We can have a look at the data distribution, which describes how often each value appears (i.e., what is its frequency).

The most common representation of a distribution is a *histogram*, which is a graph that shows the frequency of each value. Let us show the age of working men and women separately.

In[12]:
```
ml_age = ml['age']
ml_age.hist(normed = 0, histtype = 'stepfilled',
    bins = 20)
```

In[13]:
```
fm_age = fm['age']
fm_age.hist(normed = 0, histtype = 'stepfilled',
    bins = 10)
```

The output can be seen in Fig. 3.1. If we want to compare the histograms, we canplot them overlapping in the same graphic as follows:

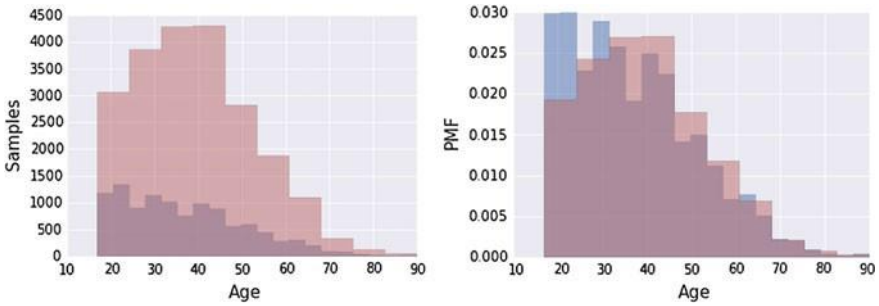


Fig. 3.2 Histogram of the age of working men (in *ochre*) and women (in *violet*) (*left*). Histogram of the age of working men (in *ochre*), women (in *blue*), and their intersection (in *violet*) after samples normalization (*right*)

In[14]:

```
import seaborn as sns
fm_age.hist(normed = 0, histtype = 'stepfilled',
            alpha = .5, bins = 20)
ml_age.hist(normed = 0, histtype = 'stepfilled',
            alpha = .5,
            color = sns.desaturate("indianred",
                .75),
            bins = 10)
```

The output can be seen in Fig. 3.2 (left). Note that we are visualizing the absolute values of the number of people in our dataset according to their age (the abscissa ofthe histogram). As a side effect, we can see that there are many more men in these conditions than women.

We can normalize the frequencies of the histogram by dividing/normalizing by n, the number of samples. The normalized histogram is called the *Probability MassFunction* (PMF).

In[15]:

```
fm_age.hist(normed = 1, histtype = 'stepfilled',
            alpha = .5, bins = 20)
ml_age.hist(normed = 1, histtype = 'stepfilled',
            alpha = .5, bins = 10,
            color = sns.desaturate("indianred",
                .75))
```

This outputs Fig. 3.2 (right), where we can observe a comparable range of individuals (men and women).

The *Cumulative Distribution Function* (CDF), or just distribution function, describes the probability that a real-valued random variable X with a given probability distribution will be found to have a value less than or equal to x. Let us show the CDF of age distribution for both men and women.

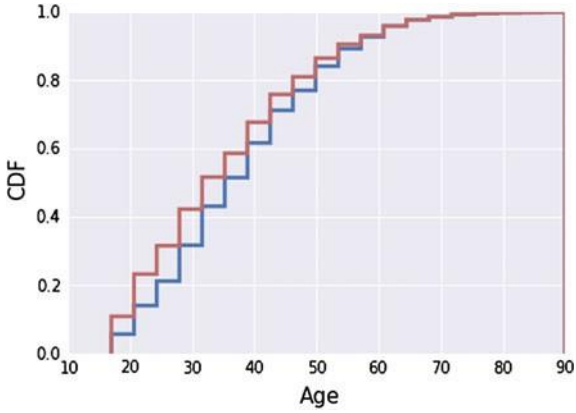


Fig. 3.3 The CDF of the ageof working male (in *blue*) and female (in *red*) samples

In[16]:

```
ml_age.hist(normed = 1, histtype = 'step',
            cumulative = True, linewidth = 3.5,
            bins = 20)
fm_age.hist(normed = 1, histtype='step',
            cumulative = True, linewidth = 3.5,
            bins = 20,
            color = sns.desaturate("indianred",
                .75))
```

The output can be seen in Fig. 3.3, which illustrates the CDF of the age distributions for both men and women.

Outlier Treatment

As mentioned before, outliers are data samples with a value that is far from the centraltendency. Different rules can be defined to detect outliers, as follows:

- Computing samples that are far from the median.
- Computing samples whose values exceed the mean by 2 or 3 standard deviations.

For example, in our case, we are interested in the age statistics of men versus women with high incomes and we can see that in our dataset, the minimum age is 17years and the maximum is 90 years. We can consider that some of these samples are due to errors or are not representable. Applying the domain knowledge, we focus onthe median age (37, in our case) up to 72 and down to 22 years old, and we considerthe rest as outliers.

In[17]:
```
df2 = df.drop(df.index[
    (df.income == '>50K\n') &
    (df['age'] > df['age'].median() + 35) &
    (df['age'] > df['age'].median() -15)
    ])
ml1_age = ml1['age']
fm1_age = fm1['age']

ml2_age = ml1_age.drop(ml1_age.index[
    (ml1_age > df['age'].median() + 35) &
    (ml1_age > df['age'].median() - 15)
    ])
fm2_age = fm1_age.drop(fm1_age.index[
    (fm1_age > df['age'].median() + 35) &
    (fm1_age > df['age'].median() - 15)
    ])
```

We can check how the mean and the median changed once the data were cleaned:

In[18]:
```
mu2ml = ml2_age.mean()
std2ml = ml2_age.std()
md2ml = ml2_age.median()
mu2fm = fm2_age.mean()
std2fm = fm2_age.std()
md2fm = fm2_age.median()

print "Men statistics:"
print "Mean:", mu2ml, "Std:", std2ml
print "Median:", md2ml
print "Min:", ml2_age.min(), "Max:", ml2_age.max()

print "Women statistics:"
print "Mean:", mu2fm, "Std:", std2fm
print "Median:", md2fm
print "Min:", fm2_age.min(), "Max:", fm2_age.max()
```

Out[18]: Men statistics: Mean: 44.3179821239 Std: 10.0197498572 Median: 44.0 Min: 19 Max: 72
Women statistics: Mean: 41.877028181 Std: 10.0364418073 Median: 41.0 Min: 19 Max: 72

Let us visualize how many outliers are removed from the whole data by:

In[19]:
```
plt.figure(figsize = (13.4, 5))
df.age[(df.income == '>50K\n')]
      .plot(alpha = .25, color = 'blue')
df2.age[(df2.income == '>50K\n')]
      .plot(alpha = .45, color = 'red')
```

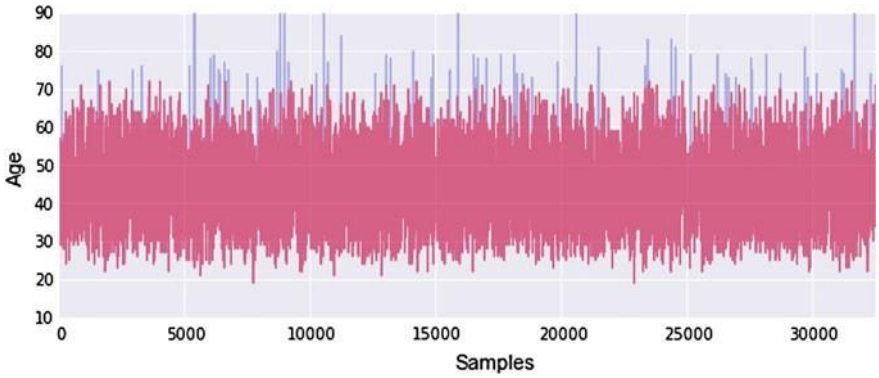


Fig. 3.4 The *red* shows the cleaned data without the considered outliers (in *blue*)

Figure 3.4 shows the outliers in blue and the rest of the data in red. Visually, wecan confirm that we removed mainly outliers from the dataset.

Next we can see that by removing the outliers, the difference between the popula-tions (men and women) actually decreased. In our case, there were more outliers inmen than women. If the difference in the mean values before removing the outliersis 2.5, after removing them it slightly decreased to 2.44:

In[20]:
```
print 'The mean difference with outliers is: %4.2f.
    '
      % (ml_age.mean() - fm_age.mean())
print 'The mean difference without outliers is:
    %4.2f.'
      % (ml2_age.mean() - fm2_age.mean())
```

Out[20]:
```
The mean difference with outliers is: 2.58.
The mean difference without outliers is: 2.44.
```

Let us observe the difference of men and women incomes in the cleaned subsetwith some more details.

In[21]:
```
countx, divisionx = np.histogram(ml2_age, normed =
    True)
county, divisiony = np.histogram(fm2_age, normed =
    True)

val = [(divisionx[i] + divisionx[i+1])/2
       for i in range(len(divisionx) - 1)]
plt.plot(val, countx - county, 'o-')
```

The results are shown in Fig. 3.5. One can see that the differences between maleand female values are slightly negative before age 42 and positive after it. Hence, women tend to be promoted (receive more than 50 K) earlier than men.

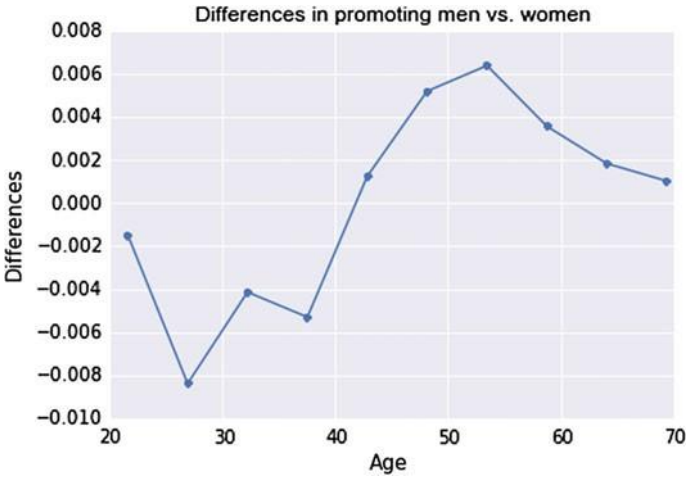


Fig. 3.5 Differences in high-income earner men versus women as a function of age

Measuring Asymmetry: Skewness and Pearson's Median Skewness Coefficient

For univariate data, the formula for *skewness* is a statistic that measures the asym-metry of the set of n data samples, x_i:

$$g_1 = \frac{1}{n}\frac{\sum_i (x_i - \mu^3)}{\sigma^3}, \tag{3.3}$$

where μ is the mean, σ is the standard deviation, and n is the number of data points.

Negative deviation indicates that the distribution "skews left" (it extends further to the left than to the right). One can easily see that the skewness for a normal distribution is zero, and any symmetric data must have a skewness of zero. Note that skewness can be affected by outliers! A simpler alternative is to look at the relationship between the mean μ and the median μ_{12}.

In[22]:

```
def skewness(x):
    res = 0
    m = x.mean()
    s = x.std()
    for i in x:
        res += (i-m) * (i-m) * (i-m)
    res /= (len(x) * s * s * s)
    return res

print "Skewness of the male population = ",
      skewness(ml2_age)
print "Skewness of the female population is = ",
      skewness(fm2_age)
```

Out[22]: Skewness of the male population = 0.266444383843 Skewness of the female population = 0.386333524913

That is, the female population is more skewed than the male, probably since mencould be most prone to retire later than women.

The **Pearson's median skewness coefficient** is a more robust alternative to the skewness coefficient and is defined as follows:

$$g_p = 3(\mu - \mu_{12})\sigma.$$

There are many other definitions for skewness that will not be discussed here. In our case, if we check the Pearson's skewness coefficient for both men and women,we can see that the difference between them actually increases:

In[23]:

```
def pearson(x):
    return 3*(x.mean() - x.median())*x.std()

print "Pearson's coefficient of the male population
    = ",
      pearson(ml2_age)
print "Pearson's coefficient of the female
   population = ",
      pearson(fm2_age)
```

Out[23]: Pearson's coefficient of the male population = 9.55830402221 Pearson's coefficient of the female population = 26.4067269073

Continuous Distribution

The distributions we have considered up to now are based on empirical observationsand thus are called *empirical distributions*. As an alternative, we may be interested in considering distributions that are defined by a continuous function and are called*continuous distributions* [2]. Remember that we defined the PMF, $f_X(x)$, of a discreterandom variable X as $f_X(x) = P(X = x)$ for all x. In the case of a continuous random variable X, we speak of the *Probability Density Function* (PDF), which

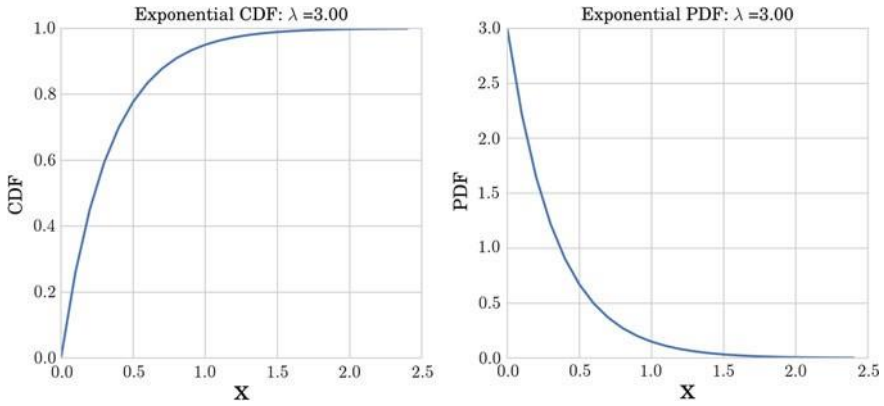


Fig. 3.6 Exponential CDF (*left*) and PDF (*right*) with $\lambda = 3.00$

is defined as $F_X(x)$ where this satisfies: $F_X(x) = \int_{-\infty}^{x} f_X(t)\delta t$ for all x. There are many continuous distributions; here, we will consider the most common ones: the exponential and the normal distributions.

The Exponential Distribution

Exponential distributions are well known since they describe the inter-arrival time between events. When the events are equally likely to occur at any time, the distri-bution of the inter-arrival time tends to an exponential distribution. The CDF and the PDF of the exponential distribution are defined by the following equations:

$$CDF(x) = 1 - e^{-\lambda x}, \qquad PDF(x) = \lambda e^{-\lambda x}.$$

The parameter λ defines the shape of the distribution. An example is given inFig. 3.6. It is easy to show that the mean of the distribution is $\frac{1}{\lambda}$, the variance is $\frac{1}{\lambda^2}$ and the median is $\frac{\ln 2}{\lambda}$.

Note that for a small number of samples, it is difficult to see that the exact empirical distribution fits a continuous distribution. The best way to observe this match is to generate samples from the continuous distribution and see if these samples match the data. As an exercise, you can consider the birthdays of a large enough group of people, sorting them and computing the inter-arrival time in days. If you plot the CDF of the inter-arrival times, you will observe the exponential distribution.

There are a lot of real-world events that can be described with this distribution, including the time until a radioactive particle decays; the time it takes before your next telephone call; and the time until default (on payment to company debt holders)in reduced-form credit risk modeling. The random variable X of the lifetime of some batteries is associated with a probability density function of the form: $PDF(x) = \frac{1}{4}e^{-\frac{x}{4}}e^{-\frac{(x-\mu)^2}{2}}$

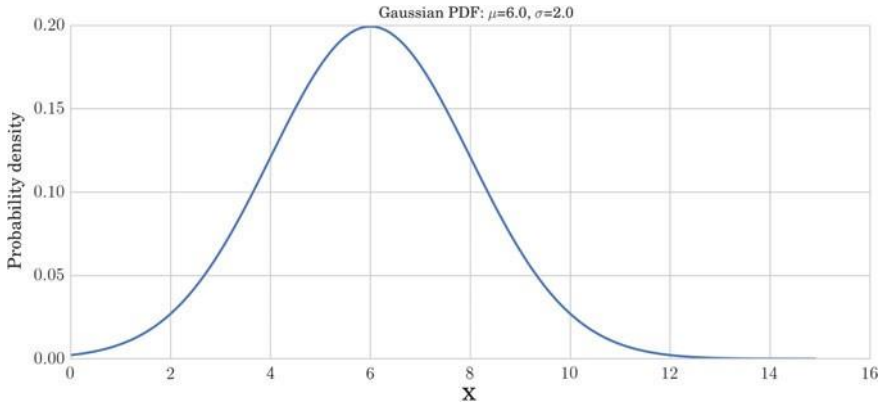


Fig. 3.7 Normal PDF with $\mu = 6$ and $\sigma = 2$

The Normal Distribution

The *normal distribution*, also called the *Gaussian distribution*, is the most common since it represents many real phenomena: economic, natural, social, and others. Some well-known examples of real phenomena with a normal distribution are as follows:

- The size of living tissue (length, height, weight).
- The length of inert appendages (hair, nails, teeth) of biological specimens.
- Different physiological measurements (e.g., blood pressure), etc.

The normal CDF has no closed-form expression and its most common represen-tation is the PDF:

$$\text{PDF}(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}.$$

The parameter σ defines the shape of the distribution. An example of the PDF ofa normal distribution with $\mu = 6$ and $\sigma = 2$ is given in Fig. 3.7.

Kernel Density

In many real problems, we may not be interested in the parameters of a particular distribution of data, but just a continuous representation of the data. In this case, we should estimate the distribution non-parametrically (i.e., making no assumptionsabout the form of the underlying distribution) using kernel density estimation. Let us imagine that we have a set of data measurements without knowing their distribution and we need to estimate the continuous representation of their distribution. In this case, we can consider a Gaussian kernel to generate the density around the data. Letus consider a set of random data generated by a bimodal normal distribution. If we consider a Gaussian kernel around the data, the sum of those kernels can give us

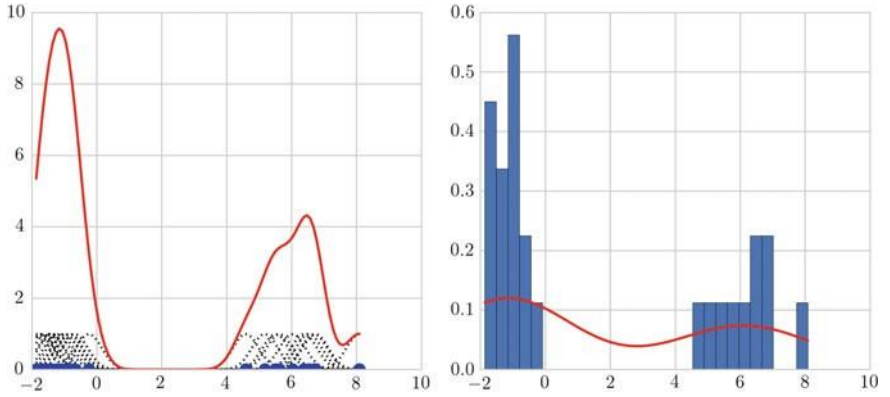


Fig. 3.8 Summed kernel functions around a random set of points (*left*) and the kernel density estimate with the optimal bandwidth (*right*) for our dataset. Random data shown in *blue*, kernel shown in *black* and summed function shown in *red*

a continuous function that when normalized would approximate the density of the distribution:

In[24]:

```
x1 = np.random.normal(-1, 0.5, 15)
x2 = np.random.normal(6, 1, 10)
y = np.r_[x1, x2] # r_ translates slice objects to
    concatenation along the first axis.
x = np.linspace(min(y), max(y), 100)

s = 0.4 # Smoothing parameter

# Calculate the kernels
kernels = np.transpose([norm.pdf(x, yi, s) for yi
    in y])
plt.plot(x, kernels, 'k:')
plt.plot(x, kernels.sum(1), 'r')
plt.plot(y, np.zeros(len(y)), 'bo', ms = 10)
```

Figure 3.8 (left) shows the result of the construction of the continuous functionfrom the kernel summarization.

In fact, the library SciPy[3] implements a Gaussian kernel density estimation that automatically chooses the appropriate bandwidth parameter for the kernel. Thus, thefinal construction of the density estimate will be obtained by:

.

In[25]:

```
from scipy.stats import kde
density = kde.gaussian_kde(y)
xgrid = np.linspace(x.min(), x.max(), 200)
plt.hist(y, bins = 28, normed = True)
plt.plot(xgrid, density(xgrid), 'r-')
```

Figure 3.8 (right) shows the result of the kernel density estimate for our example.

Estimation

An important aspect when working with statistical data is being able to use estimatesto approximate the values of unknown parameters of the dataset. In this section, we will review different kinds of estimators (estimated mean, variance, standard score,etc.).

Sample and Estimated Mean, Variance and Standard Scores

In continuation, we will deal with point estimators that are single numerical estimates of parameters of a population.

Mean

Let us assume that we know that our data are coming from a normal distribution andthe random samples drawn are as follows:

$$\{0.33, -1.76, 2.34, 0.56, 0.89\}.$$

The question is can we guess the mean μ of the distribution? One approximation isgiven by the sample mean, $\bar{x}$. This process is called *estimation* and the statistic (e.g., the sample mean) is called an *estimator*. In our case, the sample mean is 0.472, and it seems a logical choice to represent the mean of the distribution. It is not so evident ifwe add a sample with a value of 465. In this case, the sample mean will be 77.11, which does not look like the mean of the distribution. The reason is due to the fact that the last value seems to be an outlier compared to the rest of the sample. In orderto avoid this effect, we can try first to remove outliers and then to estimate the mean; or we can use the sample median as an estimator of the mean of the distribution. If there are no outliers, the sample mean x minimizes the following *mean squared error*:

$$MSE = \frac{1}{n} \sum (x - \mu)^2,$$

where n is the number of times we estimate the mean.Let us compute the MSE of a set of random data:

In[26]:
```
NTs = 200
mu = 0.0
var = 1.0
err = 0.0
NPs = 1000
for i in range(NTs):
    x = np.random.normal(mu, var, NPs)
    err += (x.mean()-mu)**2
print 'MSE: ', err/NTests
```

Out[26]: MSE: 0.00019879541147

Variance

If we ask ourselves what is the variance, σ^2, of the distribution of X, analogously we can use the sample variance as an estimator. Let us denote by σ^2 the sample varianceestimator:

$$\sigma^2 = \frac{1}{n} \sum_i (x_i - \bar{x})^2.$$

For large samples, this estimator works well, but for a small number of samplesit is biased. In those cases, a better estimator is given by:

$$\tilde{\sigma}^2 = \frac{1}{n-1} \sum_i (x_i - \bar{x})^2.$$

Standard Score

In many real problems, when we want to compare data, or estimate their correlations or some other kind of relations, we must avoid data that come in different units. For example, weight can come in kilograms or grams. Even data that come in the same units can still belong to different distributions. We need to normalize them tostandard scores. Given a dataset as a series of values, $\{x_i\}$, we convert the data to standard scores by subtracting the mean and dividing them by the standard deviation:

$$z_i = \frac{(x_i - \mu)}{\sigma}.$$

Note that this measure is dimensionless and its distribution has a mean of 0 and variance of 1. It inherits the "shape" of the dataset: if X is normally distributed, so is Z; if X is skewed, so is Z.

Covariance, and Pearson's and Spearman's Rank Correlation

Variables of data can express relations. For example, countries that tend to invest

in research also tend to invest more in education and health. This kind of relationshipis captured by the covariance.

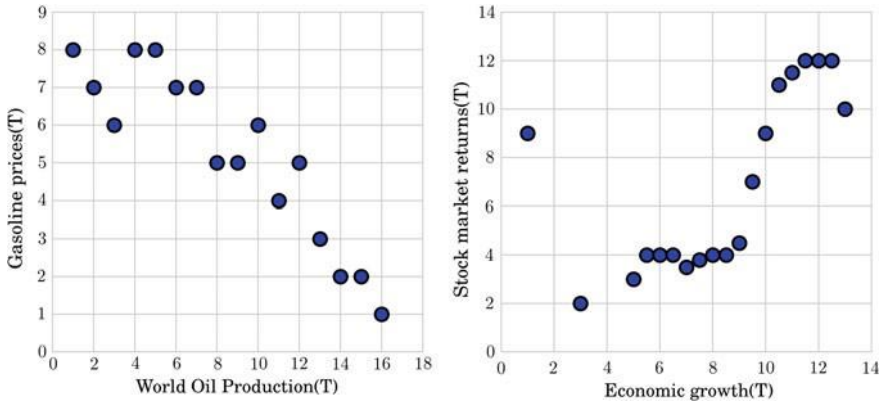


Fig. 3.9 Positive correlation between economic growth and stock market returns worldwide (*left*). Negative correlation between the world oil production and gasoline prices worldwide (*right*)

Covariance

When two variables share the same tendency, we speak about *covariance*. Let us consider two series, $\{x_i\}$ and $\{y_i\}$. Let us center the data with respect to their mean: $dx_i = x_i - \mu_X$ and $dy_i = y_i - \mu_Y$. It is easy to show that when $\{x_i\}$ and $\{y_i\}$ vary together, their deviations tend to have the same sign. The covariance is defined as the mean of the following products:

$$Cov(X,Y) = \frac{1}{n}\sum_{i=1}^{n} dx_i\, dy_i ,$$

where n is the length of both sets. Still, the covariance itself is hard to interpret.

Correlation and the Pearson's Correlation

If we normalize the data with respect to their deviation, that leads to the standardscores; and then multiplying them, we get:

$$\rho_i = \frac{x_i - \mu_X}{\sigma_X}\,\frac{y_i - \mu_Y}{\sigma_Y} .$$

The mean of this product is $\rho = \frac{1}{n}\sum_{i=1}^{n} \rho_i$. Equivalently, we can rewrite ρ in terms of the covariance, and thus obtain the *Pearson's correlation*:

$$\rho = \frac{Cov(X,Y)}{\sigma_X \sigma_Y} .$$

Note that the Pearson's correlation is always between -1 and $+1$, where themagnitude depends on the degree of correlation. If the Pearson's correlation is -1 (or1), it means that the variables are perfectly correlated (positively or negatively) (see Fig. 3.9). This means that one variable can predict the other very well. However,

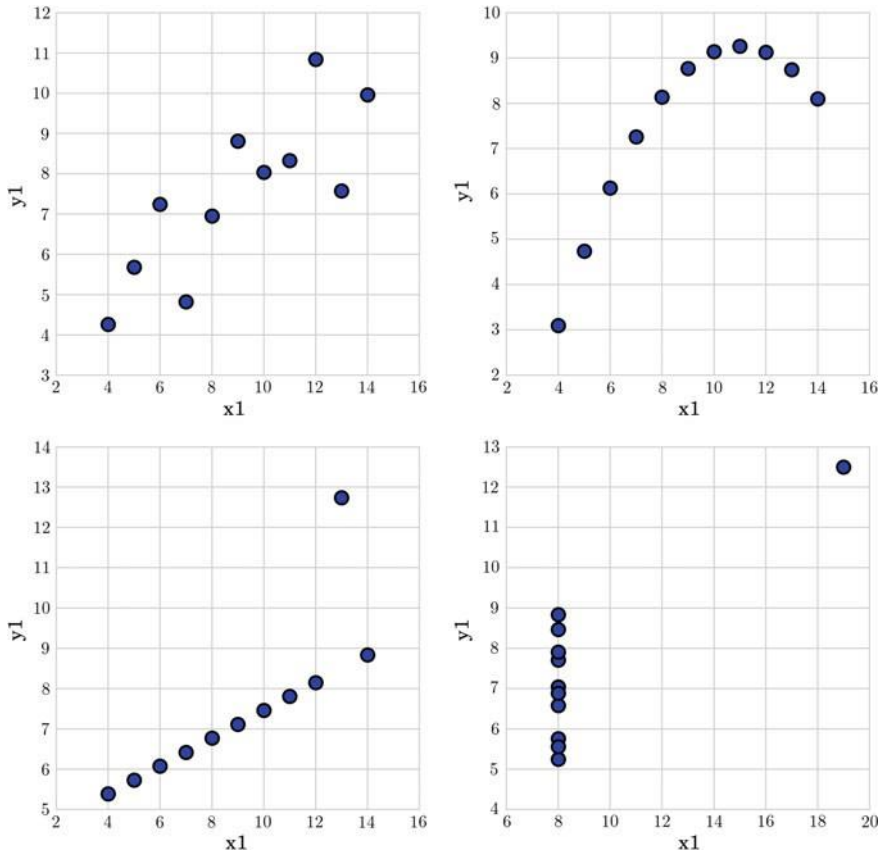


Fig. 3.10 Anscombe configurations

having ρ 0, does not necessarily mean that the variables are not correlated! Pearson's correlation captures correlations of first order, but not nonlinear correlations.Moreover, it does not work well in the presence of outliers.

Spearman's Rank Correlation

The *Spearman's rank correlation* comes as a solution to the robustness problem of Pearson's correlation when the data contain outliers. The main idea is to use the ranks of the sorted sample data, instead of the values themselves. For example, in the list [4, 3, 7, 5], the rank of 4 is 2, since it will appear second in the ordered list ([3, 4, 5, 7]). Spearman's correlation computes the correlation between the ranks

of the data. For example, considering the data: X = [10, 20, 30, 40, 1000], and Y = [70, 1000, 50, 10, 20], where we have an outlier in each one set. If we compute the ranks, they are [1.0, 2.0, 3.0, 4.0, 5.0] and [2.0, 1.0, 3.0, 5.0, 4.0]. As value of the Pearson's coefficient, we get 0.28, which does not show much

between the sets. However, the Spearman's rank coefficient, capturing the correlation between the ranks, gives as a final value of 0.80, confirming the correlation betweenthe sets. As an exercise, you can compute the Pearson's and the Spearman's rank correlations for the different Anscombe configurations given in Fig. 3.10. Observe if linear and nonlinear correlations can be captured by the Pearson's and the Spearman's rank correlations.

Statistical Inference

Introduction

There is not only one way to address the problem of statistical inference. In fact, there are two main approaches to statistical inference: the frequentist and Bayesianapproaches. Their differences are subtle but fundamental:

- In the case of the *frequentist approach*, the main assumption is that there is a population, which can be represented by several parameters, from which we can obtain numerous random samples. Population parameters are fixed but they are not accessible to the observer. The only way to derive information about these parameters is to take a sample of the population, to compute the parameters of thesample, and to use statistical inference techniques to make probable propositionsregarding population parameters.
- The *Bayesian approach* is based on a consideration that data are fixed, not the result of a repeatable sampling process, but parameters describing data can be described probabilistically. To this end, Bayesian inference methods focus on producing parameter distributions that represent all the knowledge we can extract from the sample and from prior information about the problem.

A deep understanding of the differences between these approaches is far beyondthe scope of this chapter, but there are many interesting references that will enable you to learn about it [1]. What is really important is to realize that the approaches are based on different assumptions which determine the validity of their inferences.The assumptions are related in the first case to a sampling process; and to a statistical model in the second case. Correct inference requires these assumptions to be correct. The fulfillment of this requirement is not part of the method, but it is the responsibility of the data scientist.

In this chapter, to keep things simple, we will only deal with the first approach, but we suggest the reader also explores the second approach as it is well worth it!

Statistical Inference: The Frequentist Approach

As we have said, the ultimate objective of statistical inference, if we adopt the fre- quentist approach, is to produce probable propositions concerning population

param- eters from analysis of a sample. The most important classes of propositions are as follows:

- Propositions about *point estimates*. A point estimate is a particular value that best approximates some parameter of interest. For example, the mean or the varianceof the sample.
- Propositions about *confidence intervals* or *set estimates*. A confidence interval isa range of values that best represents some parameter of interest.
- Propositions about the acceptance or rejection of a *hypothesis*.

In all these cases, the production of propositions is based on a simple assumption: we can estimate the probability that the result represented by the proposition has been caused by chance. The estimation of this probability by sound methods is oneof the main topics of statistics.

The development of traditional statistics was limited by the scarcity of computa- tional resources. In fact, the only computational resources were mechanical devices and human computers, teams of people devoted to undertaking long and tedious calculations. Given these conditions, the main results of classical statistics are theo- retical approximations, based on idealized models and assumptions, to measure the effect of chance on the statistic of interest. Thus, concepts such as the *Central Limit Theorem*, the *empirical sample distribution* or the *t-test* are central to understandingthis approach.

The development of modern computers has opened an alternative strategy for measuring chance that is based on simulation; producing computationally inten- sive methods including resampling methods (such as bootstrapping), Markov chain Monte Carlo methods, etc. The most interesting characteristic of these methods is that they allow us to treat more realistic models.

Measuring the Variability in Estimates

Estimates produced by descriptive statistics are not equal to the *truth* but they are better as more data become available. So, it makes sense to use them as central elements of our propositions and to measure its variability with respect to the sample size.

Point Estimates

Let us consider a dataset of accidents in Barcelona in 2013. This dataset can be downloaded from the OpenDataBCN website,[1] Barcelona City Hall's open data service. Each register in the dataset represents an accident via a series of features: weekday, hour, address, number of dead and injured people, etc. This dataset will represent our population: the set of all reported traffic accidents in Barcelona during2013.

In[1]:

Sampling Distribution of Point Estimates

Let us suppose that we are interested in describing the daily number of traffic acci- dents in the streets of Barcelona in 2013. If we have access to the *population*, the computation of this parameter is a simple operation: the total number of accidents divided by 365.

```
data = pd.read_csv("files/ch04/ACCIDENTS_GU_BCN_2013.csv")
data['Date'] = data[u'Dia de mes'].apply(lambda x: str(x))
               + '-' +
               data[u'Mes de any'].apply(lambda x: str(x))
data['Date'] = pd.to_datetime(data['Date'])
```

Out[1]:
Mean:
25.9095

suppose that we only have access to a limited part of the data (the *sample*): the number of accidents during *some* days of 2013. Can we still give an approximation of the population mean?

The most intuitive way to go about providing such a mean is simply to take the *sample mean*. The sample mean is a point estimate of the population mean. If we can only choose one value to estimate the population mean, then this is our best guess.

The problem we face is that estimates generally vary from one sample to another, and this sampling variation suggests our estimate may be close, but it will not be exactly equal to our parameter of interest. How can we measure this variability?

In our example, because we have access to the population, we can empirically build the *sampling distribution of the sample mean*[2] for a given number of observations. Then, we can use the sampling distribution to compute a measure of the variability. In Fig. 4.1, we can see the empirical sample distribution of the mean for *s* = 10.000 samples of 200 observations from our dataset. This empirical distribution has been built in the following way:

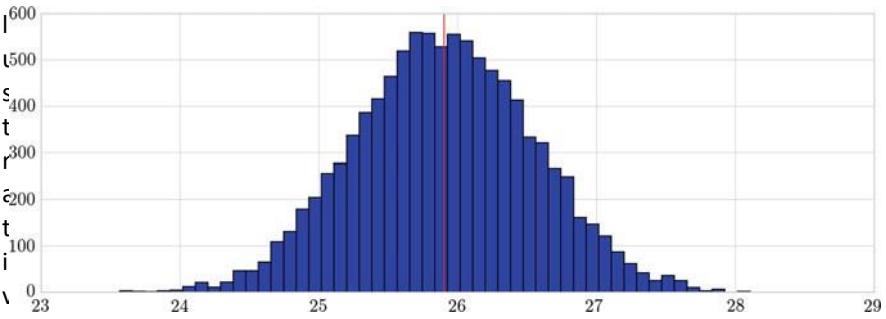


Fig. 4.1 Empirical distribution of the sample mean. In *red*, the mean value of this distribution

1. Draw *s* (a large number) independent samples $\{\mathbf{x}^1, \ldots, \mathbf{x}^s\}$ from the population where each element $\mathbf{x}^j$ is composed of $\{x_i^j\}_{i=1,\ldots,n}$.
2. Evaluate the sample mean $\hat{\mu}^j = \frac{1}{n}\sum_{i=1}^{n} x_i^j$ of each sample.
3. Estimate the sampling distribution of μ by the empirical distribution of the sample replications.

```
# population
df = accidents.to_frame()
N_test = 10000
elements = 200
# mean array of samples
means = [0] * N_test
# sample generation
for i in range(N_test):
    rows = np.random.choice(df.index.values, elements)
    sampled_df = df.ix[rows]
    means[i] = sampled_df.mean()
```

te from a sample of size n, we define its *sampling distribution* as the distribution of the point estimate based on samples of size n from its population. This definition is valid for point estimates of other population parameters, such as the population median or population standard deviation, but we will focus on the analysis of the sample mean.

In general, given a point estimate, we can define its sampling distribution as the distribution of the point estimate based on samples of size n from its population. This definition is valid for point estimates of other population parameters, such as the population median or population standard deviation, but we will focus on the analysis of the sample mean.

The sampling distribution of an estimate plays an important role in understanding the real meaning of propositions concerning point estimates. It is very useful to think of a particular point estimate as being drawn from such a distribution.

The Traditional Approach

In real problems, we do not have access to the real population and so estimation of the sampling distribution of the estimate from the empirical distribution of the sample replications is not an option. But this problem can be solved by making use of some theoretical results from traditional statistics.

It can be mathematically shown that given n independent observations $\{x_{i\ i=1,..,n}\}$ of a population with a standard deviation σ_x, the standard deviation of the samplemean $\sigma_{\bar{x}}$, or *standard error*, can be approximated by this formula:

$$SE = \frac{\sigma_x}{\sqrt{n}}$$

The demonstration of this result is based on the Central Limit Theorem: an oldtheorem with a history that starts in 1810 when Laplace released his first paper on it.This formula uses the standard deviation of the population σ_x, which is not known, but it can be shown that if it is substituted by its empirical estimate $\hat{\sigma}_x$, the estimationis sufficiently good if $n > 30$ and the population distribution is not skewed. Thisallows us to estimate the standard error of the sample mean even if we do not have access to the population.

So, how can we give a measure of the variability of the sample mean? The answeris simple: by giving the **empirical standard error of the mean distribution**.

```
rows = np.random.choice(df.index.values, 200)
sampled_df = df.ix[rows]
est_sigma_mean = sampled_df.std()/math.sqrt(200)

print 'Direct estimation of SE from one sample of
      200 elements:', est_sigma_mean[0]
print ' Estimation of the SE by simulating 10000 samples of
      200 elements:', np.array(means).std()
```

Out[3]: Direct estimation of SE from one sample of 200 elements: 0.6536Estimation of the SE by simulating 10000 samples of 200
elements: 0.6362

Unlike the case of the sample mean, there is no formula for the standard error ofother interesting sample estimates, such as the median.

The Computationally Intensive Approach

Let us consider from now that our full dataset is a sample from a hypothetical population (this is the most common situation when analyzing real data!).

A modern alternative to the traditional approach to statistical inference is the bootstrapping method [2]. In the bootstrap, we draw *n* observations *with replacement* from the original data to create a bootstrap sample or resample. Then, we can calculate the mean for this resample. By repeating this process a large number of times, we can build a good approximation of the mean sampling distribution (see Fig. 4.2).

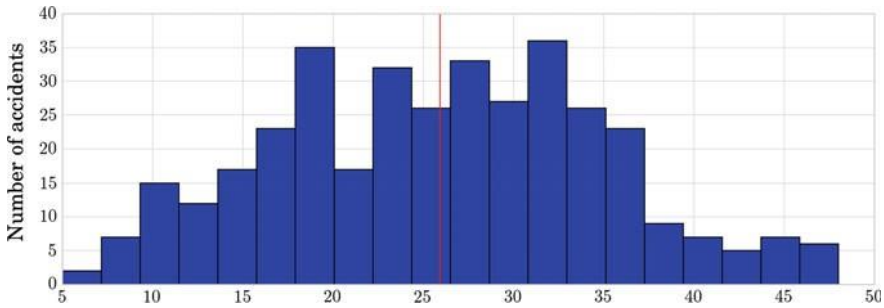


Fig. 4.2 Mean sampling distribution by bootstrapping. In *red*, the mean value of this distribution

In [4]:

```
def meanBootstrap(X, numberb):
    x = [0]*numberb
    for i in range(numberb):
        sample = [X[j]
                  for j
                  in np.random.randint(len(X), size=len(X))
                 ]
        x[i] = np.mean(sample)
    return x
m = meanBootstrap(accidents, 10000)
print "Mean estimate:", np.mean(m)
```

Out[4]: Mean estimate: 25.9094

The basic idea of the bootstrapping method is that the observed sample contains sufficient information about the underlying distribution. So, the information we can extract from resampling the sample is a good approximation of what can be expected from resampling the population.

The bootstrapping method can be applied to other simple estimates such as the median or the variance and also to more complex operations such as estimates of censored data.[3]

Confidence Intervals

A point estimate Θ, such as the sample mean, provides a *single plausible value for a parameter*. However, as we have seen, a point estimate is rarely perfect; usually there is some error in the estimate. That is why we have suggested using the standard error as a measure of its variability.

Instead of that, a next logical step would be to provide *a plausible range of values* for the parameter. A plausible range of values for the sample parameter is called a *confidence interval*.

We will base the definition of *confidence interval* on two ideas:

1. Our point estimate is the most plausible value of the parameter, so it makes senseto build the confidence interval around the point estimate.
2. The *plausibility* of a range of values can be defined from the sampling distributionof the estimate.

For the case of the mean, the Central Limit Theorem states that its samplingdistribution is normal:

Theorem 4.1 *Given a population with a finite mean μ and a finite non-zero variance σ² , the sampling distribution of the mean approaches a normal distribution with a mean of μ and a variance of σ²/n as n, the sample size, increases.*

In this case, and in order to define an interval, we can make use of a well-known result from probability that applies to normal distributions: roughly 95% of the time our estimate will be within 1.96 standard errors of the true mean of the distribution. If the interval spreads out 1.96 standard errors from a normally distributed point estimate, intuitively we can say that we are *roughly 95% confident that we have captured the true parameter*.

$$CI = [\Theta - 1.96 \times SE, \Theta + 1.96 \times SE]$$

In[5]:

```
m = accidents.mean()
se = accidents.std()/math.sqrt(len(accidents))
ci = [m - se*1.96, m + se*1.96]
print "Confidence interval:", ci
```

Out[5]: Confidence interval: [24.975, 26.8440]

Suppose we want to consider confidence intervals where the confidence level issomewhat higher than 95%: perhaps we would like a confidence level of 99%. To create a 99% confidence interval, change 1.96 in the 95% confidence interval formula to be 2.58 (it can be shown that 99% of the time a normal random variable will be within 2.58 standard deviations of the mean).

In general, if the point estimate follows the normal model with standard error SE , then a confidence interval for the population parameter is

$$\Theta \pm z \times SE$$

where z corresponds to the confidence level selected:

Confidence Level	90%	95%	99%	99.9%
z Value	1.65	1.96	2.58	3.291

This is how we would compute a 95% confidence interval of the sample mean using bootstrapping:

1. Repeat the following steps for a large number, *s*, of times:

    a. Draw *n* observations with replacement from the original data to create abootstrap sample or resample.
    b. Calculate the mean for the resample.

2. Calculate the mean of your *s* values of the sample statistic. This process givesyou a "bootstrapped" estimate of the sample statistic.
3. Calculate the standard deviation of your *s* values of the sample statistic. Thisprocess gives you a "bootstrapped" estimate of the SE of the sample statistic.
4. Obtain the 2.5th and 97.5th percentiles of your *s* values of the sample statistic.

In [6]:

```
m = meanBootstrap(accidents, 10000)
sample_mean = np.mean(m)
sample_se =  np.std(m)

print "Mean estimate:", sample_mean
print "SE of the estimate:",  sample_se

ci = [np.percentile(m, 2.5), np.percentile(m, 97.5)]
print  "Confidence interval:", ci
```

Out[6]: Mean estimate: 25.9039
SE of the estimate: 0.4705
Confidence interval: [24.9834, 26.8219]

But What Does "95% Confident" Mean?

The real meaning of "confidence" is not evident and it must be understood from thepoint of view of the generating process.

Suppose we took many (infinite) samples from a population and built a 95% confidence interval from each sample. Then about 95% of those intervals would contain the actual parameter. In Fig. 4.3 we show how many confidence intervals computed from 100 different samples of 100 elements from our dataset contain the real population mean. If this simulation could be done with infinite different samples, 5% of those intervals would not contain the true mean.

So, when faced with a sample, the correct interpretation of a confidence intervalis as follows:

> In 95% of the cases, when I compute the 95% confidence interval from this sample, the true mean of the population will fall within the interval defined by these bounds: ±1.96 × *SE*.

We cannot say either that our specific sample contains the true parameter or that the interval has a 95% chance of containing the true parameter. That interpretation would not be correct under the assumptions of traditional

statistics.

Hypothesis Testing

Giving a measure of the variability of our estimates is one way of producing a statistical proposition about the population, but not the only one. R.A. Fisher (1890–1962) proposed an alternative, known as *hypothesis testing*, that is based on the concept of *statistical significance*.

Let us suppose that a deeper analysis of traffic accidents in Barcelona results in a difference between 2010 and 2013. Of course, the difference could be caused only by chance, because of the variability of both estimates. But it could also be the case that traffic conditions were very different in Barcelona during the two periods and, because of that, data from the two periods can be considered as belonging to two different populations. Then, the relevant question is: Are the observed effects real ornot?

Technically, the question is usually translated to: *Were the observed effects statistically significant*?

The process of determining the statistical significance of an effect is called *hypothesis testing*.

This process starts by simplifying the options into two competing hypotheses:

- H_0: The mean number of daily traffic accidents is the same in 2010 and 2013 (there is only one population, one true mean, and 2010 and 2013 are just differentsamples from the same population).
- H_A: The mean number of daily traffic accidents in 2010 and 2013 is different (2010 and 2013 are two samples from two different populations).

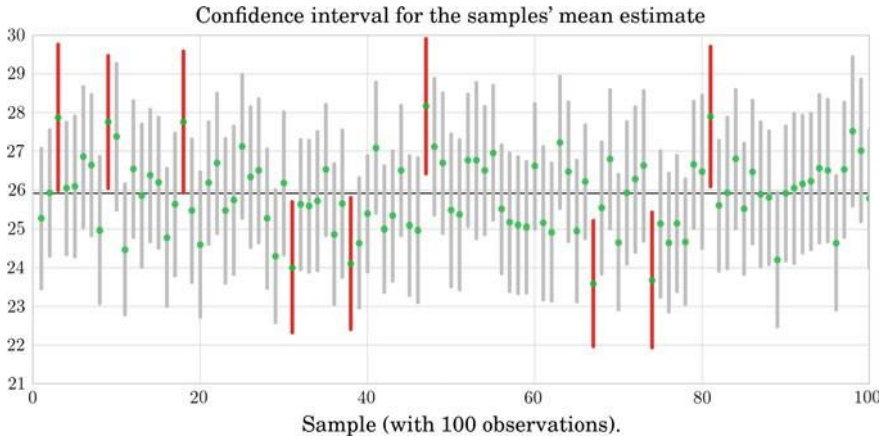


Fig. 4.3 This graph shows 100 sample means (*green points*) and its corresponding confidence intervals, computed from 100 different samples of 100 elements from our dataset. It can be observed that a few of them (those in *red*) do not contain the mean of the population (*black horizontal line*)

We call H_0 the *null hypothesis* and it represents a *skeptical* point of view: the effect we have observed is due to chance (due to the specific sample bias). H_A is the*alternative hypothesis* and it represents the other point of view: the effect is real.

The general rule of frequentist hypothesis testing: we will not *discard* H_0 (and hence we will not consider H_A) unless the observed effect is *implausible* under H_0.

Testing Hypotheses Using Confidence Intervals

We can use the concept represented by *confidence intervals* to measure the *plausi-bility* of a hypothesis.

We can illustrate the evaluation of the hypothesis setup by comparing the meanrate of traffic accidents in Barcelona during 2010 and 2013:

In[7]:
```
data = pd.read_csv("files/ch04/ACCIDENTS_GU_BCN_2010.csv",
                   encoding='latin-1')

# Create a new column which is the date
data['Date'] = data['Dia de mes'].apply(lambda x: str(x))
               + '-' +
               data['Mes de any'].apply(lambda x: str(x))
data2 = data['Date']
counts2010 = data['Date'].value_counts()
print '2010: Mean', counts2010.mean()

data = pd.read_csv("files/ch04/ACCIDENTS_GU_BCN_2013.csv",
                   encoding='latin-1')

# Create a new column which is the date
data['Date'] = data['Dia de mes'].apply(lambda x: str(x))
               + '-' +
               data['Mes de any'].apply(lambda x: str(x))
data2 = data['Date']
counts2013 = data['Date'].value_counts()
print '2013: Mean', counts2013.mean()
```

Out[7]:
```
2010: Mean 24.8109
2013: Mean 25.9095
```

This estimate suggests that in 2013 the mean rate of traffic accidents in Barcelonawas higher than it was in 2010. But is this effect statistically significant?

Based on our sample, the 95% confidence interval for the mean rate of trafficaccidents in Barcelona during 2013 can be calculated as follows:

In[8]:
```
n = len(counts2013)
mean = counts2013.mean()
s = counts2013.std()
ci = [mean - s*1.96/np.sqrt(n), mean + s*1.96/np.sqrt(n)]
print '2010 accident rate estimate:', counts2010.mean()
print '2013 accident rate estimate:', counts2013.mean()
print 'CI for 2013:',ci
```

```
Out[8]: 2010 accident rate estimate: 24.8109
        2013 accident rate estimate: 25.9095
CI for 2013: [24.9751, 26.8440]
```

Because the 2010 accident rate estimate does not fall in the range of plausible values of 2013, we say the alternative hypothesis cannot be discarded. That is, it cannot be ruled out that in 2013 the mean rate of traffic accidents in Barcelona washigher than in 2010.

Interpreting CI Tests

Hypothesis testing is built around rejecting or failing to reject the null hypothesis. That is, we do not reject H_0 unless we have strong evidence against it. But what precisely does strong evidence mean? As a general rule of thumb, for those cases where the null hypothesis is actually true, we do not want to incorrectly reject H_0 more than 5% of the time. This corresponds to a *significance level* of $\alpha = 0.05$. In this case, the correct interpretation of our test is as follows:

> If we use a 95% confidence interval to test a problem where the null hypothesis is true, we will make an error whenever the point estimate is at least 1.96 standard errors away from thepopulation parameter. This happens about 5% of the time (2.5% in each tail).

Testing Hypotheses Using *p*-Values

A more advanced notion of *statistical significance* was developed by R.A. Fisher in the 1920s when he was looking for a test to decide whether variation in crop yields was due to some specific intervention or merely random factors beyond experimental control.

Fisher first assumed that fertilizer caused no difference (*null hypothesis*) and thencalculated *P*, the probability that an observed yield in a fertilized field would occurif fertilizer had no real effect. This probability is called the *p-value*.

The *p*-value is the probability of observing data at least as favorable to the alter-native hypothesis as our current dataset, if the null hypothesis is true. We typically use a summary statistic of the data to help compute the *p*-value and evaluate the hypotheses.

Usually, if *P* is less than 0.05 (the chance of a fluke is less than 5%) the result is declared *statistically significant*.

It must be pointed out that this choice is rather arbitrary and should not be takenas a scientific truth.

The goal of classical hypothesis testing is to answer the question, "*Given a sample and an apparent effect, what is the probability of seeing such an effect by chance?*"Here is how we answer that question:

- The first step is to quantify the size of the apparent effect by choosing a test statistic. In our case, the apparent effect is a difference in accident rates, so a natural choicefor the test statistic is the **difference in means between the two periods**.

- The second step is to define a *null hypothesis*, which is a model of the system based on the assumption that the apparent effect is not real. In our case, the null hypothesis is that there is no difference between the two periods.
- The third step is to compute a *p-value*, which is the probability of seeing the apparent effect if the null hypothesis is true. In our case, we would compute the difference in means, then compute the probability of seeing a difference as big, orbigger, under the null hypothesis.
- The last step is to *interpret the result*. If the *p*-value is low, the effect is said to be *statistically significant*, which means that it is unlikely to have occurred by chance. In this case we infer that the effect is more likely to appear in the larger population.

In our case, the test statistic can be easily computed:

In [9]:
```
m= len(counts2010)
n= len(counts2013)
p = (counts2013.mean() - counts2010.mean())
print 'm:', m, 'n:', n
print 'mean difference: ', p
```

Out[9]: m: 365 n: 365
mean difference: 1.0986

To approximate the *p*-value , we can follow the following procedure:

1. Pool the distributions, generate samples with size n and compute the differencein the mean.
2. Generate samples with size n and compute the difference in the mean.
3. Count how many differences are larger than the observed one.

In [10]:
```
# pooling distributions
x = counts2010
y = counts2013
pool = np.concatenate([x, y])
np.random.shuffle(pool)

#sample generation
import random
N = 10000 # number of samples
diff = range(N)
for i in range(N):
    p1 = [random.choice(pool) for _ in xrange(n)]
    p2 = [random.choice(pool) for _ in xrange(n)]
    diff[i] = (np.mean(p1) - np.mean(p2))
```

In [11]:

```
# counting differences larger than the observed one
diff2 = np.array(diff)
w1 = np.where(diff2 > p)[0]

print 'p-value (Simulation)=', len(w1)/float(N),
      '(', len(w1)/float(N)*100 ,'%)', 'Difference =', p
if (len(w1)/float(N)) < 0.05:
    print 'The effect is likely'
else:
    print 'The effect is not likely'
```

Out[11]: p-value (Simulation)= 0.0485 (4.85%) Difference = 1.098The effect is likely

Interpreting *P*-Values

A *p*-value is the probability of an observed (or more extreme) result arising only from chance.

If *P* is less than 0.05, there are two possible conclusions: there is a real effect or the result is an improbable fluke. *Fisher's method offers no way of knowing which isthe case*.

We must not confuse the odds of getting a result (if a hypothesis is true) with the odds of favoring the hypothesis if you observe that result. If *P* is less than 0.05,we cannot say that this means that it is 95% certain that the observed effect is real and could not have arisen by chance. Given an observation E and a hypothesis H, $P(E|H)$ and $P(H|E)$ are not the same!

Another common error equates *statistical significance* to *practical importance/ relevance*. When working with large datasets, we can detect statistical significancefor small effects that are meaningless in practical terms.

We have defined the effect as *a difference in mean as large or larger than δ, considering the sign*. A test like this is called *one sided*.

If the relevant question is whether *accident rates are different*, then it makes senseto test the absolute difference in means. This kind of test is called two sided becauseit counts both sides of the distribution of differences.

Direct Approach

The formula for the standard error of the absolute difference in two means is similarto the formula for other standard errors. Recall that the standard error of a single mean can be approximated by:

$$SE_{\bar{x}_1} = \frac{\sigma_1}{\sqrt{n_1}}$$

The standard error of the difference of two sample means can be constructed fromthe standard errors of the separate sample means:

$$SE_{\bar{x}_1 - \bar{x}_2} = \sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}$$

This would allow us to define a direct test with the 95% confidence interval.

But Is the Effect *E* Real?

We do not yet have an answer for this question! We have defined a null hypothesis H_0 (the effect is not real) and we have computed the probability of the observed effect under the null hypothesis, which is $P(E \mid H_0)$, where E is an effect as big as or bigger than the apparent effect and a p-value .

We have stated that from the frequentist point of view, we cannot consider H_A unless $P(E \mid H_0)$ is less than an arbitrary value. But the real answer to this question must be based on comparing $P(H_0 \mid E)$ to $P(H_A \mid E)$, not on $P(E \mid H_0)$! One possi- ble solution to these problems is to use *Bayesian reasoning*; an alternative to the frequentist approach.

No matter how many data you have, you will still depend on intuition to decide how to interpret, explain, and use that data. Data cannot speak by themselves. Data scientists are interpreters, offering one interpretation of what the useful narrative story derived from the data is, if there is one at all.

UNIT-3

Supervised Learning: First step, learning curves, training-validation and test. Learning models generalities, support vector machines, random forest. Examples

Supervised Learning

Machine learning involves coding programs that automatically adjust their perfor- mance in accordance with their exposure to information in data. This learning is achieved via a parameterized model with tunable parameters that are automatically adjusted according to different performance criteria. Machine learning can be con- sidered a subfield of artificial intelligence (AI) and we can roughly divide the fieldinto the following three major classes.

1. Supervised learning: Algorithms which learn from a training set of labeled examples (exemplars) to generalize to the set of all possible inputs. Examples of techniques in supervised learning: logistic regression, support vector machines,decision trees, random forest, etc.
2. Unsupervised learning: Algorithms that learn from a training set of unlabeled examples. Used to explore data according to some statistical, geometric or sim- ilarity criterion. Examples of unsupervised learning include k-means clustering and kernel density estimation. We will see more on this kind of techniques in Chap. 7.
3. Reinforcement learning: Algorithms that learn via reinforcement from criticismthat provides information on the quality of a solution, but not on how to improve it. Improved solutions are achieved by iteratively exploring the solution space.

This chapter focuses on a particular class of supervised machine learning: *classification*. As a data scientist, the first step you apply given a certain problem is to identify the question to be answered. According to the type of answer we are seeking, we are directly aiming for a certain set of techniques.

Supervised Learning

- If our question is answered by YES/NO, we are facing a classification problem. Classifiers are also the tools to use if our question admits only a discrete set of answers, i.e., we want to select from a finite number of choices.
  - Given the results of a clinical test, e.g., does this patient suffer from diabetes?
  - Given a magnetic resonance image, is it a tumor shown in the image?
  - Given the past activity associated with a credit card, is the current operationfraudulent?
- If our question is a prediction of a real-valued quantity, we are faced with a

regres-sion problem. We will go into details of regression in Chap. 6.

- Given the description of an apartment, what is the expected market value of theflat? What will the value be if the apartment has an elevator?
- Given the past records of user activity on Apps, how long will a certain client be connected to our App?
- Given my skills and marks in computer science and maths, what mark will I achieve in a data science course?

Observe that some problems can be solved using both regression and classification. As we will see later, many classification algorithms are thresholded regressors. There is a certain skill involved in designing the correct question and this dramatically affects the solution we obtain.

The Problem

In this chapter we use data from the Lending Club[1] to develop our understanding of machine learning concepts. The Lending Club is a peer-to-peer lending company. It offers loans which are funded by other people. In this sense, the Lending Club acts as a hub connecting borrowers with investors. The client applies for a loan of acertain amount, and the company assesses the risk of the operation. If the applicationis accepted, it may or may not be fully covered. We will focus on the predictionof whether the loan will be fully funded, based on the scoring of and information related to the application.

We will use the partial dataset of period 2007–2011. Framing the problem a little bit more, based on the information supplied by the customer asking for a loan, we want to predict whether it will be granted up to a certain threshold *thr* . The attributes we use in this problem are related to some of the details of the loan application, such as amount of the loan applied for the borrower, monthly payment to be made by the borrower if the loan is accepted, the borrower's annual income, the number of incidences of delinquency in the borrower's credit file, and interest rate of the loan,among others.

In this case we would like to predict unsuccessful accepted loans. A loan applica-tion is unsuccessful if the funded amount (funded_amnt) or the amount funded by investors (funded_amnt_inv) falls far short of the requested loan amount (loan_amnt). That is,

$$\frac{loan - funded}{loan} \geq 0.95.$$

First Steps

Note that in this problem we are predicting a binary value: either the loan is fully funded or not. Classification is the natural choice of machine learning tools for prediction with discrete known outcomes. According to the cardinality of the target set, one usually distinguishes between *binary* classifiers when the target output onlytakes two values, i.e., the classifier answers questions with a yes or a no; or *multiclass* classifiers, for a larger number of classes. This issue is important in that not all methods can naturally handle the multiclass setting.[2]

In a formal way, classification is regarded as the problem of finding a function $h(\mathbf{x}) : \mathbb{R}^d \to K$ that maps an input space in $\mathbb{R}^d$ onto a discrete set of k target outputs or classes $K = \{1, \ldots, k\}$. In this setting, the features are arranged as a vector $\mathbf{x}$ of d real-valued numbers.[3]

We can encode both target states in a numerical variable, e.g., a successful loan target can take value 1; and it is 1, otherwise.

Let us check the dataset,[4]

In[1]:

```
import pickle
ofname = open('./files/ch05/dataset_small.pkl','rb')
# x stores input data and y target values
(x,y) = pickle.load(ofname)
```

[2]Several well-known techniques such as support vector machines or adaptive boosting (adaboost) are originally defined in the binary case. Any binary classifier can be extended to the multiclass case in two different ways. We may either change the formulation of the learning/optimization process. This requires the derivation of a new learning algorithm capable of handling the new modeling. Alternatively, we may adopt ensemble techniques. The idea behind this latter approach is that we may divide the multiclass problem into several binary problems; solve them; and then aggregate the results. If the reader is interested in these techniques, it is a good idea to look for: one-versus-all, one-versus-one, or error correcting output codes methods.

[3]Many problems are described using categorical data. In these cases either we need classifiers that are capable of coping with this kind of data or we need to change the representation of those variables into numerical values.

[4]The notebook companion shows the preprocessing steps, from reading the dataset, cleaning and imputing data, up to saving a subsampled clean version of the original dataset.

A problem in Scikit-learn is modeled as follows:

- Input data is structured in Numpy arrays. The size of the array is expected to be [n_samples, n_features]:
  - n_samples: The number of samples (*n*). Each sample is an item to process (e.g., classify). A sample can be a document, a picture, an audio file, a video, an astronomical object, a row in a database or CSV file, or whatever you can describe with a fixed set of quantitative traits.
  - n_features: The number of features (*d*) or distinct traits that can be used to describe each item in a quantitative manner. Features are generally real-valued,but may be Boolean, discrete-valued or even categorical.

$$\text{feature matrix}: \quad \mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ x_{31} & x_{32} & \cdots & x_{3d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix}$$

$$\text{label vector}: \quad \mathbf{y}^T = [y_1, y_2, y_3, \cdots y_n]$$

The number of features must be fixed in advance. However, it can be very great(e.g., millions of features).

In[2]:
```
dims = x.shape[1]
N = x.shape[0]
print 'dims: ' + str(dims) + ', samples: ' + str(N)
```

Out[2]: dims: 15, samples: 4140

Considering data arranged as in the previous matrices we refer to:

- the columns as features, attributes, dimensions, regressors, covariates, predictors,or independent variables;
- the rows as instances, examples, or samples;
- the target as the label, outcome, response, or dependent variable.

All objects in Scikit-learn share a uniform and limited API consisting of three complementary interfaces:

- an estimator interface for building and fitting models (fit());
- a predictor interface for making predictions (predict());
- a transformer interface for converting data (transform()).

Let us apply a classifier using Python's Scikit-learn libraries,

In [3]:

```
from sklearn import neighbors
from sklearn import datasets
# Create an instance of K-nearest neighbor classifier
knn = neighbors.KNeighborsClassifier(n_neighbors = 11)
# Train the classifier
knn.fit(x, y)
# Compute the prediction according to the model
yhat = knn.predict(x)
# Check the result on the last example
print 'Predicted value: ' + str(yhat[-1]),
      ', real target: ' + str(y[-1])
```

Out[3]: Predicted value: -1.0 , real target: -1.0

The basic measure of performance of a classifier is its *accuracy*. This is defined as the number of correctly predicted examples divided by the total amount of examples. Accuracy is related to the error as follows: $acc = 1 - err$.

$$acc = \frac{\text{Number of correct predictions}}{n}$$

Each estimator has a score()method that invokes the default scoring metric. In the case of k-nearest neighbors, this is the classification accuracy.

In [4]:
```
knn.score(x,y)
```

Out[4]: 0.83164251207729467

It looks like a really good result. But how good is it? Let us first understand a little bit more about the problem by checking the distribution of the labels.

Let us load the dataset and check the distribution of labels:

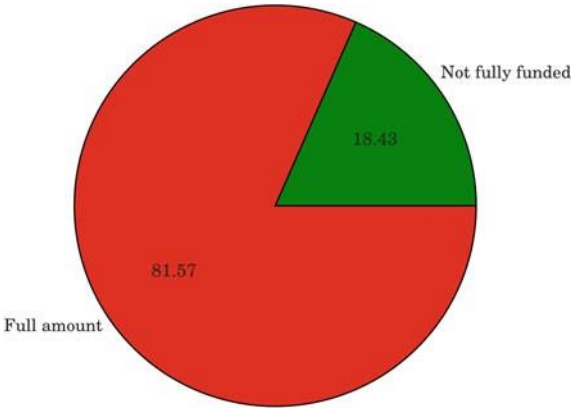
In [5]:
```
plt.pie(np.c_[np.sum(np.where(y == 1, 1, 0)),
              np.sum(np.where(y == -1, 1, 0))][0],
              labels = ['Not fully funded','Full amount'],
              colors = ['r', 'g'],shadow = False,
              autopct = '%.2f' )
plt.gcf().set_size_inches((7, 7))
```

with the result observed in Fig. 5.1.

Note that there are far more positive labels than negative ones. In this case, the dataset is referred to as *unbalanced*.[5] This has important consequences for a classifier as we will see later on. In particular, a very simple rule such as always predict the

[5]The term unbalanced describes the condition of data where the ratio between positives and negatives is a small value. In these scenarios, always predicting the majority class usually yields accurate performance, though it is not very informative. This kind of problems is very common when we want to model unusual events such as rare diseases, the occurrence of a failure in machinery, fraudulent credit card operations, etc. In these scenarios, gathering data from usual events is very easy but collecting data from unusual events is difficult and results in a comparatively small dataset.

Fig. 5.1 Pie chart showing the distribution of labels inthe dataset



majority class, will give us good performance. In our problem, always predicting that the loan will be fully funded correctly predicts 81.57% of the samples. Observethat this value is very close to that obtained using the classifier.

Although accuracy is the most normal metric for evaluating classifiers, there arecases when the business value of correctly predicting elements from one class is different from the value for the prediction of elements of another class. In those cases, accuracy is not a good performance metric and more detailed analysis is needed. The *confusion matrix* enables us to define different metrics considering such scenarios. The confusion matrix considers the concepts of the classifier outcome and the actual ground truth or gold standard. In a binary problem, there are four possiblecases:

- *True positives (TP)*: When the classifier predicts a sample as positive and it really is positive.
- *False positives (FP)*: When the classifier predicts a sample as positive but in fact it is negative.
- *True negatives (TN)*: When the classifier predicts a sample as negative and it really is negative.
- *False negatives (FN)*: When the classifier predicts a sample as negative but in fact it is positive.

We can summarize this information in a matrix, namely the confusion matrix, asfollows:

		Gold Standard: Positive	Gold Standard: Negative	
Prediction	Positive	TP	FP	→ Precision
	Negative	FN	TN	→ Negative Predictive Value
		↓ Sens itivity		
		↓ Spec ificity (Recall)		

The combination of these elements allows us to define several performance metrics:

- *Accuracy:*

$$\text{accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

- Column-wise we find these two partial performance metrics:
  - *Sensitivity or Recall:*

$$\text{sensitivity} = \frac{\text{TP}}{\text{Real Positives}} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

  - *Specificity:*

$$\text{specificity} = \frac{\text{TN}}{\text{Real Negatives}} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

- Row-wise we find these two partial performance metrics:
  - *Precision or Positive Predictive Value:*

$$\text{precision} = \frac{\text{TP}}{\text{Predicted Positives}} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

  - *Negative predictive value:*

$$\text{NPV} = \frac{\text{TN}}{\text{Predicted Negative}} = \frac{\text{TN}}{\text{TN} + \text{FN}}$$

These partial performance metrics allow us to answer questions concerning how often a classifier predicts a particular class, e.g., what is the rate of predictions for not fully funded loans that have actually not been fully funded? This question is answered by recall. In contrast, we could ask: Of all the fully funded loans predicted by the classifier, how many have been fully funded? This is answered by the precision metric.

Let us compute these metrics for our problem.

In [6]:
```
yhat = knn.predict(x)
TP = np.sum(np.logical_and(yhat == -1, y == -1))
TN = np.sum(np.logical_and(yhat == 1, y == 1))
FP = np.sum(np.logical_and(yhat == -1, y == 1))
FN = np.sum(np.logical_and(yhat == 1, y == -1))
print 'TP: '+ str(TP), ', FP: '+ str(FP)
print 'FN: '+ str(FN), ', TN: '+ str(TN)
```

Out[6]:
```
TP: 3370 ,FP: 690
FN: 7 ,      TN: 73
```

Scikit-learn provides us with the confusion matrix,

In [7]:
```
from sklearn import metrics
metrics.confusion_matrix(yhat, y)
# sklearn uses a transposed convention for the confusion
# matrix thus I change targets and predictions
```

Out[7]:
```
3370, 690
7,    73
```

Let us check the following example. Let us select a nearest neighbor classifier with the number of neighbors equal to one instead of eleven, as we did before, andcheck the training error.

In [8]:
```
# Train a classifier using .fit()
knn = neighbors.KNeighborsClassifier(n_neighbors = 1)
knn.fit(x, y)
yhat = knn.predict(x)

print "classification accuracy:" +
      str(metrics.accuracy_score(yhat, y))
print "confusion matrix: \n" +
      str(metrics.confusion_matrix(yhat, y))
```

Out[8]:
```
classification accuracy: 1.0 confusion matrix:
3377  0
   0  763
```

The performance measure is perfect! 100% accuracy and a diagonal confusion matrix! This looks good. However, up to this point we have checked the classifier performance on the same data it has been trained with. During exploitation, in real applications, we will use the classifier on data not previously seen. Let us simulate this effect by splitting the data into two sets: one will be used for learning (*trainingset*) and the other for testing the accuracy (*test set*).

In [9]:
```
# Simulate a real case: Randomize and split data into
# two subsets PRC*100\% for training and the rest
# (1-PRC)*100\% for testing
perm = np.random.permutation(y.size)
PRC = 0.7
split_point = int(np.ceil(y.shape[0]*PRC))

X_train = x[perm[:split_point].ravel(),:]
y_train = y[perm[:split_point].ravel()]

X_test = x[perm[split_point:].ravel(),:]
y_test = y[perm[split_point:].ravel()]
```

If we check the shapes of the training and test sets we obtain,

Out[9]:
```
Training shape: (2898, 15), training targets shape: (2898,)
Testing shape: (1242, 15), testing targets shape: (1242,)
```

With this new partition, let us train the model

In [10]:
```
#Train a classifier on training data
knn = neighbors.KNeighborsClassifier(n_neighbors = 1)
knn.fit(X_train, y_train)
yhat = knn.predict(X_train)

print "\n TRAINING STATS:"
print "classification accuracy:" +
      str(metrics.accuracy_score(yhat, y_train))
print "confusion matrix: \n" +
      str(metrics.confusion_matrix(y_train, yhat))
```

Out[10]:
```
TRAINING STATS:
classification accuracy: 1.0
confusion matrix:
 2355  0
    0  543
```

As expected from the former experiment, we achieve a perfect score. Now let ussee what happens in the simulation with previously unseen data.

In [11]:
```
#Check on the test set
yhat = knn.predict(X_test)
print "TESTING STATS:"
print "classification accuracy:",
      metrics.accuracy_score(yhat, y_test)
print "confusion matrix: \n" +
      str(metrics.confusion_matrix(yhat, y_test))
```

Out[11]:
```
TESTING STATS:
classification accuracy: 0.754428341385
confusion matrix:
 865 148
 157 72
```

Observe that each time we run the process of randomly splitting the dataset and train a classifier we obtain a different performance. A good simulation for approxi-mating the test error is to run this process many times and average the performances.Let us do this![6]

In [12]:

```
# Spitting done by using the tools provided by sklearn:
from sklearn.cross_validation import train_test_split

PRC = 0.3
acc = np.zeros((10,))
for i in xrange(10):
    X_train, X_test, y_train, y_test =
        train_test_split(x, y, test_size = PRC)
    knn = neighbors.KNeighborsClassifier(n_neighbors = 1)
    knn.fit(X_train, y_train)
    yhat = knn.predict(X_test)
    acc[i] = metrics.accuracy_score(yhat, y_test)
acc.shape = (1, 10)
print "Mean expected error:" + str(np.mean(acc[0]))
```

Out[12]: Mean expected error: 0.754669887279

As we can see, the resulting error is below 81%, which was the result of the mostnaive decision process. What is wrong with this result?

Let us introduce the nomenclature for the quantities we have just computed anddefine the following terms.

- *In-sample error* E_{in}: The in-sample error or training error is the error measuredover all the observed data samples in the training set, i.e.,

$$E_{\text{in}} = \frac{1}{N} \sum_{i=1}^{N} e(x_i, y_i)$$

- *Out-of-sample error* E_{out}: The out-of-sample error or generalization error measures the expected error on unseen data. We can approximate/simulate this quantity by holding back some training data for testing purposes.

$$E_{\text{out}} = \mathrm{E}_{x,y}(e(x, y))$$

Note that the definition of the instantaneous error $e(x_i, y_i)$ is still missing. For example, in classification we could use the indicator function to account for a cor- rectly classified sample as follows:

$$e(x_i, y_i) = I[h(x_i) = y_i] = \begin{cases} 1, \text{ if } h(x_i) = y_i \\ 0 \text{ otherwise.} \end{cases}$$

[6] *sklearn* allows us to easily automate the train/test splitting using the function train_test_split(...).

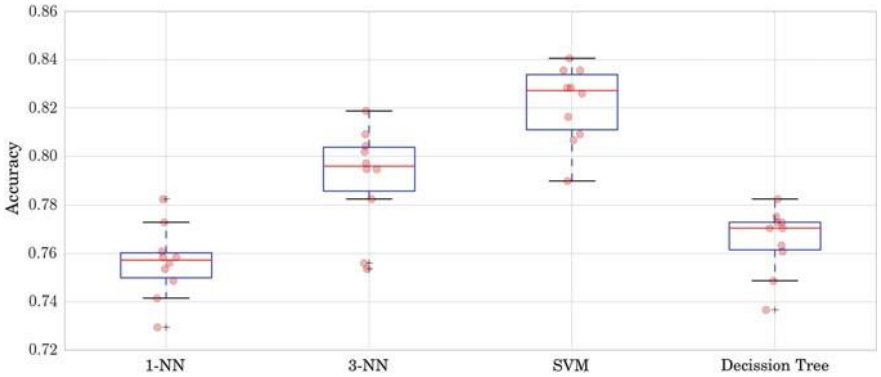


Fig. 5.2 Comparison of the methods using the accuracy metric

Observe that:

$$E_{\text{out}} \geq E_{\text{in}}$$

Using the expected error on the test set, we can select the best classifier for our application. This is called model selection. In this example we cover the most simplistic setting. Suppose we have a set of different classifiers and want to select the "best" one. We may use the one that yields the lowest error rate.

In [13]:

```
from sklearn import tree
from sklearn import svm
PRC = 0.1
acc_r = np.zeros((10, 4))
for i in xrange(10):
    X_train, X_test, y_train, y_test =
        train_test_split(x, y, test_size = PRC)
    nn1 = neighbors.KNeighborsClassifier(n_neighbors = 1)
    nn3 = neighbors.KNeighborsClassifier(n_neighbors = 3)
    svc = svm.SVC()
    dt = tree.DecisionTreeClassifier()

    nn1.fit(X_train, y_train)
    nn3.fit(X_train, y_train)
    svc.fit(X_train, y_train)
    dt.fit(X_train, y_train)

    yhat_nn1 = nn1.predict(X_test)
    yhat_nn3 = nn3.predict(X_test)
    yhat_svc = svc.predict(X_test)
    yhat_dt = dt.predict(X_test)

    acc_r[i][0] = metrics.accuracy_score(yhat_nn1, y_test)
    acc_r[i][1] = metrics.accuracy_score(yhat_nn3, y_test)
    acc_r[i][2] = metrics.accuracy_score(yhat_svc, y_test)
    acc_r[i][3] = metrics.accuracy_score(yhat_dt, y_test)
```

Figure 5.2 shows the results of applying the code.

This process is one particular form of a general model selection technique called *cross-validation*. There are other kinds of cross-validation, such as *leave-one-out* or *K-fold cross-validation*.

- In leave-one-out, given N samples, the model is trained with $N-1$ samples and tested with the remaining one. This is repeated N times, once per training sampleand the result is averaged.
- In K-fold cross-validation, the training set is divided into K nonoverlapping splits. K-1 splits are used for training and the remaining one used to assess the mean. This process is repeated K times leaving one split out each time. The results are then averaged.

What Is Learning?

Let us recall the two basic values defined in the last section. We talk of *training error* or in-sample error, E_{in}, which refers to the error measured over all the observed datasamples in the training set. We also talk of *test error* or *generalization error*, E_{out}, as the error expected on unseen data.

We can empirically estimate the generalization error by means of cross-validationtechniques and observe that:

$$E_{\text{out}} \geq E_{\text{in}}.$$

The goal of learning is to minimize the generalization error; but how can we guarantee this minimization using only training data?

From the above inequality it is easy to derive a couple of very intuitive ideas.

- Because E_{out} is greater than or equal to E_{in}, it is desirable to have
$$E_{\text{in}} \rightarrow 0.$$
- Additionally, we also want the training error behavior to track the generalization error so that if one minimizes the in-sample error the out-of-sample error follows,i.e.,
$$E_{\text{out}} \approx E_{\text{in}}.$$

We can rewrite the second condition as

$$E_{\text{in}} \leq E_{\text{out}} \leq E_{\text{in}} + \Omega,$$

with $\Omega \rightarrow 0$.

We would like to characterize Ω in terms of our problem parameters, i.e., thenumber of samples (N), dimensionality of the problem (d), etc.

Statistical analysis offers an interesting characterization of this quantity[7]

[7]The reader should note that there are several bounds in machine learning to characterize the generalization error. Most of them come from variations of Hoeffding's inequality.

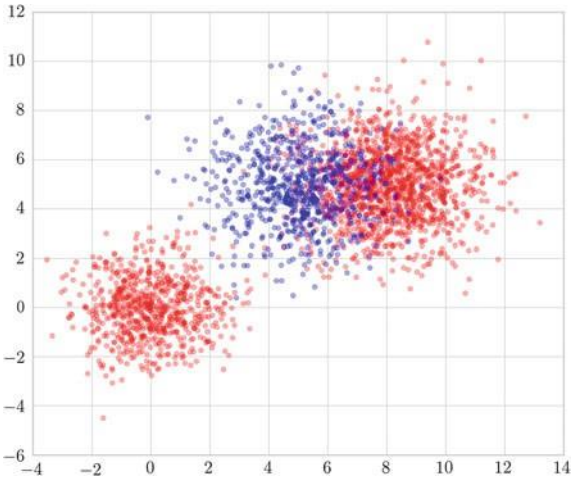


Fig. 5.3 Toy problem data

$$E \quad E \; (C) \quad + \Omega \frac{\log C}{N},$$

where C is a measure of the complexity of the model class we are using. Technically, we may also refer to this model class as the hypothesis space.

Learning Curves

Let us simulate the effect of the number of examples on the training and test errorsfor a given complexity. This curve is called the *learning curve*. We will focus for amoment in a more simple case. Consider the toy problem in Fig. 5.3.

Let us take a classifier and vary the number of examples we feed it for training purposes, then check the behavior of the training and test accuracies as the numberof examples grows. In this particular case, we will be using a decision tree with fixedmaximum depth.

Observing the plot in Fig. 5.4, we can see that:

- As the number of training samples increases, both errors tend to the same
- value. When we have few training data, the training error is very small but the test erroris very large.

Now check the learning curve when the degree of complexity is greater in Fig. 5.5. We simulate this effect by increasing the maximum depth of the tree.

And if we put both curves together, we have the results shown in Fig. 5.6.

Although both show similar behavior, we can note several differences:

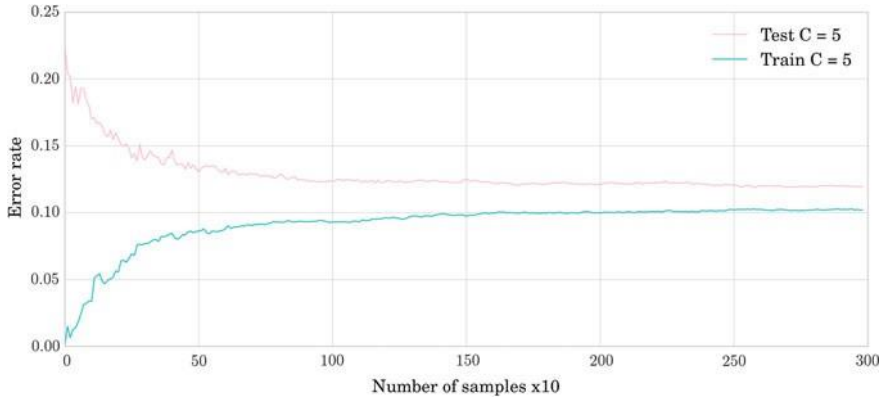


Fig. 5.4 Learning curves (training and test errors) for a model with a high degree of complexity

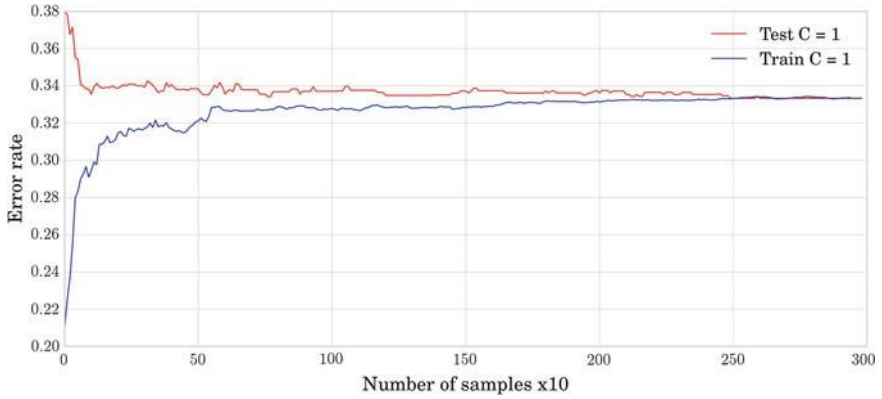


Fig. 5.5 Learning curves (training and test errors) for a model with a low degree of complexity

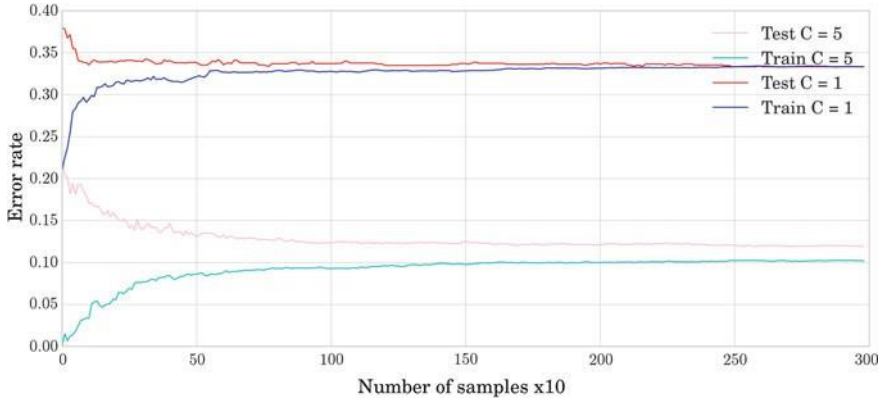
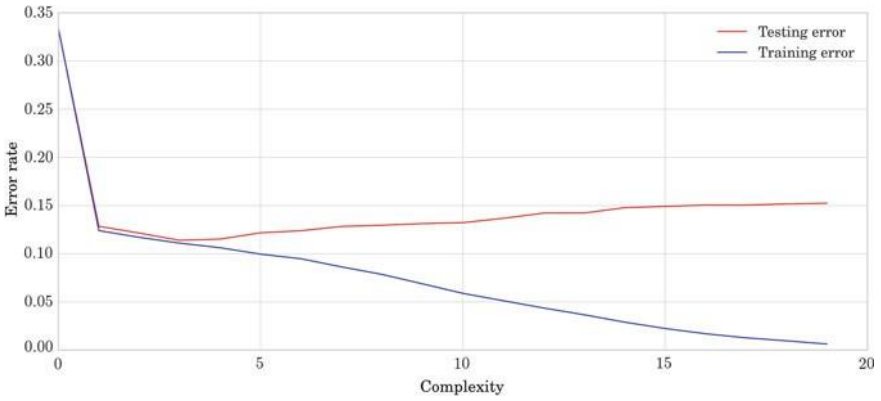


Fig. 5.6 Learning curves (training and test errors) for models with a low and a high degree of complexity



Curves 81

Fig. 5.7 Learning curves (training and test errors) for a fixed number of data samples, as the complexity of the decision tree increases

- With a low degree of complexity, the training and test errors converge to the biassooner/with fewer data.
- Moreover, with a low degree of complexity, the error of convergence is larger thanwith increased complexity.

The value both errors converge towards is also called the *bias*; and the difference between this value and the test error is called the *variance*. The *bias/variance* decomposition of the learning curve is an alternative approach to the training and generalization view.

Let us now plot the learning behavior for a fixed number of examples with respectto the complexity of the model. We may use the same data but now we will changethe maximum depth of the decision tree, which governs the complexity of the model.Observe in Fig. 5.7 that as the complexity increases the training error is reduced; but above a certain level of complexity, the test error also increases. This effect is called *overfitting*. We may enact several cures for overfitting:

- Observe that models are usually parameterized by some hyperparameters. Select-ing the complexity is usually governed by some such parameters. Thus, we are faced with a model selection problem. A good heuristic for selecting the model isto choose the value of the hyperparameters that yields the smallest estimated testerror. Remember that this can be done using cross-validation.
- We may also change the formulation of the objective function to penalize complex models. This is called *regularization*. Regularization accounts for estimating the value of Ω in our out-of-sample error inequality. In other words, it models the complexity of the technique. This usually becomes implicit in the algorithm but has huge consequences in real applications. The most common regularization strategies are as follows:

- L2 weight regularization: Adding an L2 penalization term to the weights of a weight-controlled model implies looking for solutions with small weight values. Intuitively, adding an L2 penalization term can be seen as a surrogate for the notion of smoothness. In this sense, a low complexity model means a very smooth model.
- L1 weight regularization: Adding an L1 regularization term forces sparsity in the weights of the model. In this sense, a low complexity model means a modelwith few components or few active terms.

These terms are added to the objective function. They trade off with the error function in the objective and are governed by a hyperparameter. Thus, we still have to select this parameter by means of model selection.

- We can use "ensemble techniques". A third cure for overfitting is to use ensemble techniques. The best known are *bagging* and *boosting*.

Training, Validation and Test

Going back to our problem, we have to select a model and control its complexity according to the number of training data. In order to do this, we can start by usinga model selection technique. We have seen model selection before when we wantedto compare the performance of different classifiers. In that case, our best bet was to select the classifier with the smallest E_{out}. Analogous to model selection, we may think of selecting the best hyperparameters as choosing the classifier with parameters that performs the best. Thus, we may select a set of hyperparameter values and usecross-validation to select the best configuration.

The process of selecting the best hyperparameters is called *validation*. This intro-duces a new set into our simulation scheme; we now need to divide the data we haveinto three sets: training, validation, and test sets. As we have seen, the process of assessing the performance of the classifier by estimating the generalization error iscalled testing. And the process of selecting a model using the estimation of the gen-eralization error is called validation. There is a subtle but critical difference betweenthe two and we have to be aware of it when dealing with our problem.

- Test data is used exclusively for assessing performance at the end of the processand will never be used in the learning process.[8]
- Validation data is used explicitly to select the parameters/models with the best performance according to an estimation of the generalization error. This is a formof learning.
- Training data are used to learn the instance of the model from a model class.

In practice, we are just given training data, and in the most general case we explicitly have to tune some hyperparameter. Thus, how do we select the different splits?

How we do this will depend on the questions regarding the method that we wantto answer:

- Let us say that our customer asks us to deliver a classifier for a given problem. If we just want to provide the best model, then we may use cross-validation on our training dataset and select the model with the best performance. In this scenario,when we return the trained classifier to our customer, we know that it is the one that achieves the best performance. But if the customer asks about the expected performance, we cannot say anything.
  A practical issue: once we have selected the model, we use the complete trainingset to train the final model.
- If we want to know about the performance of our model, we have to use unseen data. Thus, we may proceed in the following way:

1. Split the original dataset into training and test data. For example, use 30% of the original dataset for testing purposes. This data is held back and will only beused to assess the performance of the method.
2. Use the remaining training data to select the hyperparameters by means of cross-validation.
3. Train the model with the selected parameter and assess the performance using the test dataset.

A practical issue: Observe that by splitting the data into three sets, the classifieris trained with a smaller fraction of the data.

- If we want to make a good comparison of classifiers but we do not care aboutthe best parameters, we may use *nested cross-validation*. Nested cross-validation runs two cross-validation processes. An external cross-validation is used to assessthe performance of the classifier and in each loop of the external cross-validationanother cross-validation is run with the remaining training set to select the best parameters.

If we want to select the best complexity of a decision tree, we can use tenfold cross- validation checking for different complexity parameters. If we change the maximumdepth of the method, we obtain the results in Fig. 5.8.

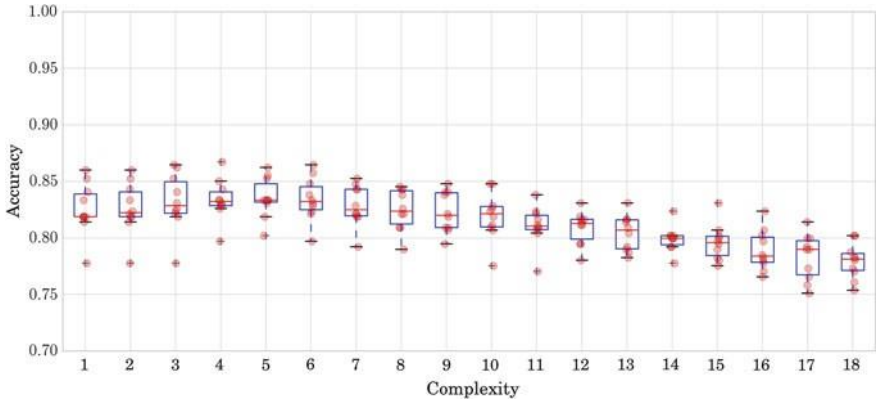


Fig. 5.8 Box plot showing accuracy for different complexities of the decision tree

In [14]:

```
# Create a 10-fold cross-validation set
kf = cross_validation.KFold(n = y.shape[0],
                            n_folds = 10,
                            shuffle = True,
                            random_state = 0)

# Search for the parameter among the following:
C = np.arange(2, 20,)

acc = np.zeros((10, 18))
i = 0
for train_index, val_index in kf:
    X_train, X_val = X[train_index], X[val_index]
    y_train, y_val = y[train_index], y[val_index]
    j = 0
    for c in C:
        dt = tree.DecisionTreeClassifier(
            min_samples_leaf = 1,
            max_depth = c)
        dt.fit(X_train, y_train)
        yhat = dt.predict(X_val)
        acc[i][j] = metrics.accuracy_score(yhat, y_val)
        j = j + 1
    i = i + 1
```

Checking Fig. 5.8, we can see that the best average accuracy is obtained by the fifth model, a maximum depth of 6. Although we can report that the best accuracy is estimated to be found with a complexity value of 6, we cannot say anything about the value it will achieve. In order to have an estimation of that value, we need to runthe model on a new set of data that are completely unseen, both in training and in model selection (the model selection value is positively biased). Let us put everything together. We will be considering a simple train_test split for testing purposes and then run cross-validation for model selection.

In [15]:

```
# Train_test split
X_train, X_test, y_train, y_test = cross_validation
    .train_test_split(X, y, test_size = 0.20)

# Create a 10-fold cross-validation set
kf = cross_validation.KFold(n = y_train.shape[0],
                            n_folds = 10,
                            shuffle = True,
                            random_state = 0)

# Search the parameter among the following
C = np.arange(2, 20,)
acc = np.zeros((10, 18))
i = 0
for train_index, val_index in kf:
    X_t, X_val = X_train[train_index], X_train[val_index]
    y_t, y_val = y_train[train_index], y_train[val_index]
    j = 0
    for c in C:
        dt = tree.DecisionTreeClassifier(
            min_samples_leaf = 1,
            max_depth = c)
        dt.fit(X_t, y_t)
        yhat = dt.predict(X_val)
        acc[i][j] = metrics.accuracy_score(yhat, y_val)
        j = j + 1
    i = i + 1
print 'Mean accuracy: ' + str(np.mean(acc, axis = 0))
print 'Selected model index: ' +
      str(np.argmax(np.mean(acc, axis = 0)))
```

Out[15]:
```
Mean accuracy: [0.8254832  0.83031158 0.83091854 0.83423816
0.83363939 0.83303516 0.82759983 0.82337022 0.82034725
0.81642795 0.80947567 0.79951316 0.80162614 0.79226695
0.79589324 0.785928 0.78049267 0.78320988]
Selected model index: 3
```

If we run the output of this code, we observe that the best accuracy is provided by the fourth model. In this example it is a model with complexity 5.[9] The selectedmodel achieves a success rate of 0.83423816 in validation. We then train the modelwith the complete training set and verify its test accuracy.

In [16]:

```
# Train the model with the complete training set with the
    selected complexity
dt = tree.DecisionTreeClassifier(
    min_samples_leaf = 1,
    max_depth = C[np.argmax(np.mean(acc, axis = 0))])
dt.fit(X_train,y_train)

# Test the model with the test set
yhat = dt.predict(X_test)
print 'Test accuracy: ' +
      str(metrics.accuracy_score(yhat, y_test))
```

Out[16]:Test accuracy: 0.826086956522

As expected, the value is slightly reduced; it achieves 0.82608. Finally, the model is trained with the complete dataset. This will be the model used in exploitation andwe expect to at least achieve an accuracy rate of 0.82608.

In [17]:

```
# Train the final model
dt = tree.DecisionTreeClassifier(min_samples_leaf = 1,
    max_depth = C[np.argmax(np.mean(acc, axis = 0))])
dt.fit(X, y)
```

Two Learning Models

Let us return to our problem and check the performance of different models. There are many learning models in the machine learning literature. However, in this shortintroduction we focus on two of the most important and pragmatically effective approaches[10]: support vector machines (SVM) and random forests (RF).

Generalities Concerning Learning Models

Before going into some of the details of the models selected, let us check the com-ponents of any learning algorithm. In order to be able to learn, an algorithm has to define at least three components:

- *The model class/hypothesis space* defines the family of mathematical models thatwill be used. The target decision boundary will be approximated from one elementof this space. For example, we can consider the class of linear models. In this caseour decision boundary will be a line if the problem is defined in $\mathbf{R}^2$ and the modelclass is the space of all possible lines in $\mathbf{R}^2$.

Model classes define the geometric properties of the decision function. There are different taxonomies but the best known are the families of *linear* and *nonlinear* models. These families usually depend on some parameters; and the solution to a learning problem is the selection of a particular set of parameters, i.e., the selection of an instance of a model from the model class space. The model class space is also called the *hypothesis space*.

The selection of the best model will depend on our problem and what we want to obtain from the problem. The primary goal in learning is usually to achieve the minimum error/maximum performance; but according to what else we want from the algorithm, we can come up with different algorithms. Other common desirable properties are interpretability, behavior when faced with missing data, fast training, etc.

- *The problem model* formalizes and encodes the desired properties of the solution. In many cases, this formalization takes the form of an optimization problem. In its most basic instantiation, the problem model can be the *minimization of an error function*. The error function measures the difference between our model and the target. Informally speaking, in a classification problem it measures how "irritated" we are when our model misses the right label for a training sample. For example, in classification, the ideal error function is the *0–1 loss*. This function takes value 1 when we incorrectly classify a training sample and zero otherwise. In this case, we can interpret it by saying that we are only irritated by "one unit of irritation" when one sample is misclassified.

  The problem model can also be used to impose other constraints on our solution,[11] such as finding a smooth approximation, a model with a low degree of small complexity, a sparse solution, etc.
- *The learning algorithm* is an optimization/search method or algorithm that, given a model class, fits it to the training data according to the error function. According to the nature of our problem there are many different algorithms. In general, we are talking about finding the minimum error approximation or maximum probable model. In those cases, if the problem is convex/quasi-convex we will typically use first- or second-order methods (i.e., gradient descent, coordinate descent, Newton's method, interior point methods, etc.). Other searching techniques such as genetic algorithms or Monte Carlo techniques can be used if we do not have access to the derivatives of the objective function.

Support Vector Machines

SVM is a learning technique initially designed to fit a linear boundary between the samples of a binary problem, ensuring the maximum robustness in terms of tolerance to isotropic uncertainty. This effect is observed in Fig. 5.9. Note that the boundary displayed has the largest distance to the closest point of both classes. Any other

[11] Remember the regularization cure for overfitting.

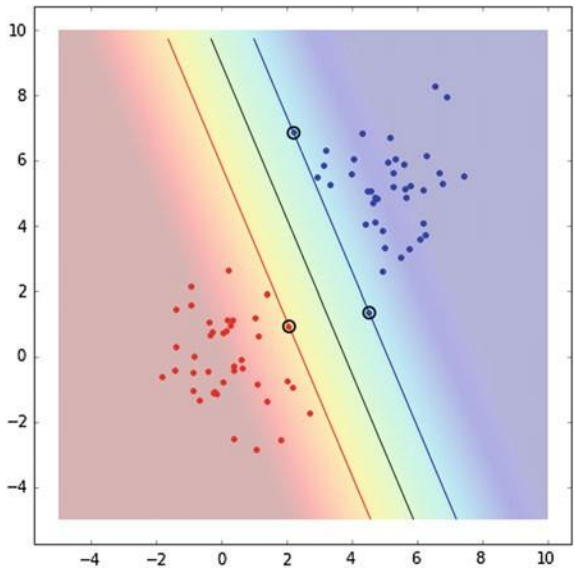


Fig. 5.9 Support vector machine decision boundaryand the support vectors

separating boundary will have a point of a class closer to it than this one. The figure also shows the closest points of the classes to the boundary. These points are called *support vectors*. In fact, the boundary only depends on those points. If we remove any other point from the dataset, the boundary remains intact. However, in general, if any of these special points is removed the boundary will change.

A Brief Note on Deriving Hard Margin Support Vector Machines In order to understand the model, we have to be able to approximately derive its for-mulation. For this purpose it is important to understand a couple of things about basic geometry of a hyperplane. A hyperplane in $\mathbf{R}^d$ is defined as an affine combination of the variables: $\pi \quad a^T x \quad b \quad 0$. A hyperplane splits the space into two half-spaces. The evaluation of the equation of the hyperplane on any element belonging to one of the half-spaces is a positive value. It is a negative value for all the elements in the other half-space. The distance of a point $x \in \mathbf{R}^d$ to the hyperplane π is

$$d(x, \pi) = \frac{|a^T x + b|}{\|a\|_2}$$

Given a binary classification problem with training data $D = \{(x_i, y_i)\}$, $i = 1 \ldots N$, $y_i \in \{+1, -1\}$, consider $S \subseteq D$ the subset of all data points belonging to class +1, $S = \{x_i \,|\, y_i = +1\}$, and $R = \{x_i \,|\, y_i = -1\}$ its complement.

Then the problem of finding a separating hyperplane consists of fulfilling the following constraints[12]

$$a^T s_i + b > 0 \text{ and } a^T r_i + b < 0 \quad \forall s_i \in S, r_i \in R.$$

This is a *feasibility problem* and it is usually written in the following way in optimization standard notation:

$$\begin{aligned} &\text{minimize} \quad 1 \\ &\text{subject to} \quad y_i(a^T x_i + b) \geq 1, \ \forall x_i \in D \end{aligned}$$

The solution of this problem is not unique. Selecting the maximum margin hyper-plane requires us to add a new constraint to our problem. Remember from the geom-etry of the hyperplane that the distance of any point to a hyperplane is given by $d(x, \pi) = \frac{|a^T x + b|}{\|a\|_2}$.

Recall also that we want positive data to be beyond value 1 and negative data below -1. Thus, what is the distance value we want to maximize?

The positive point closest to the boundary is at $1/\|a\|_2$ and the negative point closest to the boundary data point is also at $-1/\|a\|_2$. Thus, data points from differentclasses are at least $2/\|a\|_2$ apart.

Recall that our goal is to find the separating hyperplane with maximum margin, i.e., with maximum distance between elements in the different classes. Thus, we can complete the former formulation with our last requirement as follows:

$$\begin{aligned} &\text{minimize} \quad \|a\|_2/2 \\ &\text{subject to} \quad y_i(a^T x_i + b) \geq 1, \ \forall x_i \in D \end{aligned}$$

This formulation has a solution as long as the problem is linearly separable.

In order to deal with misclassifications, we are going to introduce a new set of variables ξ_i, that represents the amount of violation in the i-th constraint. If the constraint is already satisfied, then $\xi_i = 0$; while $\xi_i > 0$ otherwise. Because ξ_i is related to the errors, we would like to keep this amount as close to zero as possible. This makes us introduce an element in the objective trade-off with the maximum margin.

[12]Note the strict inequalities in the formulation. Informally, we can consider the smallest satisfiedconstraint, and observe that the rest must be satisfied with a larger value. Thus, we can arbitrarilyset that value to 1 and rewrite the problem as

$$a^T s_i + b \geq 1 \text{ and } a^T r_i + b \leq -1.$$

The new model becomes:

$$\begin{aligned} &\text{minimize} \quad \|a\|_2/2 + C \sum_{i=1}^{N} \xi_i \\ &\text{subject to} \quad y_i(a^T x_i + b) \geq 1 - \xi_i, \; i = 1 \ldots N \\ &\qquad \xi_i \geq 0 \end{aligned}$$

where C is the trade-off parameter that roughly balances the rates of margin and misclassification. This formulation is also called *soft-margin SVM*.

The larger the C value is, the more importance one gives to the error, i.e., the method will be more accurate according to the data at hand, at the cost of being moresensitive to variations of the data.

The decision boundary of most problems cannot be well approximated by a linearmodel. In SVM, the extension to the nonlinear case is handled by means of kernel theory. In a pragmatic way, a kernel can be referred to as any function that capturesthe similarity between any two samples in the training set. The kernel has to be a positive semi-definite function as follows:

- *Linear kernel*:

$$k(x_i, x_j) = x^T_i x_j$$

- *Polynomial kernel*:

$$k(x_i, x_j) = (1 + x^T_i x_j)^p$$

- *Radial Basis Function kernel*:

$$k(x_i, x_j) = e^{-\frac{\|x_j - x_j\|^2}{2\sigma^2}}$$

Note that selecting a polynomial or a Radial Basis Function kernel means that we have to adjust a second parameter p or σ, respectively. As a practical summary, theSVM method will depend on two parameters (C, γ) that have to be chosen carefully using cross-validation to obtain the best performance.

Random Forest

Random Forest (RF) is the other technique that is considered in this work. RF is an ensemble technique. Ensemble techniques rely on combining different classifiersusing some aggregation technique, such as majority voting. As pointed out earlier, ensemble techniques usually have good properties for combating overfitting. In this case, the aggregation of classifiers using a voting technique reduces the variance of the final classifier. This increases the robustness of the classifier and usually achievesa very good classification performance. A critical issue in the ensemble of classifiers is that for the combination to be successful, the errors made by the members of the ensemble should be as uncorrelated as possible. This is sometimes referred to in the

literature as the diversity of the classifiers. As the name suggests, the base classifiersin RF are decision trees.

A Brief Note on Decision Trees

A decision tree is one of the most simple and intuitive techniques in machine learning, based on the divide and conquer paradigm. The basic idea behind decision trees is topartition the space into patches and to fit a model to a patch. There are two questionsto answer in order to implement this solution:

- How do we partition the space?
- What model shall we use for each patch?

Tackling the first question leads to different strategies for creating decision tree. However, most techniques share the axis-orthogonal hyperplane partition policy, i.e., a threshold in a single feature. For example, in our problem "Does the applicanthave a home mortgage?". This is the key that allows the results of this method to be interpreted. In decision trees, the second question is straightforward, each patch is given the value of a label, e.g., the majority label, and all data falling in that part ofthe space will be predicted as such.

The RF technique creates different trees over the same training dataset. The word"random" in RF refers to the fact that only a subset of features is available to each of the trees in its building process. The two most important parameters in RF are thenumber of trees in the ensemble and the number of features each tree is allowed tocheck.

Ending the Learning Process

With both techniques in mind, we are going to optimize and check the results usingnested cross-validation. Scikit-learn allows us to do this easily using several model selection techniques. We will use a grid search, GridSearchCV (a cross-validation using an exhaustive search over all combinations of parameters provided).

In [16]:
```
parameters = {'C': [1e4, 1e5, 1e6],
              'gamma': [1e-5, 1e-4, 1e-3]}
N_folds = 5

kf=cross_validation.KFold(n = y.shape[0],
                          n_folds = N_folds ,
                          shuffle = True,
                          random_state = 0)

acc = np.zeros((N_folds,))
i = 0
# We will build the predicted y from the partial predictions
    on the test of each of the folds
yhat = y.copy()
for train_index, test_index in kf:
    X_train, X_test = X[train_index,:], X[test_index,:]
    y_train, y_test = y[train_index], y[test_index]
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    clf = svm.SVC(kernel = 'rbf')
    clf = grid_search.GridSearchCV(clf, parameters, cv = 3)
    clf.fit(X_train, y_train.ravel())
    X_test = scaler.transform(X_test)
    yhat[test_index] = clf.predict(X_test)

print metrics.accuracy_score(yhat, y)
print metrics.confusion_matrix(yhat, y)
```

Out[16]:
```
classification accuracy: 0.856038647343confusion matrix:
3371 590
   6 173
```

The result obtained has a large error in the non-fully funded class (negative). This is because the default scoring for cross-validation grid-search is mean accuracy. Depending on our business, this large error in recall for this class may be unaccept-able. There are different strategies for diminishing the impact of this effect. On the one hand, we may change the default scoring and find the parameter setting that cor-responds to the maximum average recall. On the other hand, we could mitigate thiseffect by imposing a different weight on an error on the critical class. For example,we could look for the best parameterization such than one error on the critical classis equivalent to one thousand errors on the noncritical class. This is important in business scenarios where monetization of errors can be derived.

A Toy Business Case

Consider that clients using our service yield a profit of 100 units per client (we will use abstract units but keep in mind that this will usually be accounted in euros/dollars). We design a campaign with the goal of attracting investors in order to cover all non-fully funded loans. Let us assume that the cost of the campaign is α unitsper client. With this policy we expect to keep our customers

satisfied and engaged with our service, so they keep using it. Analyzing the confusion matrix we can

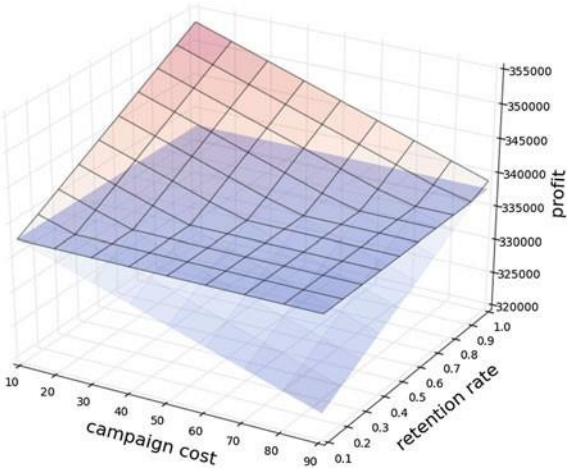


Fig. 5.10 Surfaces for two different campaign and attraction factors. The horizontal plane correspondsto the profit if no campaign is launched. The slanted plane is the profit for a certain confusion matrix

give precise meaning to different concepts in this campaign. The real positive set $(TP + FN)$ consists of the number of clients that are fully funded. According to our assumption, each of these clients generates a profit of 100 units. The total profitis 100 $(TP + FN)$. The campaign to attract investors will be cast considering all the clients we predict are not fully funded. These are those that the classifier predict as negative, i.e., $(FN + TN)$. However, the campaign will only have an effect on the investors/clients that are actually not funded, i.e., TN ; and we expect to attract a certain fraction β of them. After deploying our campaign, a simplified model of theexpected profit is as follows:

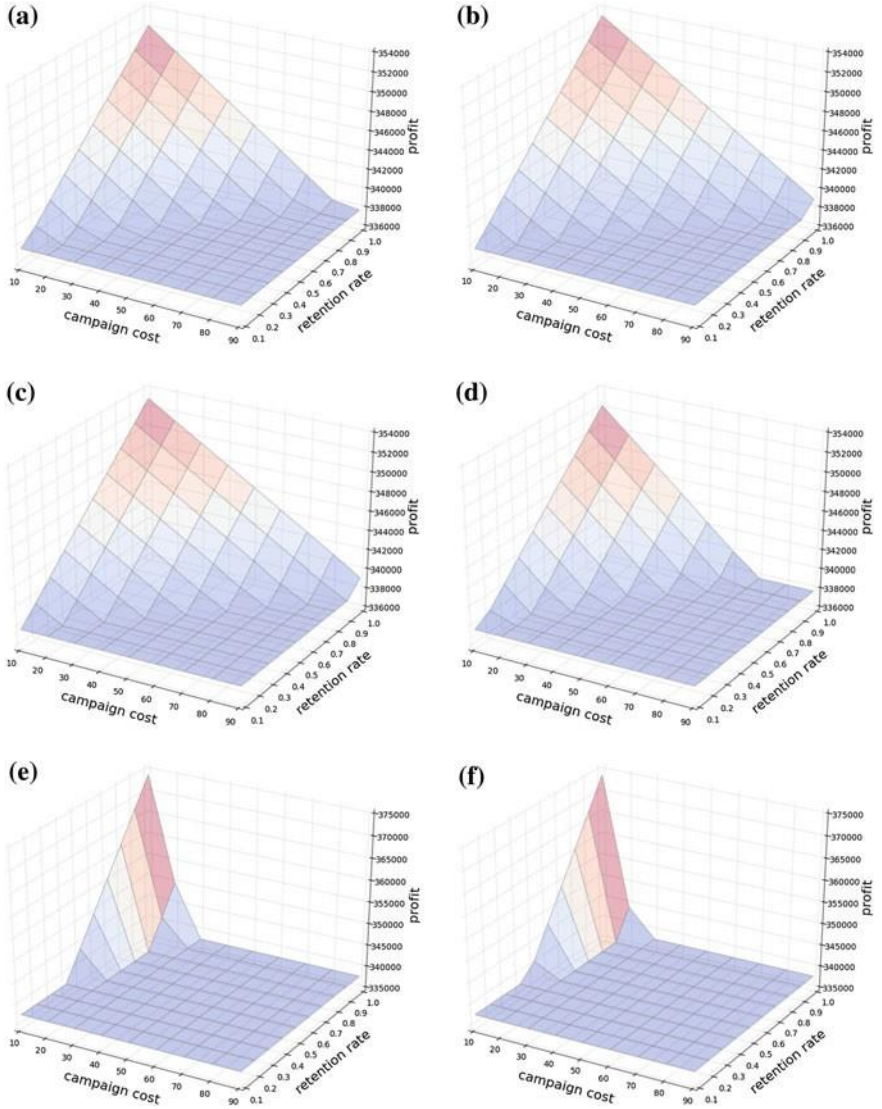
$$100 \cdot (TP + FN) - \alpha(TN + FN) + 100\beta TN$$

When optimizing the classifier for accuracy, we do not consider the business needs. In this case, optimizing an SVM using cross-validation for different parameters of the C and γ, we have an accuracy of 85.60% and a confusion matrix with the followingvalues:

$$\begin{matrix} 3371. & 590. \\ 6. & 173. \end{matrix}$$

If we check how the profit changes for different values of α and β, we obtain the plot in Fig. 5.10. The figure shows two hyperplanes. The horizontal plane is the expected profit if the campaign is not launched, i.e., 100 $(TP + FN)$. The other hyperplanerepresents the profit of the campaign for different values of α and β using a particular classifier. Remember that the cost of the campaign is given by α, and the success rate of the campaign is represented by β. For the campaign to be successful we would like to select values for both parameters so that the profit of the campaign is larger than the cost of launching it. Observe in the figure that certain costs and attraction rates result in losses.

We may launch different classifiers with different configurations and toy with dif-



ferent weights *(2, 4, 8, 16)* for elements of different classes in order to bias the classi

Fig. 5.11 3D surfaces of the profit obtained for different classifiers and configurations of retention campaign cost and retention rate. **a** RF, **b** SVM with the same cost per class, **c** SVM with double cost for the target class, **d** SVM with a cost for the target class equal to 4, **e** SVM with a cost for the target class equal to 8, **f** SVM with a cost for the target class equal to 16

fier towards obtaining different values for the confusion matrix.[13] The weights define

Table 5.1 Different configurations of classifiers and their respective profit rates and accuracies

	Max profit rate (%)	Profit rate at 60% (%)	Accuracy (%)
Random forest	4.41	2.41	87.87
SVM {1 : 1}	4.59	2.54	85.60
SVM {1 : 2}	4.52	2.50	85.60
SVM {1 : 4}	4.30	2.28	83.81
SVM {1 : 8}	10.69	3.57	52.51
SVM {1 : 16}	10.68	2.88	41.40

how much a misclassification in one class counts with respect to a misclassificationin another. Figure 5.11 shows the different landscapes for different configurations of the SVM classifier and RF.

In order to frame the problem, we consider a very successful campaign with a 60% investor attraction rate. We can ask several questions in this scenario:

- What is the maximum amount to be spent on the campaign?
- How much will I gain?
- From all possible configurations of the classifier, which is the most profitable?
- Is it the one with the best accuracy?

Checking the values in Fig. 5.11, we find the results collected in Table 5.1. Observe that the most profitable campaign with 60% corresponds to a classifier that considers the cost of mistaking a sample from the non-fully funded class eight times larger than the one from the other class. Observe also that the accuracy in that case is muchworse than in other configurations.

The take-home idea of this section is that business needs are often not aligned with the notion of accuracy. In such scenarios, the confusion matrix values have specificmeanings. This must be taken into account when tuning the classifier.

may tackle many more different settings. For example, we may have different target labels for a single example; this is called multilabel learning. Or, data can come from streams or be time dependent; in these settings, sequential learning or sequence learning can be the methods of choice. Moreover, each data example can be a non-vector or have a variable size, such as a graph, a tree, or a string. In such scenarios kernel learning or structural learning may be used. During these last years we are also seeing the revival of neural networks under the name of deep learning and achieving impressive results in different domains such as computer vision or natural language processing. Nonetheless, all of these methods will behave as explained in this chapter and most of the lessons learned here can be readily applied to these techniques.

UNIT-4

Regression analysis, Regression: linear regression simple linear regression, multiple & Polynomial regression, Sparse model. Unsupervised learning, clustering, similarity and distances, quality measures of clustering, case study.

Regression Analysis

Introduction

In this chapter, we introduce *regression analysis* and some of its applications in data science. Regression is related to how to make predictions about real-world quantities such as, for instance, the predictions alluded to in the following questions. How does sales volume change with changes in price? How is sales volume affected by the weather? How does the title of a book affect its sales? How does the amount of a drug absorbed vary with the patient's body weight; and does this relationship depend on blood pressure? How many customers can I expect today? At what time should Igo home to avoid traffic jams? What is the chance of rain on the next two Mondays; and what is the expected temperature?

All these questions have a common structure: they ask for a *response* that canbe expressed as a combination of one or more (independent) *variables* (also called*covariates* or *predictors*). The role of regression is to build a model to predict theresponse from the variables. This process involves the transition *from data to model*.More specifically, the model can be useful in different tasks, such as the following:

(1) analyzing the behavior of data (the relation between the response and the vari- ables), (2) predicting data values (whether continuous or discrete), and (3) finding important variables for the model.

In order to understand how a regression model can be suitable for tackling these tasks, we will introduce three practical cases for which we use three real datasets and solve different questions. These practical cases will motivate *simple linear regression*, *multiple linear regression*, and *logistic regression*, as presented in the following sections.

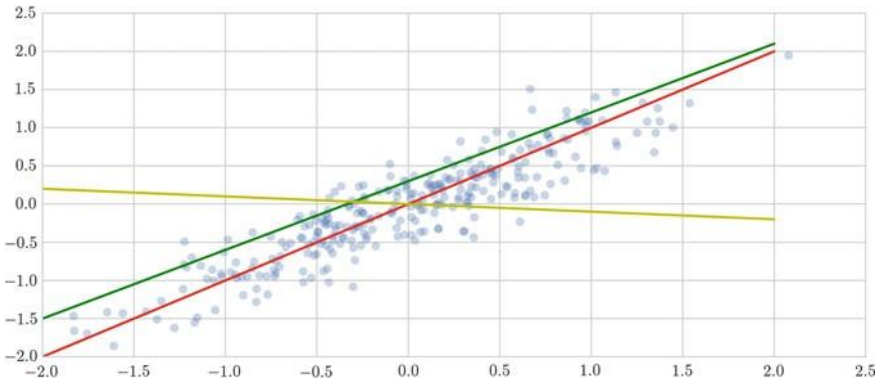


Fig. 6.1 Illustration of different simple linear regression models. *Blue points* correspond to a set of random points sampled from a univariate normal (Gaussian) distribution. *Red*, *green* and *yellow lines* are three different simple linear regression models

Linear Regression

The objective of performing a regression is to build a model to express the relation between the response $\mathbf{y} \in \mathrm{R}^n$ and a combination of one or more (independent) variables $\mathbf{x}_i \in \mathrm{R}^n$. [1] The model allows us to predict the response $\mathbf{y}$ from the variables. The simplest model which can be considered is a *linear model*, where the response $\mathbf{y}$ depends linearly on the d variables $\mathbf{x}_i$:

$$\mathbf{y} = a_1\mathbf{x}_1 + \cdots + a_d\mathbf{x}_d. \tag{6.1}$$

The variables a_i are termed the *parameters* or *coefficients* of the model. This equation can be rewritten in a more compact matrix form: $\mathbf{y} = \mathbf{X}\mathbf{w}$, where

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \mathbf{X} = \begin{pmatrix} x_{11} \ldots x_{1d} \\ x_{21} \ldots x_{2d} \\ \vdots \\ x_{n1} \ldots x_{nd} \end{pmatrix}, \mathbf{w} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_d \end{pmatrix}.$$

Linear regression is the technique for creating these linear models.

Simple Linear Regression

Simple linear regression considers n samples of a single variable $\mathbf{x} \in \mathrm{R}^n$ and describes the relationship between the variable and the response with the model:

$$\mathbf{y} = a_0 + a_1\mathbf{x}, \tag{6.2}$$

where the parameter a_0 is called the *intercept* or the *constant term*.

Given a set of samples $(\mathbf{x}, \mathbf{y})$, such as the set illustrated in Fig. 6.1, we can create a linear model to explain the data, as in Eq. (6.2). But how do we know which is the

best model (best parameters) for this particular set of samples? See the three different models (straight lines in different colors) in Fig. 6.1.

Ordinary least squares (OLS) is the simplest and most common *estimator* in which the parameters (a's) are chosen to minimize the square of the distance between thepredicted values and the actual values with respect to a_0, a_1:

$$||a_0 + a_1\mathbf{x} - \mathbf{y}||^2 = \sum_{j=1}^{n} (a_0 + a_1 x_j - y_j)^2.$$

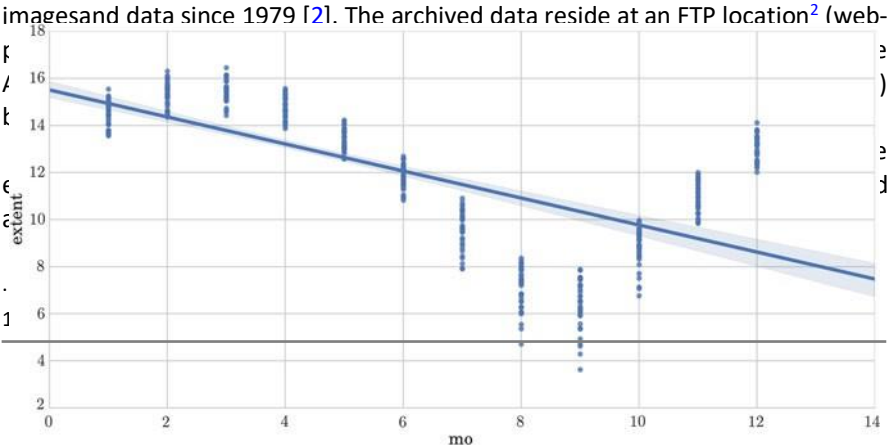
We are concerned here with the y-axis distance, since it does not consider the error in the variables. This error expression is often called the *sum of squared errors of prediction* (SSE). The SSE function is quadratic in the parameters, **w**, with positive-definite Hessian, and therefore this function possesses a unique global minimum at $\mathbf{w} \quad (a_0, a_1)$. The resulting model is represented as follows: $\mathbf{y} \quad a_0 \; a_1\mathbf{x}$, where the hats on the variables represent the fact that they are estimated from the data available.

OLS is a popular approach for several reasons. It makes it computationally cheap to calculate the coefficients. It is also easier to interpret than the other more sophisticated models. In situations where the goal is to understand a simple model in detail, ratherthan to estimate the response well, it can provide insight into what the model captures. Finally, in situations where there is a lot of noise, as in many real scenarios, it maybe hard to find the true functional form, so a constrained model can perform quite well compared to a complex model which can be more affected by noise.

Practical Case: Sea Ice Data and Climate Change

In this practical case, we pose the question: Is the climate really changing? More concretely, we want to show the effect of the climate change by determining whether the *sea ice area* (or *extent*) has decreased over the years. Sea ice area refers to the total area covered by ice, whereas sea ice extent is the area of ocean with at least 15% sea ice. Reliable measurement of sea ice edges began with the satellite era in the late 1970s. Before then, sea ice area and extent were monitored less precisely bya combination of ships, buoys, and aircraft.

We will use the sea ice data from the National Snow & Ice Data Center[1] which provides measurements of the area and extend of sea ice at the poles over the last 36 years. The center has given access to the archived monthly Sea Ice Index imagesand data since 1979 [2]. The archived data reside at an FTP location[2] (web-



0 ession Analysis
0

Fig. 6.2 Ice extent data by month

6

In[1]:

R
e
g

First, we read the data, previously downloaded, and create a DataFrame (*Pandas*) as follows:

```
ice = pd.read_csv('files/ch06/SeaIce.txt',
                  delim_whitespace=True)
print 'shape:', ice.shape
```

Out[1]: shape: (424, 6)

For data cleaning, we check the values of all the fields to detect any potential error. We find that there is a ' 9999' value in the data_type field which should contain 'Goddard' or 'NRTSI-G' (the type of the input dataset). So we can easily clean the data, removing these instances.

In[2]:

```
ice2 = ice[ice.data_type != '-9999']
```

Next, we visualize the data. The lmplot() function from the *Seaborn* toolbox is intended for exploring linear relationships of different forms in multidimensional datasets. For instance, we can illustrate the relationship between the month of the year (variable) and the extent(response) as follows:

In[3]:

```
import Seaborn as sns
sns.lmplot("mo", "extent", ice2)
```

This outputs Fig. 6.2. We can observe a monthly fluctuation of the sea ice extent,as would be expected for the different seasons of the year.

We should normalize the data before performing the regression analysis to avoid this fluctuation and be able to study the evolution of the extent over the years. To capture the variation for a given interval of time (month), we can compute the mean

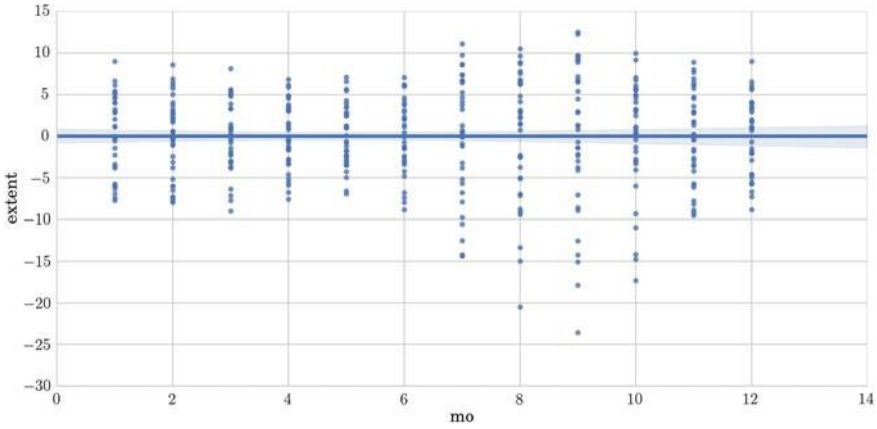


Fig. 6.3 Ice extent data by month after the normalization

for the i-th interval of time (using the period from 1979 through 2014 for the meanextent) μ_i , and subtract it from the set of extent values for that month $\{e_j^i\}$. This value can be converted to a relative percentage difference by dividing it by the totalaverage (1979–2014) μ, and then multiplying by 100:

$$\tilde{e}_j^i = 100 * \frac{e_j^i - \mu_i}{\mu}, i = 1, \ldots, 12.$$

We implement this normalization and plot the relationship again as follows:

In[4]:

```
for i in range(12):
    ice2.extent[ice2.mo == i+1] =
        100*(ice2.extent[ice2.mo == i+1]
            - month_means[i+1])
        /month_means.mean()
sns.lmplot("mo", "extent", ice2)
```

The new output is in Fig. 6.3. We now observe a comparable range of values forall months.

Next, the normalized values can be plotted for the entire time series to analyze the tendency. We compute the trend as a simple linear regression. We use the lmplot() function for visualizing linear relationships between the year (variable) and the extent (response).

In[5]:

```
sns.lmplot("year", "extent", ice2)
```

This outputs Fig. 6.4 showing the regression model fitting the extent data. This plot has two main components. The first is a scatter plot, showing the observed data points. The second is a regression line, showing the estimated linear model relating

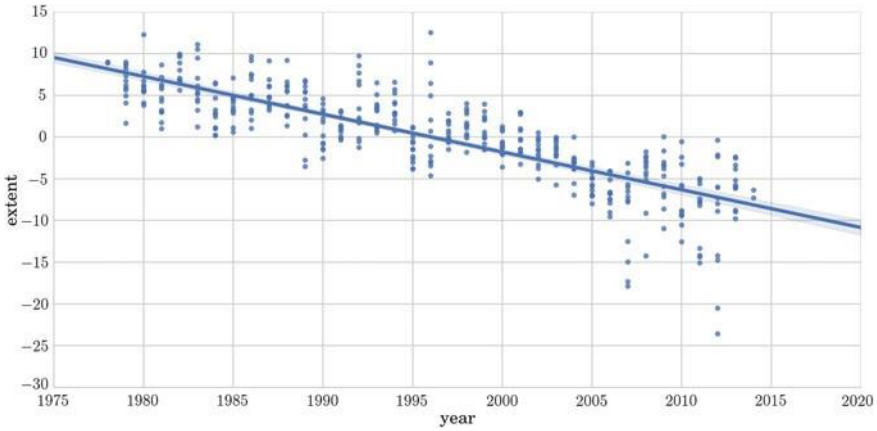


Fig. 6.4 Regression model fitting sea ice extent data for all months by year using lmplot

the two variables. The regression line is plotted with a 95% confidence band to givean impression of the uncertainty in the model.

In this figure, we can observe that the data show a long-term negative trend overyears. The negative trend can be attributed to global warming, although there is alsoa considerable amount of variation from year to year.

Up until here, we have qualitatively shown the linear regression using a useful visu- alization tool. We can also analyze the linear relationship in the data using the *Scikit- learn* library, which allows a quantitative evaluation. As was explained in the previous chapter, Scikit-learn provides an object-oriented interface centered around the con-cept of an estimator. The sklearn.linear_model.LinearRegression estimator sets the state of the estimator based on the training data using the function fit. Moreover, it allows the user to specify whether to fit an intercept term in the object construction. This is done by setting the corresponding constructor argumentsof the estimator object as follows:

In[6]:
```
from sklearn . linear_model import LinearRegression
est = LinearRegression(fit_intercept = True)
```

During the fitting process, the state of the estimator is stored in instance attributes that have a trailing underscore ('_'). For example, the coefficients of a LinearRegression estimator are stored in the attribute coef_. We fit a regres- sion model using years as variables (**x**) and the extent values as the response (**y**).

In[7]:
```
x = ice2[['year']]
y = ice2[['extent']]
est.fit(x, y)
print "Coefficients:", est.coef_
print "Intercept:", est.intercept_
```

Out[7]: Coefficients: [[-0.45275459]]
Intercept: [903.71640207]

Estimators that can generate predictions provide an Estimator.predict method. In the case of regression, Estimator.predict will return the predicted regression values. We can evaluate the model fitting by computing the mean squared error (MSE) and the coefficient of determination (R^2) of the model. The coefficient R^2 is defined as $(1 - u/v)$, with $u = (y - y)^2$ and $v = (y - y)^2$, where y is the mean. The best possible score for R^2 is 1.0, lower values are worse (it can also be negative). These measures can provide a quantitative answer to the question we are facing: Is there a negative trend in the evolution of sea ice extent over recent years? We can perform this analysis for a particular month or for all months together, as done in the following lines:

In [8]:
```
from sklearn import metrics
y_hat = est.predict(x)
print "MSE:", metrics.mean_squared_error(y_hat, y)
print "R^2:", metrics.r2_score(y_hat, y)
print 'var:', y.var()
```

Out[8]: MSE: 10.5391316398
R^2: 0.50678703821
var: 31.98324

The negative trend seen in Fig. 6.4 is validated by the MSE value which is small, 0.1%, and the R^2 value which is acceptable, given the variance of the data, 0.3%.

Given the model, we can also predict the extent value for the coming years. For instance, the predicted extent for January 2025 can be computed as follows:

In [9]:
```
x = [2025]
y_hat = model.predict(x)
m = 1 # January
y_hat = (y_hat*month_means.mean()/100) + month_means[m]
print "Prediction of extent for January 2025
      (in millions of square km):", y_hat
```

Out[9]: Prediction of extent for January 2025 (in millions of squarekm): [12.93603933].

Multiple Linear Regression and Polynomial Regression

As we have seen in the previous section, with simple linear regression we describe the relationship between the variable and the response with a straight line. In the case of *multiple linear regression*, we extend this idea by fitting a d-dimensional hyperplane to our d variables, as defined in Eq. (6.1).

Multiple linear regression may seem a very simple model, but even when the response depends on the variables in nonlinear ways, this model can still be used by

considering nonlinear transformations $\varphi(\cdot)$ of the variables:

$$\mathbf{y} = a_1\varphi(\mathbf{x}_1) + \cdots + a_d\varphi(\mathbf{x}_d)$$

This model is called *polynomial regression* and it is a popular nonlinear regression technique which models the relationship between the response and the variablesas an *p*-th order polynomial. The higher the order of the polynomial, the more complex the functions you can fit. However, using higher-order polynomial can involve *computational complexity* and *overfitting*. Overfitting occurs when a model fits the characteristics of the training data and loses the capacity to generalize fromthe seen to predict the unseen.

Sparse Model

Often, in real problems, there are uninformative variables in the data which prevent proper modeling of the problem and thus, the building of a correct regression model. In such cases, a feature selection process is crucial to select only the informative features and discard non-informative ones. This can be achieved by *sparse methods* which use a penalization approach, such as *LASSO* (least absolute shrinkage and selection operator) to set some model coefficients to zero (thereby discarding thosevariables). Sparsity can be seen as an application of Occam's razor: prefer simpler models to complex ones.

Given the set of samples $(\mathbf{X}, \mathbf{y})$, the objective of a sparse model is to minimize the SSE through a restriction (or penalty):

$$\frac{1}{2n}||\mathbf{Xw} - \mathbf{y}||_2^2 + \alpha||\mathbf{w}||_1,$$

where $||\mathbf{w}||_1$ is the $L1$-norm of the parameter vector $\mathbf{w} = (a_0, \ldots, a_d)$.

Practical Case: Prediction of the Price of a New Housing Market

In this practical case we want to solve the question: Can we predict the price of a new market given any of its attributes?

We will use the Boston housing dataset from Scikit-learn, which provides recorded measurements of 13 attributes of housing markets around Boston, as well as the median house price.[3] Once we load the dataset (506 instances), the description of the dataset can easily be shown by printing the field DESCR. The data (**x**), feature names, and target (**y**) are stored in other fields of the dataset.

We first consider the task of predicting median house values in the Boston area using as the variable one of the attributes, for instance, LSTAT, defined as the "pro-portion of lower status of the population".

Seaborn visualization can be used to show this linear relationships easily:

[3]Copy of UCI ML housing dataset: http://archive.ics.uci.edu/ml/datasets/Housing.

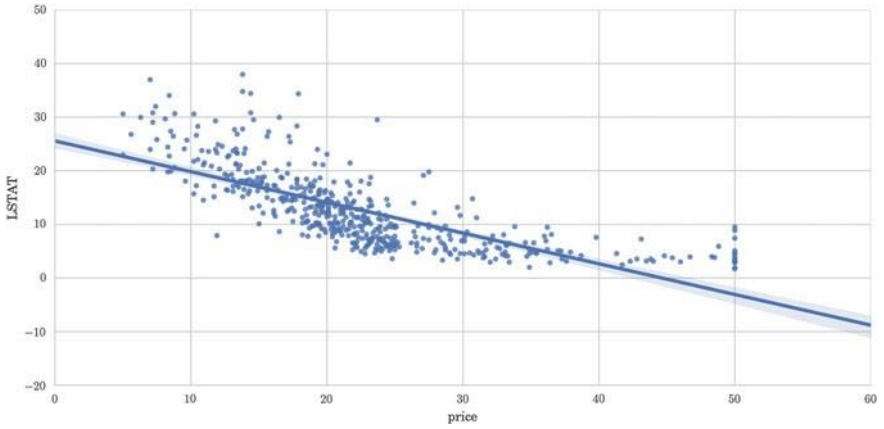


Fig. 6.5 Scatter plot of Boston data (LSTATversus price) and their linear relationship (using lmplot)

In [10]:
```
from sklearn import datasets
boston = datasets.load_boston()
X_boston, y_boston = boston.data, boston.target
print 'Shape of data:', X_boston.shape, y_boston.shape
print 'Feature names:',boston.feature_names
df_boston = pd.DataFrame(boston.data,
                         columns = boston.feature_names)
df_boston['price'] = boston.target
sns.lmplot("price", "LSTAT", df_boston)
```

Out[10]:Shape of data: (506L, 13L) (506L,)
Feature names: ['CRIM' 'ZN' 'INDUS' 'CHAS' 'NOX' 'RM' 'AGE'
'DIS' 'RAD' 'TAX' 'PTRATIO' 'B' 'LSTAT']

In Fig. 6.5, we can clearly see that the relationship between price and LSTAT is nonlinear, since the straight line is a poor fit. We can examine whether a better fit can be obtained by including higher-order terms. For example, a quadratic model:

$$\mathbf{y}_i \approx a_0 + a_1\mathbf{x}_i + a_2\mathbf{x}^2{}_i$$

The lmplotfunction allows to easily change the order of the model as is done in the next code, which outputs Fig. 6.6, where we observe a better fit.

In [11]:
```
sns.lmplot("price", "LSTAT", df_boston, order = 2)
```

To study the relation among multiple variables in a dataset, there are different options. We can study the relationship between several variables in a dataset by using the functions corr and heatmap which allow to calculate a correlation matrix for a dataset and draws a *heat map* with the correlation values. The heat mapis a matricial image which helps to interpret the correlations among variables. For the sake of visualization, we do not consider all the 13 variables in the Boston housing data, but six: CRIM, per capita crime rate by town; INDUS, proportion of non-retail

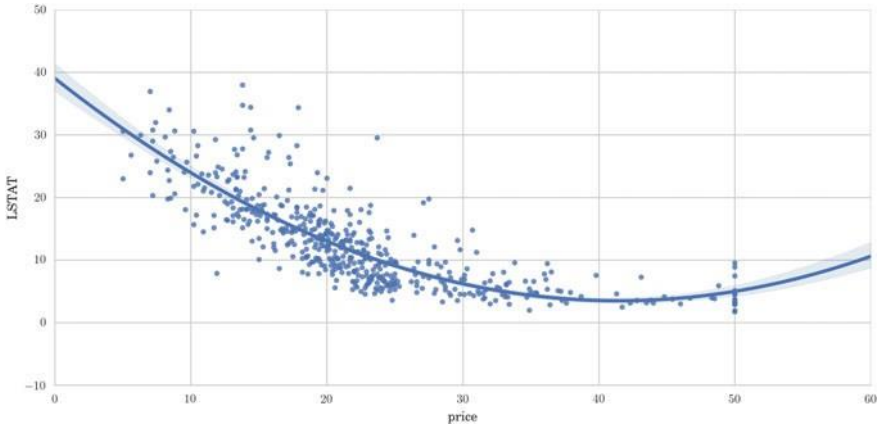


Fig. 6.6 Scatter plot of Boston data (LSTAT versus price) and their polynomial relationship(using lmplotwith order 2)

business acres per town; NOX, nitric oxide concentrations (parts per 10 million); RM,average number of rooms per dwelling; AGE, proportion of owner-occupied units built prior to 1940; and LSTAT. These variables are indicated by their indexes in thefollowing code:

In[12]:
```
indexes = [0,2,4,5,6,12]
df2 = pd.DataFrame(boston.data[:,indexes],
                   columns = boston.feature_names[indexes])
df2['price'] = boston.target
corrmat = df2.corr()
sns.heatmap(corrmat, vmax = .8, square = True)
```

Figure 6.7 shows a heat map representing the correlation between pairs of variables; specifically, the six variables selected and the price of houses. The color bar shows the range of values used in the matrix. This plot is a useful way of summa- rizing the correlation of several variables. It can be seen that LSTAT and RM are the variables that are most correlated with price.

Another good way to explore multiple variables is the *scatter plot* from Pandas. The scatter plot is a grid of plots of multiple variables one against the others, illus-trating the relationship of each variable with the rest. For the sake of visualization,we do not consider all the variables, but just three: RM, AGE, and LSTAT defined by indexesin the following code:

In[13]:
```
indexes =[5,6,12]
df2 = pd.DataFrame(boston.data[:,indexes],
                   columns = boston.feature_names[indexes])
df2['price'] = boston.target
pd.scatter_matrix(df2, figsize = (12.0, 12.0))
```

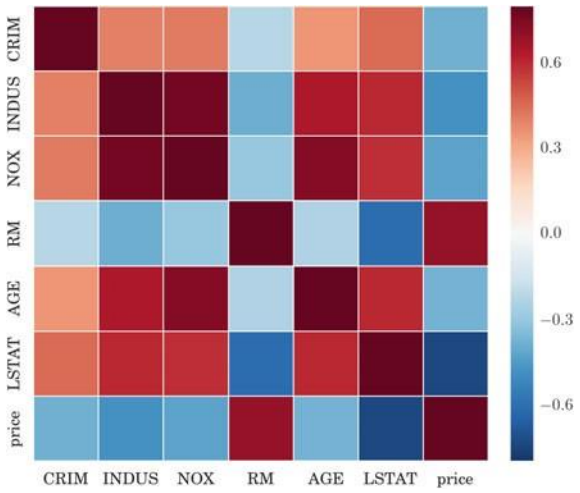


Fig. 6.7 Correlation plot: heat map representing thecorrelation between sevenpairs of variables in the Boston housing dataset

This code outputs Fig. 6.8, where we obtain visual information concerning the density function for every variable, in the diagonal, as well as the scatter plots of thedata points for pairs of variables. In the last column, we can appreciate the relationbetween the three variables selected and house prices. It can be seen that RMfollowsa linear relation with price; whereas AGE does not. LSTAT follows a higher-order relation with price. This plot gives us an indication of how good or bad every attribute would be as a variable in a linear model.

For the evaluation of the prediction power of the model with new samples, we split the data into a training set and a testing set, and we compute the linear regression score, which returns the coefficient of determination R^2 of the prediction. We can also calculate the MSE.

In [14]:
```
from sklearn import linear_model
train_size = X_boston.shape[0]/2
X_train = X_boston[:train_size]
X_test = X_boston[train_size:]
y_train = y_boston[:train_size]
y_test = y_boston[train_size:]
print 'Training and testing set sizes',
      X_train.shape, X_test.shape
regr = LinearRegression()
regr.fit(X_train, y_train)
print 'Coeff and intercept:',
      regr.coef_, regr.intercept_
print 'Testing Score:', regr.score(X_test, y_test) print '
    Training
MSE: ',
      np.mean((regr.predict(X_train) - y_train)**2)
print 'Testing MSE: ',
      np.mean((regr.predict(X_test) - y_test)**2)
```

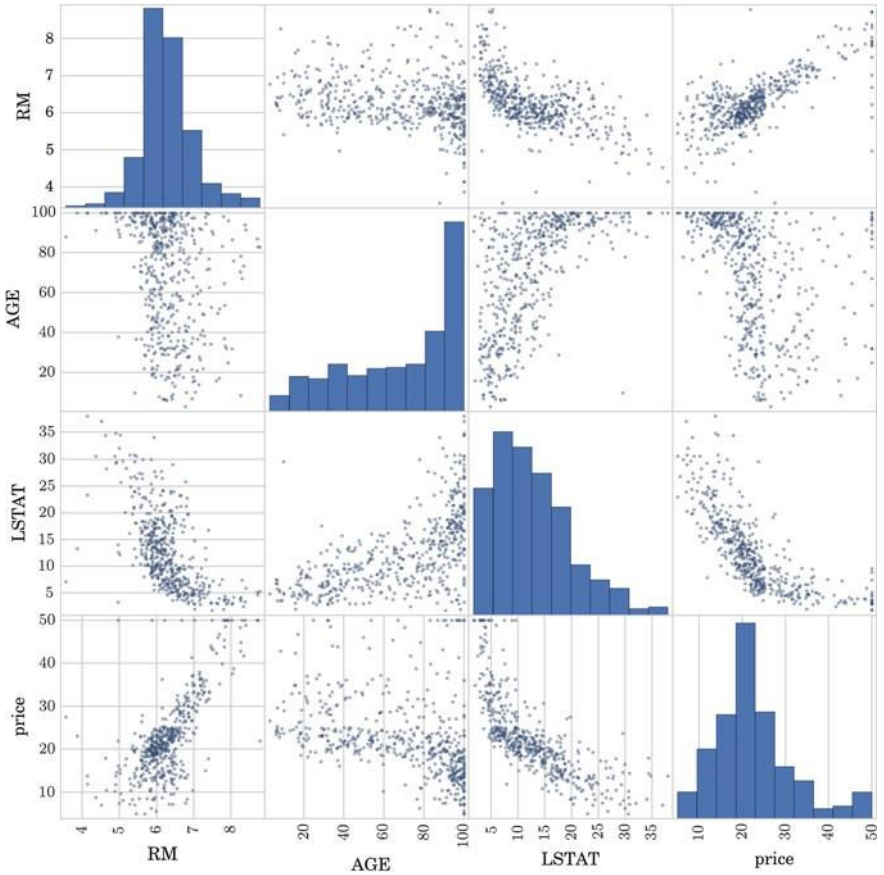


Fig. 6.8 Scatter plot of Boston housing dataset

Out[14]:Training and testing set sizes (253, 13) (253, 13)
Coeff and intercept: [1.20133313 0.02449686 0.00999508
0.42548672 -8.44272332 8.87767164 -0.04850422 -1.11980855
0.20377571 -0.01597724 -0.65974775 0.01777057 -0.11480104]
-10.0174305829
Testing Score: -2.24420202674
Training MSE: 9.98751732546
Testing MSE: 302.64091133

We can see that all the coefficients obtained are different from zero, meaning that no variable is discarded. Next, we try to build a sparse model to predict the price using the most important factors and discarding the non-informative ones. To do this, we can create a LASSO regressor, forcing zero coefficients.

In [15]:
```
regr_lasso = linear_model.Lasso(alpha = .3)
regr_lasso.fit(X_train, y_train) print 'Coeff and intercept:
    ',regr_lasso.coef_
print 'Tesing Score:', regr_lasso.score(X_test,
y_test) print 'Training MSE: ',
      np.mean((regr_lasso.predict(X_train) - y_train)**2)
print 'Testing MSE: ',
      np.mean((regr_lasso.predict(X_test) - y_test)**2)
```

Out[15]:
```
Coeff and intercept: [ 0. 0.01996512 -0. 0. -0. 7.69894744
-0.03444803 -0.79380636 0.0735163 -0.0143421 -0.66768539
0.01547437 -0.22181817] -6.18324183615
Testing Score: 0.501127529021
Training MSE: 10.7343110095
Testing MSE: 46.5381680949
```

It can now be seen that the result of the model fitting for a set of sparse coefficientsis much better than before (using all the variables), with the score increasing from
−2.24 to 0.5. This demonstrates that four of the initial variables are not importantfor the prediction and in fact they confuse the regressor.

With the LASSO result, we can also emphasize the most important factors for determining the price of a new market, based on the coefficient values:

In [16]:
```
ind = np.argsort(np.abs(regr_lasso.coef_))
print 'Ordered variable (from less to more important):',
      boston.feature_names[ind]
```

Out[16]:
```
Ordered variable (from less to more important): ['CRIM' 'INDUS' 'CHAS' 'NOX' 'TAX' 'B' 'ZN' 'AGE'
'RAD' 'LSTAT' 'PTRATIO' 'DIS''RM']
```

There are also other strategies for feature selection. For instance, we can select the k 5 best features, according to the k highest scores, using the function SelectKBestfrom Scikit-learn:

In [17]:
```
import sklearn.feature_selection as fs
selector = fs.SelectKBest(score_func = fs.f_regression,
                          k = 5)
selector.fit_transform(X_train, y_train) per
selector.fit(X_train,y_train)
print 'Selected features:',
      zip(selector.get_support(), boston.feature_names)
```

Out[17]:
```
Selected features: [(False, 'CRIM'), (False, 'ZN'), (True,
'INDUS'), (False, 'CHAS'), (False, 'NOX'), (True, 'RM'), (True,
'AGE'), (False, 'DIS'), (False, 'RAD'), (False, 'TAX'), (True,'PTRATIO'), (False, 'B'), (True, 'LSTAT')]
```

The set of selected features is now different, since the criterion has changed. However, three of the most important features: RM, PTRATIO, and LSTAT.

In order to evaluate the prediction, it could be interesting to visualize the targetand predicted responses in a scatter plot, as it is done in the next code:

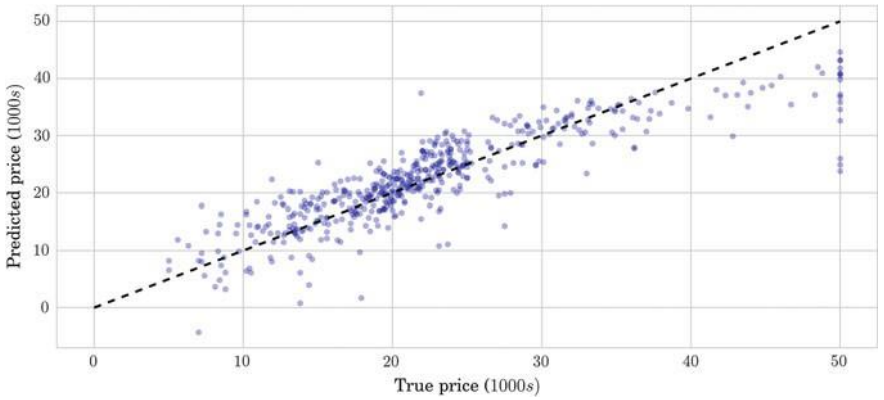


Fig. 6.9 Relation between true (x-axis) and predicted (y-axis) prices

In [18]:

```
clf = LinearRegression()
clf.fit(boston.data, boston.target)
predicted = clf.predict(boston.data)
plt.scatter(boston.target, predicted, alpha = 0.3)
plt.plot([0, 50], [0, 50], '--k')
plt.axis('tight')
plt.xlabel('True price ($1000s)')
plt.ylabel('Predicted price ($1000s)')
```

The output is shown in Fig. 6.9, where we can observe that the original prices are properly estimated by the predicted ones, except for the higher values, around
$50.000 (points in the top right corner).

Finally, it is worth noting that we can work with statistical evaluation of a linearregression with the *OLS* toolbox of the *Stats Model* toolbox.[4] This toolbox is usefulto study several statistics concerning the regression model. To know more about thetoolbox, go to the Documentation related to Stats Models.

Logistic Regression

Logistic regression is a type of model of probabilistic statistical classification. It is used as a binary model to predict a binary response, the outcome of a categorical dependent variable (i.e., a class label), based on one or more variables.

The form of the logistic function is:

$$f(x) = \frac{1}{1 + e^{-\lambda x}}$$

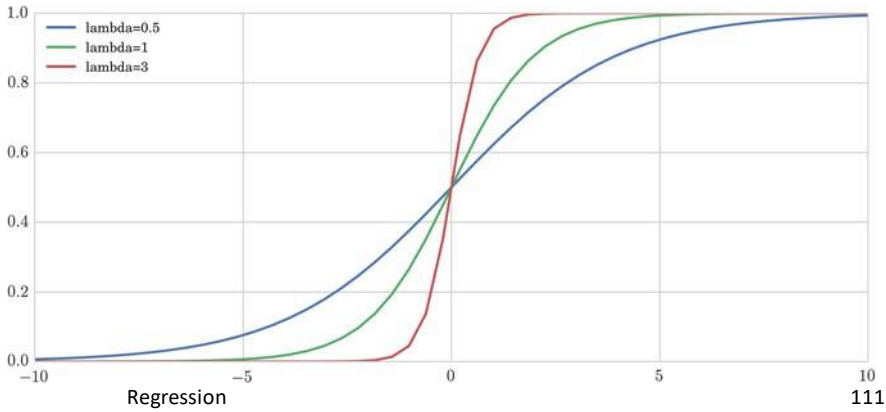


Fig. 6.10 Logistic function for different lambda values

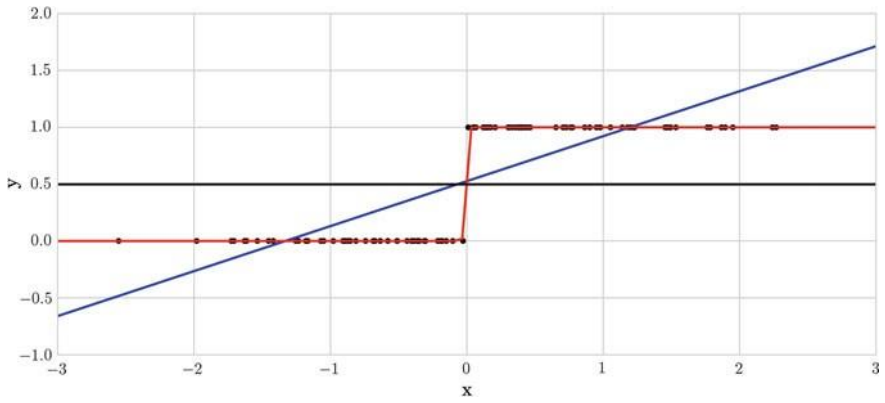


Fig. 6.11 Linear regression (*blue*) versus logistic regression (*red*) for fitting a set of data (*black points*) normally distributed across the 0 and 1 y-values

Figure 6.10 illustrates the logistic function with different values of λ. This function is useful because it can take as its input any value from negative infinity to positive infinity, whereas the output is restricted to values between 0 and 1 and hence can beinterpreted as a probability.

The set of samples (**X**, **y**), illustrated as black points in Fig. 6.11, defines a fitting problem suitable for a logistic regression. The blue and red lines show the fitting result for linear and logistic models, respectively. In this case, a logistic model can clearly explain the data; whereas a linear model cannot.

Practical Case: Winning or Losing Football Team

Now, we pose the question: What number of goals makes a football team the winner or the loser? More concretely, we want to predict victory or defeat in a football match when we are given the number of goals a team scores. To do this we consider

the set of results of the football matches from the Spanish league[5] and we build a classification model with it.

We first read the data file in a DataFrame and select the following columns in a new DataFrame: HomeTeam, AwayTeam, FTHG (home team goals), FTAG (away team goals), and FTR (H=home win, D draw, A away win). We then build a *d*-dimensional vector of variables with all the scores, **x**, and a binary response indicating victory or defeat, **y**. For that, we create two extra columns containing W the number of goals of the winning team and L the number of goals of the losing team and we concatenate these data. Finally, we can compute and visualize a logistic regression model to predict the discrete value (victory or defeat) using these data.

In [19]:

```
from sklearn.linear_model import LogisticRegression
data = pd.read_csv('files/ch06/SP1.csv')
s = data[['HomeTeam','AwayTeam', 'FTHG', 'FTAG', 'FTR']]
def my_f1(row):
    return max(row['FTHG'], row['FTAG'])
def my_f2(row):
    return min(row['FTHG'], row['FTAG'])
s['W'] = s.apply(my_f1, axis = 1)
s['L'] = s.apply(my_f2, axis = 1)
x1 = s['W'].values
y1 = np.ones(len(x1), dtype = np.int)
x2 = s['L'].values
y2 = np.zeros(len(x2), dtype = np.int)
x = np.concatenate([x1, x2])
x = x[:, np.newaxis]
y = np.concatenate([y1, y2])
logreg = LogisticRegression()
logreg.fit(x, y)
X_test = np.linspace(-5, 10, 300)
def lr_model(x):
    return 1 / (1+np.exp(-x))
loss = lr_model(X_test*logreg.coef_ + logreg.intercept_)
    .ravel()
X_test2 = X_test[:,np.newaxis]
losspred = logreg.predict(X_test2)
plt.scatter(x.ravel(), y,
            color = 'black',
            s = 100, zorder = 20,
            alpha = 0.03)
plt.plot(X_test, loss, color = 'blue', linewidth = 3)
plt.plot(X_test, losspred, color = 'red', linewidth = 3)
```

Figure 6.12 shows a scatter plot with transparency so we can appreciate the overlapping in the discrete positions of the total numbers of victories and defeats. It also shows the fitting of the logistic regression model, in blue, and prediction of the logistic regression model, in red, for the Spanish football league results. With this information we can estimate that the cutoff value is 1. This means that a team, in general, has to score more than one goal to win.

[5] http://www.football-data.co.uk/mmz4281/1213/SP1.csv.

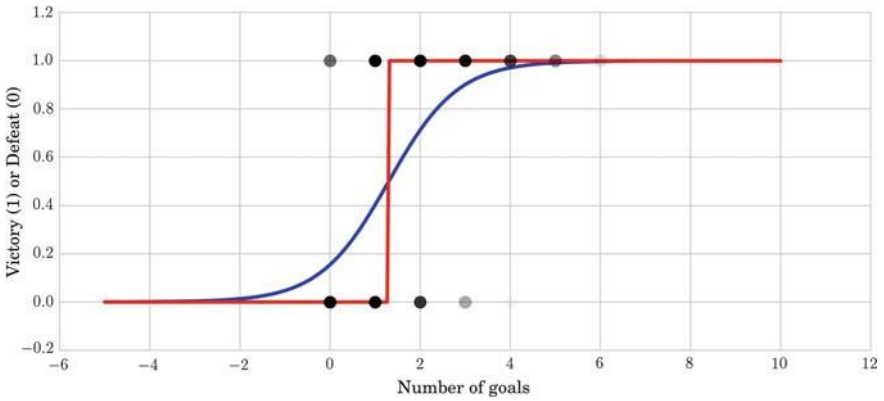


Fig. 6.12 Fitting of the logistic regression model (*blue*) and prediction of the logistic regression model (*red*) for the Spanish football league results

Unsupervised Learning

Introduction

In machine learning, the problem of unsupervised learning is that of trying to find hidden structure in unlabeled data. Since the examples given to the learner are unla- beled, there is no error or reward signal to evaluate the goodness of a potential solution. This distinguishes unsupervised from supervised learning. Unsupervised learning is defined as the task performed by algorithms that learn from a training setof unlabeled or unannotated examples, using the features of the inputs to categorizethem according to some geometric or statistical criteria.

Unsupervised learning encompasses many techniques that seek to summarize and explain key features or structures of the data. Many methods employed in unsuper- vised learning are based on data mining methods used to preprocess data. Most unsupervised learning techniques can be summarized as those that tackle the follow-ing four groups of problems:

- *Clustering*: has as a goal to partition the set of examples into groups.
- *Dimensionality reduction*: aims to reduce the dimensionality of the data. Here, weencounter techniques such as Principal Component Analysis (PCA), independentcomponent analysis, and nonnegative matrix factorization.
- *Outlier detection*: has as a purpose to find unusual events (e.g., a malfunction),that distinguish part of the data from the rest according to certain criteria.
- *Novelty detection*: deals with cases when changes occur in the data (e.g., in stream-ing data).

The most common unsupervised task is clustering, which we focus on in this chapter.

Clustering

Clustering is a process of grouping similar objects together; i.e., to partition unlabeled examples into disjoint subsets of clusters, such that:

- Examples within a cluster are similar (in this case, we speak of *high intraclass similarity*).
- Examples in different clusters are different (in this case, we speak of *low interclass similarity*).

When we denote data as similar and dissimilar, we should define a measure for this similarity/dissimilarity. Note that grouping similar data together can help in discov- ering new categories in an unsupervised manner, even when no sample category labels are provided. Moreover, two kinds of inputs can be used for grouping:

(a) in *similarity-based clustering*, the input to the algorithm is an $n \times n$ *dissimilarity matrix* or *distance matrix*;
(b) in *feature-based clustering*, the input to the algorithm is an $n \times D$ *feature matrix* or *design matrix*, where n is the number of examples in the dataset and D the dimensionality of each sample.

Similarity-based clustering allows easy inclusion of domain-specific similarity, while feature-based clustering has the advantage that it is applicable to potentially noisy data.

Therefore, several questions regarding the clustering process arise.

- What is a natural grouping among the objects? We need to define the “groupness” and the “similarity/distance” between data.
- How can we group samples? What are the best procedures? Are they efficient? Are they fast? Are they deterministic?
- How many clusters should we look for in the data? Shall we state this numbera priori? Should the process be completely data driven or can the user guide the grouping process? How can we avoid “trivial” clusters? Should we allow final clustering results to have very large or very small clusters? Which methods work when the number of samples is large? Which methods work when the number ofclasses is large?
- What constitutes a good grouping? What objective measures can be defined to evaluate the quality of the clusters?

There is not always a single or optimal answer to these questions. It used to be said that clustering is a “subjective” issue. Clustering will help us to describe, analyze, and gain insight into the data, but the quality of the partition depends to a great extent on the application and the analyst.

Similarity and Distances

To speak of similar and dissimilar data, we need to introduce a notion of the similarity of data. There are several ways for modeling of similarity. A simple way to model this is by means of a Gaussian kernel:

$$s(a, b) = e^{-\gamma d(a,b)}$$

where $d(a, b)$ is a metric function, and γ is a constant that controls the decay of the function. Observe that when $a=b$, the similarity is maximum and equal to one. On the contrary, when a is very different to b, the similarity tends to zero. The former modeling of the similarity function suggests that we can use the notion of distance as a surrogate. The most widespread distance metric is the *Minkowski distance*:

$$d(a, b) = \left(\sum_{i=1}^{d} |a_i - b_i|^p \right)^{1/p}$$

where $d(a, b)$ stands for the distance between two elements $a, b \in \mathrm{R}^d$, d is the dimensionality of the data, and p is a parameter.

The best-known instantiations of this metric are as follows:

- when $p = 2$, we have the *Euclidean distance*,
- when $p = 1$, we have the *Manhattan distance*, and
- when $p = \inf$, we have the *max-distance*. In this case, the distance corresponds tothe component $|a_i - b_i|$ with the highest value.

What Constitutes a Good Clustering? Defining Metricsto Measure Clustering Quality

When performing clustering, the question normally arises: How do we measure thequality of the clustering result? Note that in unsupervised clustering, we do not have groundtruth labels that would allow us to compute the accuracy of the algorithm. Still, there are several procedures for assessing quality. We find two families of techniques: those that allow us to compare clustering techniques, and those that check on specific properties of the clustering, for example “compactness”.

Rand Index, Homogeneity, Completeness and V-measureScores

One of the best-known methods for comparing the results in clustering techniques in statistics is the *Rand index* or Rand measure (named after William M. Rand). TheRand index evaluates the similarity between two results of data clustering. Since in unsupervised clustering, class labels are not known, we use the Rand index to compare the coincidence of different clusterings obtained by different approaches or criteria. As an alternative, we later discuss the

Silhouette coefficient: instead of

comparing different clusterings, this evaluates the compactness of the results of applying a specific clustering approach.

Given a set of n elements $S = \{o_1, \ldots, o_n\}$ we can compare two partitions of S[1]: $X = \{X_1, \ldots, X_r\}$, a partition of S into r subsets; and $Y = \{Y_1, \ldots, Y_s\}$, a partition of S into s subsets. Let us use the annotations as follows:

- a is the number of pairs of elements in S that are in the same subset in both X and Y;
- b is the number of pairs of elements in S that are in different subsets in both X and Y;
- c is the number of pairs of elements in S that are in the same subset in X, but indifferent subsets in Y; and
- d is the number of pairs of elements in S that are in different subsets in X, but inthe same subset in Y.

The Rand index, R, is defined as follows:

$$R = \frac{a+b}{a+b+c+d},$$

ensuring that its value is between 0 and 1.

One of the problems of the Rand index is that when given two datasets with random labelings, it does not take a constant value (e.g., zero) as expected. Moreover, when the number of clusters increases it is desirable that the upper limit tends to the unity. To solve this problem, a form of the Rand index, called the *Adjusted Rand index*, is used that adjusts the Rand index with respect to chance grouping of elements. It is defined as follows:

$$AR = \frac{\binom{n}{2}(a+d) - [(a+b)(a+c)+(c+d)(b+d)]}{\binom{n}{2}^2 - [(a+b)(a+c)+(c+d)(b+d)]}.$$

Another way for comparing clustering results is the V-measure. Let us first introduce some concepts. We say that a clustering result satisfies a *homogeneity* criterion if all of its clusters contain only data points which are members of the same original (single) class. A clustering result satisfies a *completeness* criterion if all the data points that are members of a given class are elements of the same predicted cluster. Note that both scores have real positive values between 0.0

In[1]:
```
print("%.3f" % metrics.homogeneity_score([0, 0, 1, 1],
                                         [0, 0, 0, 0]))
```

Out[1]: 0.000

.

The homogeneity is 0 since the samples in the predicted cluster 0 come fromoriginal cluster 0 and cluster 1.

In[2]:

```
print metrics.completeness_score([0, 0, 1, 1],
                                 [1, 1, 0, 0])
```

Out[2]: 1.0

The completeness is 1 since all the samples from the original cluster with label 0 go into the same predicted cluster with label 1, and all the samples from the originalcluster with label 1 go into the same predicted cluster with label 0.

However, how can we define a measure that takes into account the completenessas well as the homogeneity? The *V-measure* is the harmonic mean between the homogeneity and the completeness defined as follows:

$$v = 2 * (\text{homogeneity} * \text{completeness}) / (\text{homogeneity} + \text{completeness}).$$

Note that this metric is not dependent of the absolute values of the labels: a permutation of the class or cluster label values will not change the score value in any way. Moreover, the metric is symmetric with respect to switching between the predicted and the original cluster label. This is very useful to measure the agreement of two independent label assignment strategies applied to the same dataset even when the real groundtruth is not known. If class members are completely split across different clusters, the assignment is totally incomplete, hence the V-measure is null:

In [3]:
```
print("%.3f" % metrics.v_measure_score([0, 0, 0, 0],
                                        [0, 1, 2, 3]))
```

Out[3]: 0.000

In contrast, clusters that include samples from different classes destroy the homo-geneity of the labeling, hence:

In [4]:
```
print("%.3f" % metrics.v_measure_score([0, 0, 1, 1],
                                        [0, 0, 0, 0]))
```

Out[4]: 0.000

In summary, we can say that the advantages of the V-measure include that it has bounded scores: 0.0 means the clustering is extremely bad; 1.0 indicates a per-fect clustering result. Moreover, it can be interpreted easily: when analyzing the V-measure, low completeness or homogeneity explain in which direction the clus- tering is not performing well. Furthermore, we do not assume anything about the cluster structure. Therefore, it can be used to compare clustering algorithms suchas *K-means*, which assume isotropic blob shapes, with results of other clustering algorithms such as spectral clustering (see Sect. 7.2.3.2), which can find clusters with "folded" shapes. As a drawback, the previously introduced metrics are not normalized with regard to random labeling. This means that depending on the num- ber of samples, clusters and groundtruth classes, a completely random labeling will

not always yield the same values for homogeneity, completeness and hence, the

V-measure. In particular, random labeling will not yield a zero score, and they will tend further from zero as the number of clusters increases. It can be shown that this prob-lem can reliably be overcome when the number of samples is high, i.e., more than athousand, and the number of clusters is less than 10. These metrics require knowl- edge of the groundtruth classes, while in practice this information is almost never available or requires manual assignment by human annotators. Instead, as mentioned before, these metrics can be used to compare the results of different clusterings.

Silhouette Score

An alternative to the former scores is to evaluate the final 'shape' of the clustering result. This is the underlying idea behind the Silhouette coefficient. It is defined asa function of the intracluster distance of a sample in the dataset, *a* and the nearest- cluster distance, *b* for each sample.[2] Later, we will discuss different ways to compute the distance between clusters. The Silhouette coefficient for a sample *i* can be written as follows:

$$Silhouette(i) = \frac{b - a}{\max(a, b)}.$$

Hence, if the Silhouette *s(i)* is close to 0, it means that the sample is on the border ofits cluster and the closest one from the rest of the dataset clusters. A negative valuemeans that the sample is closer to the neighbor cluster. The average of the Silhouettecoefficients of all samples of a given cluster defines the "goodness" of the cluster. A high positive value, i.e., close to 1 would mean a compact cluster, and vice versa. And the average of the Silhouette coefficients of all clusters gives idea of the quality of the clustering result. Note that the Silhouette coefficient only makes sense whenthe number of labels predicted is less than the number of samples clustered.

The advantage of the Silhouette coefficient is that it is bounded between -1 and $+1$. Moreover, it is easy to show that the score is higher when clusters are dense and well separated; a logical feature when speaking about clusters. Furthermore, theSilhouette coefficient is generally higher when clusters are compact.

Taxonomies of Clustering Techniques

Within different clustering algorithms, one can find *soft partition* algorithms, which assign a probability of the data belonging to each cluster, and also *hard partition* algorithms, where each datapoint is assigned precise membership of one cluster. A typical example of a soft partition algorithm is the Mixture of Gaussians [1], which can be viewed as a density estimator method that assigns a confidence or

[2]The intracluster distance of sample *i* is obtained by the distance of the sample to the nearest sample from the same class, and the nearest-cluster distance is given by the distance to the closest samplefrom the cluster nearest to the cluster of sample *i*.

probability to each point in the space. A Gaussian mixture model is a probabilistic model that assumes all the data points are generated from a mixture of a finite number of Gaussian distributions with unknown parameters. The universally used generative unsupervised clustering using a Gaussian mixture model is also known as *EM Clustering*. Each point in the dataset has a soft assignment to the *K* clusters. One can convert this soft probabilistic assignment into membership by picking out the most likely clusters (those with the highest probability of assignment).

An alternative to soft algorithms are the *hard partition* algorithms, which assign a unique cluster value to each element in the feature space. According to the grouping process of the hard partition algorithm, there are two large families of clustering techniques:

- *Partitional algorithms*: these start with a random partition and refine it iteratively. That is why sometimes these algorithms are called "flat" clustering. In this chapter, we will consider two partitional algorithms in detail: K-means and *spectral clus-tering*.
- *Hierarchical algorithms*: these organize the data into hierarchical structures, wheredata can be agglomerated in the bottom-up direction, or split in a top-down manner. In this chapter, we will discuss and illustrate *agglomerative clustering*.

A typical hard partition algorithm is K-means clustering. We will now discuss it in some detail.

K-means Clustering

K-means algorithm is a hard partition algorithm with the goal of assigning each datapoint to a single cluster. K-means algorithm divides a set of *n* samples *X* into *k* disjoint clusters $c_i, i = 1, \ldots, k$, each described by the mean μ_i of the samples in the cluster. The means are commonly called cluster *centroids*. The K-means algorithm assumes that all *k* groups have equal variance.

K-means clustering solves the following minimization problem:

$$\arg\min_c \sum_{j=1}^{k} \sum_{x \in c_j} d(x, \mu_j) = \arg\min_c \sum_{j=1}^{k} \sum_{x \in c_j} ||x - \mu_j||^2 \quad (7.1)$$

where c_i is the set of points that belong to cluster *i* and μ_i is the center of the class c_i. K-means clustering objective function uses the square of the Euclidean distance $d(x, \mu_j) = ||x - \mu_j||^2$, that is also referred to as the *inertia* or *within-cluster sum-of-squares*. This problem is not trivial to solve (in fact, it is NP-hard problem), so the algorithm only hopes to find the global minimum, but may become stuck at a different solution.

In other words, we may wonder whether the centroids should belong to the original set of points:

$$inertia = \sum_{i=0}^{n} \min_{\mu_j \in C}(||x_i - \mu_j||^2)). \quad (7.2)$$

The K-means algorithm, also known as Lloyd's algorithm, is an iterative procedure that searches for a solution of the *K* -means clustering problem and works as follows. First, we need to decide the number of clusters, *k*. Then we apply the following procedure:

1. Initialize (e.g., randomly) the *k* cluster centers, called *centroids*.
2. Decide the class memberships of the *n* data samples by assigning them to thenearest-cluster centroids (e.g., the center of gravity or mean).
3. Re-estimate the *k* cluster centers, c_i, by assuming the memberships found aboveare correct.
4. If none of the *n* objects changes its membership from the last iteration, then exit.Otherwise go to step 2.

Let us illustrate the algorithm in Python. First, we will create three sample distri-butions:

In[5]:

```
MAXN = 40
X = np.concatenate([
    1.25*np.random.randn(MAXN, 2),
    5 + 1.5*np.random.randn(MAXN, 2)])
X = np.concatenate([
    X, [8, 3] + 1.2*np.random.randn(MAXN, 2)])
```

The sample distributions generated are shown in Fig. 7.1 (left). However, the algorithm is not aware of their distribution. Figure 7.1 (right) shows what the algorithm sees. Let us assume that we expect to have three=clusters (*k* 3) and apply the K-means command from the Scikit-learn library:

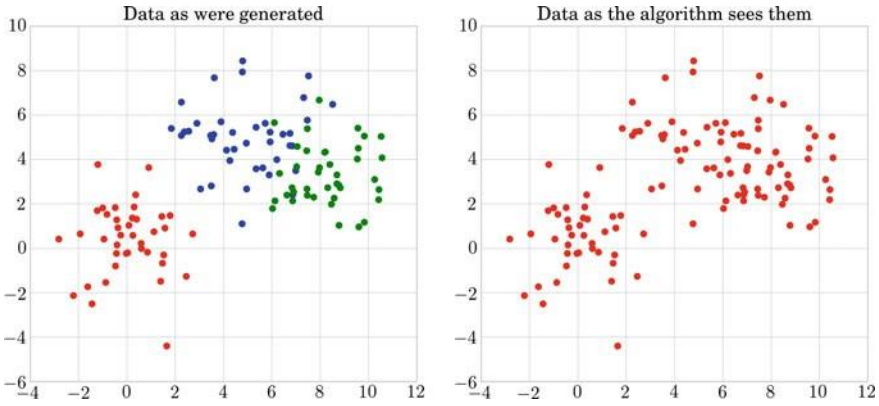


Fig. 7.1 Initial samples as generated (*left*), and samples seen by the algorithm (*right*)

In [6]:
```
from sklearn import cluster

K = 3 # Assuming we have 3 clusters!
clf = cluster.KMeans(init = 'random', n_clusters = K)
clf.fit(X)
```

Out[6]: KMeans(copy_x=True, init='random', max_iter=300,
n_clusters=3, n_init=10, n_jobs=1, precompute_distances=True,random_state=None,
tol=0.0001, verbose=0)

Each clustering algorithm in Scikit-learn is used as follows. First, an object from the clustering technique is instantiated. Then we can use the fitmethod to adjust the learning parameters. We also find the method predict that, given new data, returns the cluster they belong to. For the class, the labels over the training data canbe found in the labels_ attribute or alternatively they can be obtained using the predict method.

How many "mis-clusterings" do we have? In order to see this, we tessellate the space and color all grid points from the same cluster with the same color. Then, weoverlay the initial sample distributions (see Fig. 7.2). In the ideal case, we expect thatin each partitioned subspace the sample points are of the same color. However, as shown in Fig. 7.2, the resulting clustering, which is represented in the figure by the color subspace in gray, does not usually coincide exactly with the initial distribution, which is represented by the color of the data. For example, in the same figure, if most of the blue points belong to the same cluster, there are a few ones that belong to thespace occupied by the green data.

When computing the Rand index, we get:

In [7]:
```
print ('The Adjusted Rand index is: %.2f' %
       metrics.adjusted_rand_score(y.ravel(), clf.labels_))
```

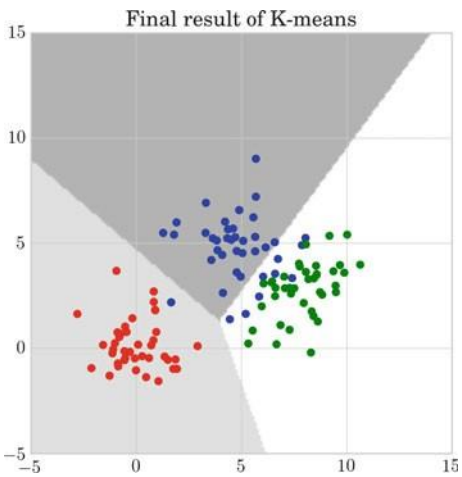


Fig. 7.2 Original samples (*dots*) generated by three distributions and the partitionof the space according to theK-means clustering

Out[7]: The Adjusted Rand index is: 0.66

Taking into account that the Adjusted Rand index belongs to the interval [0, 1], the result of 0.66 in our example means that although most of the clusters were discovered, not 100% of them were; as confirmed by Fig. 7.2.

The inertia can be seen as a measure of how internally coherent the clusters are. Several issues should be taken into account:

- The inertia assumes that clusters are isotropic and convex, since the Euclidean distance is applied, which is isotropic with regard to the different dimensions of the data. However, we cannot expect that the data fulfill this assumption by default. Hence, the K-means algorithm responds poorly to elongated clusters or manifoldswith irregular shapes.
- The algorithm may not ensure convergence to the global minimum. It can be shown that K-means will always converge to a local minimum of the inertia (Eq. (7.2)). It depends on the random initialization of the seeds, but some seeds can result in a poor convergence rate, or convergence to suboptimal clustering. To alleviate the problem of local minima, the K-means computation is often per-formed several times, with different centroid initializations. One way to address this issue is the k-means++initialization scheme, which has been implemented in Scikit-learn (use the init='kmeans++' parameter). This parameter initializes the centroids to be (generally) far from each other, thereby probably leading to better results than random initialization.
- This algorithm requires the number of clusters to be specified. Different heuristics can be applied to predetermine the number of seeds of the algorithm.
- It scales well to a large number of samples and has been used across a large rangeof application areas in many different fields.

In summary, we can conclude that K-means has the advantages of allowing the easy use of heuristics to select good seeds; initialization of seeds by other methods;multiple points to be tried. However, in contrast, it still cannot ensure that the localminima problem is overcome; it is iterative and hence slow when there are a lot of high-dimensional samples; and it tends to look for spherical clusters.

Spectral Clustering

Up to this point, the clustering procedure has been considered as a way to find datagroups following a notion of compactness. Another way of looking at what a clusteris is provided by connectivity (or similarity). Spectral clustering [2] refers to a familyof methods that use spectral techniques. Specifically, these techniques are related tothe eigendecomposition of an affinity or similarity matrix and solve the problem ofclustering according to the connectivity of the data. Let us consider an ideal similarity matrix of two clear sets.

Let us denote the similarity matrix, S, as the matrix $S_{ij} = s(x_i, x_j)$ which gives the similarity between observations x_i and x_j. Remember that we can model similarity

using the Euclidean distance, $d(x_i, x_j) = ||x_i - x_j||^2$, by means of a Gaussian Kernel as follows:

$$s(x_i, x_j) = exp(-\alpha||x_i - x_j||^2),$$

where α is a parameter. We expect two points from different clusters to be far away from each other. However, if there is a sequence of points within the cluster that forms a "path" between them, this also would lead to big distance among some of the points from the same cluster. Hence, we define an affinity matrix A based on the similarity matrix S, where A contains positive values and is symmetric. This can be done, for example, by applying a k-nearest neighbor that builds a graph connecting just the k closest data points. The symmetry comes from the fact that A_{ij} and A_{ji} give the distance between the same points. Considering the affinity matrix, the clustering can be seen as a graph partition problem, where connected graph components correspond to clusters. The graph obtained by spectral clustering will be partitioned so that graph edges connecting different clusters have low weights, and vice versa. Furthermore, we define a degree matrix D, where each diagonal value is the degree of the respective graph node and all other elements are 0. Finally, we can compute the unnormalized graph Laplacian (U D A) and/or a normalized version of the Laplacian (L), as follows:

- *Simple Laplacian*: $L = D^{-1}A$, which corresponds to a random walk, being D^{-1} the transition matrix. Spectral clustering obtains groups of nodes such that the random walk corresponds to seldom transitions from one group to another.
- *Normalized Laplacian*: $L = D^{-1} U D^{-1}$.
- *Generalized Laplacian*: $L = D^{-1}U$.

If we assume that there are k clusters, the next step is to find the k small-est eigenvectors, without considering the trivial constant eigenvector. Each row of the matrix formed by the k smallest eigenvectors of the Laplacian matrix defines a transformation of the data x_i. Thus, in this transformed space, we can apply K-means clustering in order to find the final clusters. If we do not know in advance the number of clusters, k, we can look for sudden changes in the sorted eigenvalues of the matrix, U, and keep the smallest ones.

Hierarchical Clustering

Another well-known clustering technique of particular interest is hierarchical cluster-ing. Hierarchical clustering is comprised of a general family of clustering algorithms that construct nested clusters by successive merging or splitting of data. The hier-archy of clusters is represented as a tree. The tree is usually called a *dendrogram*. The root of the dendrogram is the single cluster that contains all the samples; the leaves are the clusters containing only one sample each. This is a nice tool, since it can be straightforwardly interpreted: it "explains" how clusters are formed and visualizes clusters at different scales. The tree that results from the technique shows

the similarity between the samples. Partitioning is computed by selecting a cut onthe tree at a certain level.

In general, there are two types of hierarchical clustering:

- *Top-down* divisive clustering applies the following algorithm:
  - Start with all the data in a single cluster.
  - Consider every possible way to divide the cluster into two.
  - Choose the best division.
  - Recursively, it operates on both sides until a stopping criterion is met. That can be something as follows: there are as much clusters as data; the predetermined number of clusters has been reached; the maximum distance between all possible partition divisions is smaller than a predetermined threshold; etc.

- *Bottom-up* agglomerative clustering applies the following algorithm:
  - Start with each data point in a separate cluster.
  - Repeatedly join the closest pair of clusters.
  - At each step, a stopping criterion is checked: there is only one cluster; a prede-termined number of clusters has been reached; the distance between the closestclusters is greater than a predetermined threshold; etc.

This process of merging forms a binary tree or hierarchy.

When merging two clusters, a question naturally arises: How to measure the similarity of two clusters? There are different ways to define this with different results for the agglomerative clustering. The linkage criterion determines the metricused for the cluster merging strategy:

- *Maximum* or *complete* linkage minimizes the maximum distance between observations of pairs of clusters. Based on the similarity of the two least similar membersof the clusters, this clustering tends to give tight spherical clusters as a final result.*Average* linkage averages similarity between members, i.e., minimizes the averageof the distances between all observations of pairs of clusters.
- *Ward* linkage minimizes the sum of squared differences within all clusters. It is thus a variance-minimizing approach and in this sense is similar to the K-means objective function, but tackled with an agglomerative hierarchical approach.

Let us illustrate how the different linkages work with an example. Let us generatethree clusters as follows:

In [8]:

```
MAXN1 = 500
MAXN2 = 400
MAXN3 = 300
X1 = np.concatenate([
    2.25 * np.random.randn(MAXN1, 2),
    4 + 1.7*np.random.randn(MAXN2, 2)])
X1 = np.concatenate([
    X1, [8, 3] + 1.9*np.random.randn(MAXN3, 2)])

y1 = np.concatenate([
    np.ones((MAXN1, 1)),
    2 * np.ones((MAXN2, 1))])
y1 = np.concatenate([
    y1, 3 * np.ones((MAXN3, 1))]).ravel()
y1 = np.int_(y1)
labels_y1 = ['+', '*', 'o']
colors = ['r', 'g', 'b']
```

Let us apply agglomerative clustering using the different linkages:

In [9]:

```
from sklearn.cluster import AgglomerativeClustering

for linkage in ('ward', 'complete', 'average'):
    clustering = AgglomerativeClustering(linkage = linkage,
                                         n_clusters =3)
    clustering.fit(X1)

    x_min , x_max = np.min (X1, axis = 0) , np.max (X1,axis
        = 0)
    X1 = (X1 - x_min ) / ( x_max - x_min )
    plt.figure ( figsize =(5 , 5))
    for i in range (X1.shape [0]) :
        plt.text(X1[i, 0], X1[i, 1], labels_y1[y1[i]-1],
                 color = colors[y1[i]-1])
    plt.title("\%s linkage " \% linkage, size = 20)
    plt.tight_layout()

plt.show()
```

The results of the agglomerative clustering using the different linkages: complete, average, and Ward are given in Fig. 7.3. Note that agglomerative clustering exhibits “rich get richer” behavior that can sometimes lead to uneven cluster sizes, with average linkage being the worst strategy in this respect and Ward linkage giving themost regular sizes. Ward linkage is an attempt to form clusters that are as compactas possible, since it considers inter- and intra-distances of the clusters. Meanwhile, for non-Euclidean metrics, average linkage is a good alternative. Average linkage can produce very unbalanced clusters, it can even separate a single data point into aseparate cluster. This fact would be useful if we want to detect outliers, but it may be undesirable when two clusters are very close to each other, since it would tend tomerge them.

Agglomerative clustering can scale to a large number of samples when it is usedjointly with a connectivity matrix, but it is computationally expensive when no con-

nectivity constraints are added between samples: it considers all the possible mergesat each step.

Adding Connectivity Constraints

Sometimes, we are interested in introducing a connectivity constraint into the clus- tering process so that merging of nonadjacent points is avoided. This can be achieved by constructing a connectivity matrix that defines which are the neighboring samplesin the dataset. For instance, in the example in Fig. 7.4, we want to avoid the forma-tion of clusters of samples from the different circles. A sample code to compute agglomerative clustering with connectivity would be as follows:

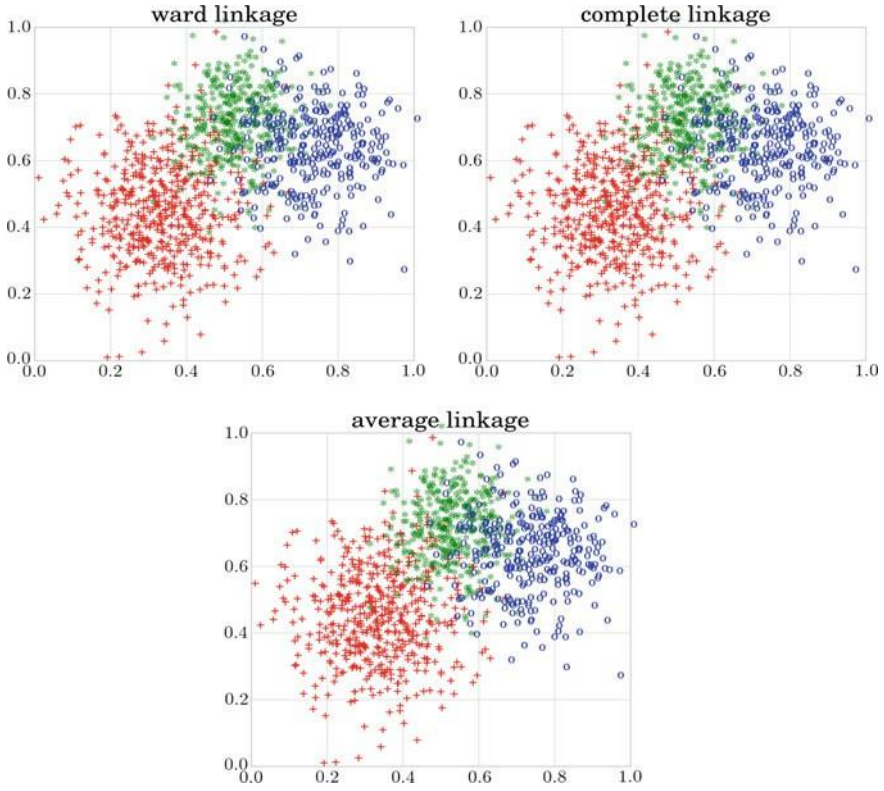


Fig. 7.3 Illustration of agglomerative clustering using different linkages: Ward, complete, and average. The symbol of each data point corresponds to the original class generated and the color corresponds to the cluster obtained

Fig. 7.4 Illustration of agglomerative clustering without (*top row*) and with (*bottom row*) a connectivity graph using the three linkages (from *left* to *right*): average, complete, and Ward. The *colors*correspond to the clusters obtained

In [10]:

```
connectivity = kneighbors_graph(X, 30)
model = AgglomerativeClustering(linkage = 'average',
    connectivity = connectivity, n_clusters = 8)
model.fit(X)
```

A connectivity constraint is useful to impose a certain local structure, but it also makes the algorithm faster, especially when the number of the samples is large. A connectivity constraint is imposed via a *connectivity matrix*: a sparse matrix that only has elements at the intersection of a row and a column with indexes of the dataset that should be connected. This matrix can be constructed from a priori information or can be learned from the data, for instance using kneighbors_graph to restrict merging to nearest neighbors or using image.grid_to_graphto limit mergingto neighboring pixels in an image, both from Scikit-learn. This phenomenon can beobserved in Fig. 7.4, where in the first row we see the results of the agglomerative clustering without using a connectivity graph. The clustering can join data from different circles (e.g., the black cluster). At the bottom, the three linkages use a connectivity graph and thus two of them avoid joining data points that belong to different circles (except the Ward linkage that attempts to form compact and isotropic clusters).

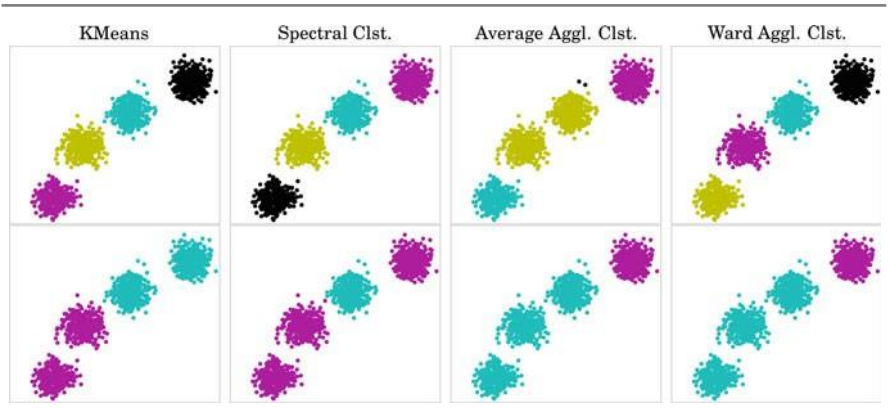


Fig. 7.5 Comparison of the different clustering techniques (from *left* to *right*): K-means, spectral clustering, and agglomerative clustering with average and Ward linkage on simple compact datasets. In the *first row*, the expected number of clusters is $k = 2$ and in the *second row*: $k = 4$

Comparison of Different Hard Partition Clustering Algorithms Let us compare the behavior of the different clustering algorithms discussed so far.For this purpose, we generate three different datasets' configurations:

(a) 4 spherical groups of data;
(b) a uniform data distribution; and
(c) a non-flat configuration of data composed of two moon-like groups of data.

An easy way to generate these datasets is by using Scikit-learn that has predefined functions for it: datasets.make_blobs(), datasets.ma- ke_ moons(), etc.

We apply the clustering techniques discussed above, namely K-means, agglom-erative clustering with average linkage, agglomerative clustering with Ward linkage, and spectral clustering. Let us test the behavior of the different algorithms assuming $k = 2$ and $k = 4$. Connectivity is applied in the algorithms where it is applicable.

In the simple case of separated clusters of data and $k = 4$, most of the clustering algorithms perform well, as expected (see Fig. 7.5). The only algorithm that could not discover the four groups of samples is the average agglomerative clustering. Since it allows highly unbalanced clusters, the two noisy data points that are quite separated from the closest two blobs were considered as a different cluster, while the two central blobs were merged in one cluster. In case of $k = 2$, each of the methods is obligated to join at least two blobs in a cluster.

Regarding the uniform distribution of data (see Fig. 7.6), K-means, Ward linkage agglomerative clustering and spectral clustering tend to yield even and compact clusters; while the average linkage agglomerative clustering attempts to join close points as much as possible following the "rich get richer" rule. This results in a

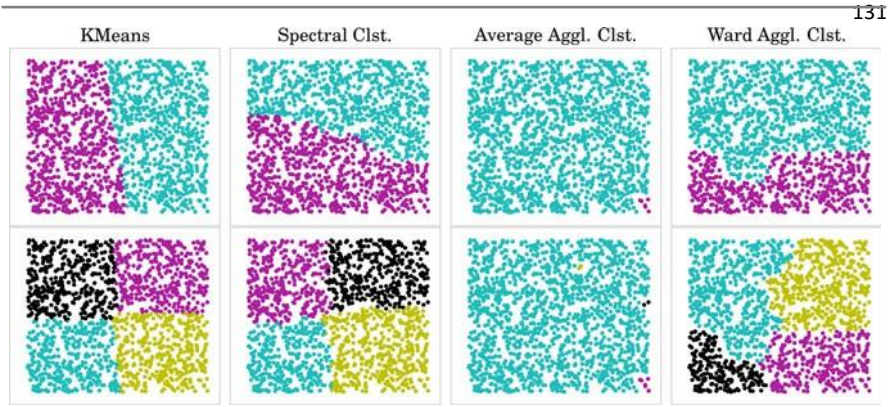


Fig. 7.6 Comparison of the different clustering techniques (from *left* to *right*): K-means, spectral clustering, and agglomerative clustering with average and Ward linkage on uniformly distributeddata. In the *first row*, the number of clusters assumed is $k = 2$ and in the *second row*: $k = 4$

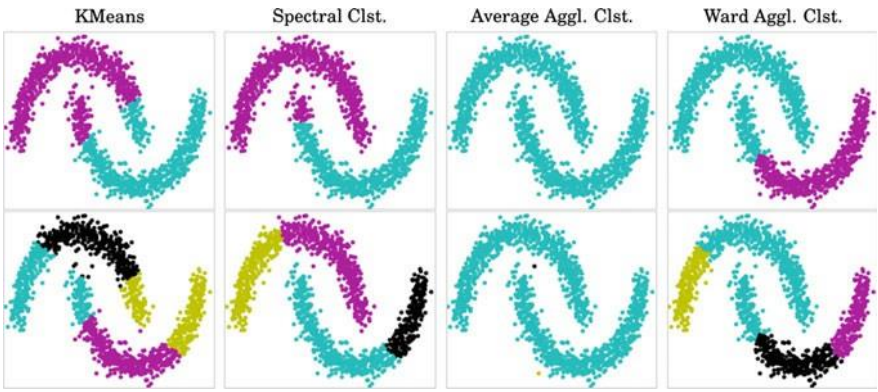


Fig. 7.7 Comparison of the different clustering techniques (from *left* to *right*): K-means, spectral clustering, and agglomerative clustering with average and Ward linkage on non-flat geometry datasets. In the *first row*, the expected number of clusters is $k = 2$ and in the *second row*: $k = 4$

second cluster of a small set of data. This behavior is observed in both cases: $k = 2$ and $k = 4$.

Regarding datasets with more complex geometry, like in the moon dataset (see Fig. 7.7), K-means and Ward linkage agglomerative clustering attempt to construct compact clusters and thus cannot separate the moons. Due to the connectivity con- straint, the spectral clustering and the average linkage agglomerative clustering sep-arated both moons in case of $k = 2$, while in case of $k = 4$, the average linkage agglomerative clustering clustered most of datasets correctly separating some of thenoisy data points as two separate single clusters. In the case of spectral clustering, looking for four clusters, the method splits each of the two moon datasets into two clusters.

	0	1	2	3	4	5	6	7	8	9	10	11
GEO												
Albania	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Austria	0.52	5.25	9.98	0.64	1.22	0.61	1.01	2.64	1.63	5.89	5.90	11.20
Belgium	0.34	6.25	11.90	0.32	0.61	0.78	1.54	2.79	1.46	6.57	6.44	12.51
Bulgaria	0.63	3.35	8.96	0.74	1.99	0.92	0.80	1.76	0.61	4.10	4.18	10.95
Croatia	0.26	4.24	NaN	0.03	NaN	0.65	1.87	0.97	0.78	4.27	4.42	NaN

Fig. 7.8 Expenditure on different educational indicators for the first five countries in the Eurostat dataset

Case Study

In order to illustrate clustering with a real dataset, we will now analyze the indicators of spending on education among the European Union member states, provided by the Eurostat data bank.[3] The data are organized by year (TIME) from 2002 until 2011 and country (GEO): ('Albania', 'Austria', 'Belgium', 'Bulgaria', etc.). Twelve indicators (INDIC_ED) of financing of education with their corresponding values (Value) are given: (1) Expenditure on educational institutions from private sources as % of gross domestic product (GDP), for all levels of education combined; (2) Expenditure on educational institutions from public sources as % of GDP, for all levels of government combined, (3) Expenditure on educational institutions from public sources as % of total public expenditure, for all levels of education combined,
(4) Public subsidies to the private sector as % of GDP, for all levels of education combined, (5) Public subsidies to the private sector as % of total public expenditure, for all levels of education combined, etc. We can store the 12 indicators for a given year (e.g., 2010) in a table. Figure 7.8 provides visualization of the first five countries in the table.

As we can observe, this is not a clean dataset, since there are values missing. Some countries have very limited information and should be excluded. Other countries may still not collect or have access to a few indicators. For these last cases, we can proceed in two ways: (a) fill in the gaps with some non-informative, non-biasing data; or (b) drop the features with missing values for the analysis. If we have many features and only a few have missing values, then it is not very harmful to drop them. However, if missing values are spread across most of the features, we eventually have to deal with them. In our case, both options seem reasonable, as long as the number of missing features for a country is not too large. We will proceed in both ways at the same time. We apply both options: filling the gap with the mean value of the feature and the dropping option, ignoring the indicators with missing values. Let us now apply K-means clustering to these data in order to partition the countries according to

.

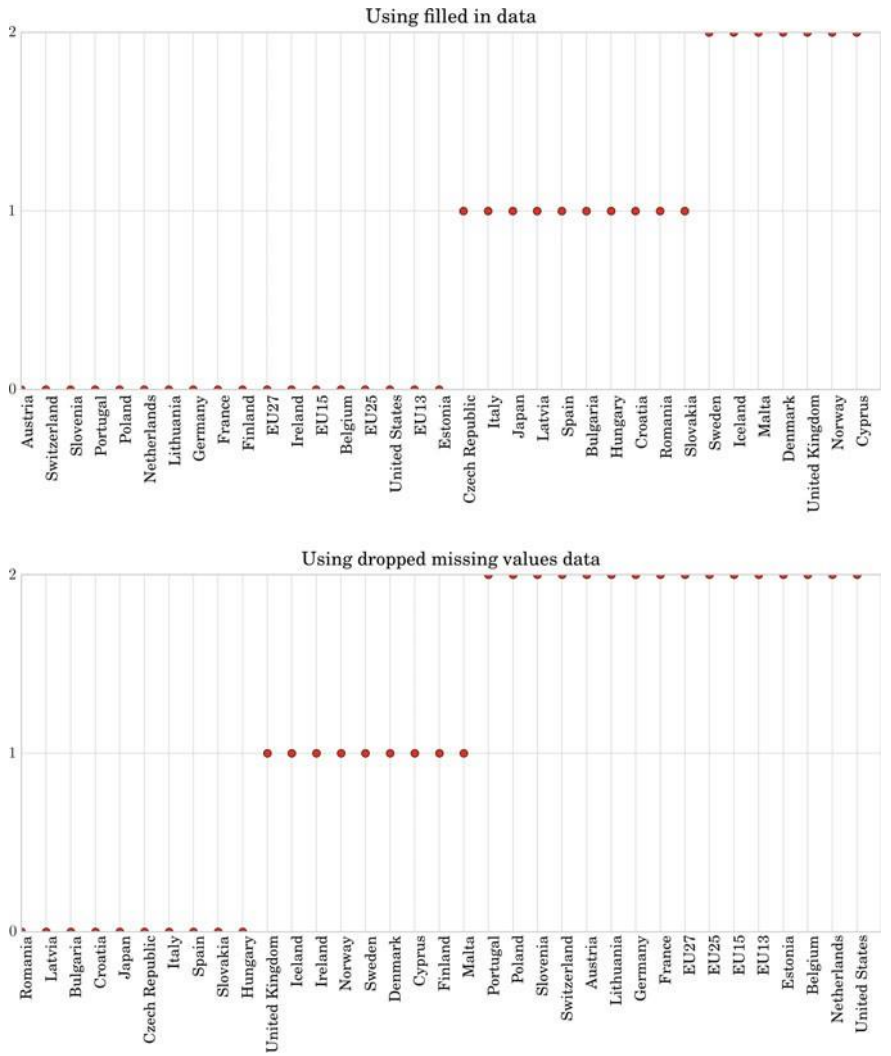


Fig. 7.9 Clustering of the countries according to their educational expenditure using filled-in (*top row*) and dropped (*bottom row*) missing values

their investment in education and check their profiles. Figure 7.9 shows the results of this K-means clustering. We have sorted the data for better visualization. Ata simple glance, we can see that the partitions (top and bottom of Fig. 7.9) are different. Most countries in cluster 2 in the filled-in dataset correspond to cluster 0in the dropped missing values dataset. Analogously, most of cluster 0 in the filled-in dataset correspond to cluster 1 in the dropped missing values dataset; and most countries from cluster 1 in the filled-in dataset correspond to cluster 2 in the dropped

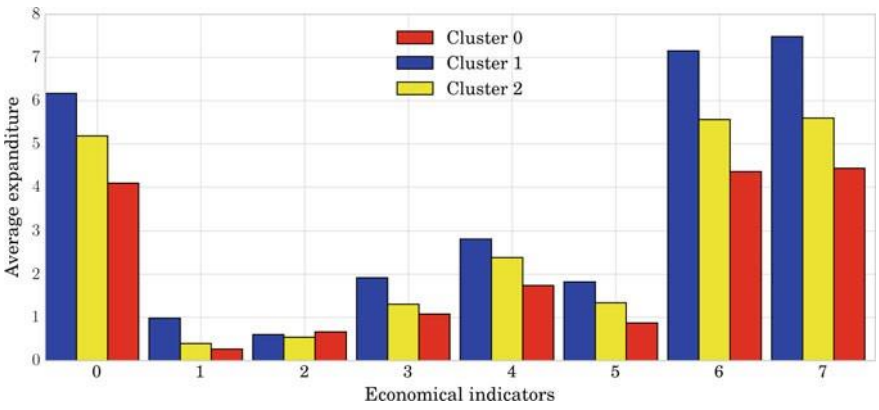


Fig. 7.10 Mean expenditure of the different clusters according to the 8 indicators of the indicators-dropped dataset

set. Still, there are some countries that do not follow this rule. That is, looking at both clusterings, they may yield similar (up to label permutation) results, but theywill not necessarily always coincide. This is mainly due to two aspects: the randominitialization of the K-means clustering and the fact that each method works in adifferent space (i.e., dropped data in 8D space *vs* filled-in data, working in 12Dspace). Note that we should not consider the assigned absolute cluster value, sinceit is irrelevant. The mean expenditure of the different clusters is shown by differentcolors according to the 8 indicators of the indicators-dropped dataset (see Fig. 7.10).So, without loss of generality, we continue analyzing the set obtained by dropping missing values. Let us now check the clusters and check their profile by looking atthe centroids. Visualizing the eight values of the three clusters (see Fig. 7.10), we cansee that cluster 1 spends more on education for the 8 educational indicators, while cluster 0 is the one with least resources invested in education.

Let us consider a specific country, e.g., Spain and its expenditure on education. If we refine cluster 0 further and check how close members are from this cluster to cluster 1, it may give us a hint as to a possible ordering. When visualizing the distance to cluster 0 and 1, we can observe that Spain, while being from cluster 0, hasa smaller distance to cluster 1 (see Fig. 7.11). This should make us realize that using 3 clusters probably does not sufficiently represent the groups of countries. So we redothe process, but applying $k = 4$: we obtain 4 clusters. This time cluster 0 includes the EU members with medium expenditure (Fig. 7.12). This reinforce the intuition about Spain being a limit case in the former clustering. The clusters obtained are asfollows:

- Cluster 0: (‘Austria’, ‘Estonia’, ‘EU13’, ‘EU15’, ‘EU25’, ‘EU27’, ‘France’, ‘Germany’, ‘Hungary’, ‘Latvia’, ‘Lithuania’, ‘Netherlands’, ‘Poland’, ‘Portugal’, ‘Slovenia’, ‘Spain’, ‘Switzerland’, ‘United Kingdom’, ‘United States’)

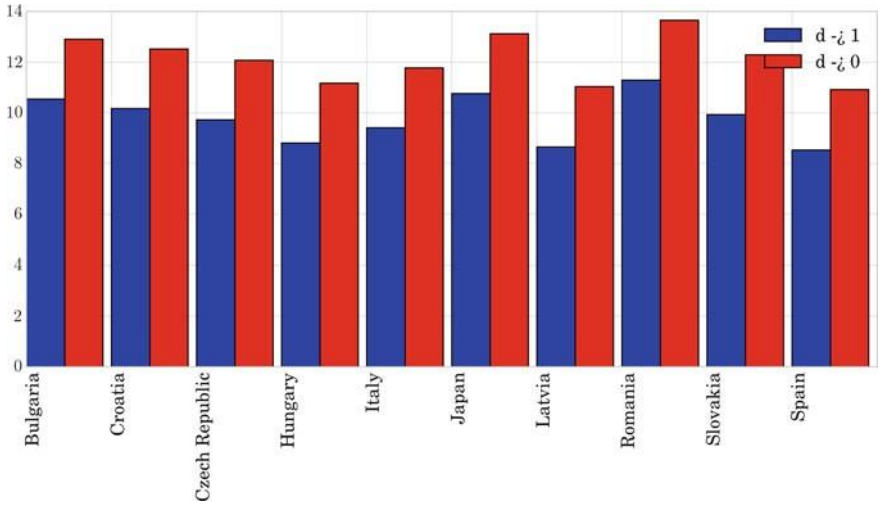


Fig. 7.11 Distance of countries in cluster 0 to centroids of cluster 0 (in *red*) and cluster 1 (in *blue*)

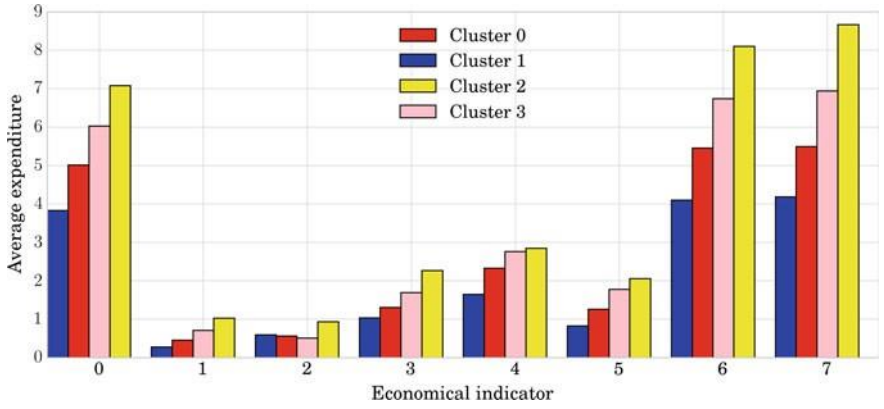


Fig. 7.12 K-means applied to the Eurostat dataset grouping the countries into four clusters

- Cluster 1: ('Bulgaria', 'Croatia', 'Czech Republic', 'Italy', 'Japan', 'Romania','Slovakia')
- Cluster 2: ('Cyprus', 'Denmark', 'Iceland')
- Cluster 3: ('Belgium', 'Finland', 'Ireland', 'Malta', 'Norway', 'Sweden')

We can repeat the process using the alternative clustering techniques and compare their results. Let us first apply spectral clustering. The corresponding code will be as follows:

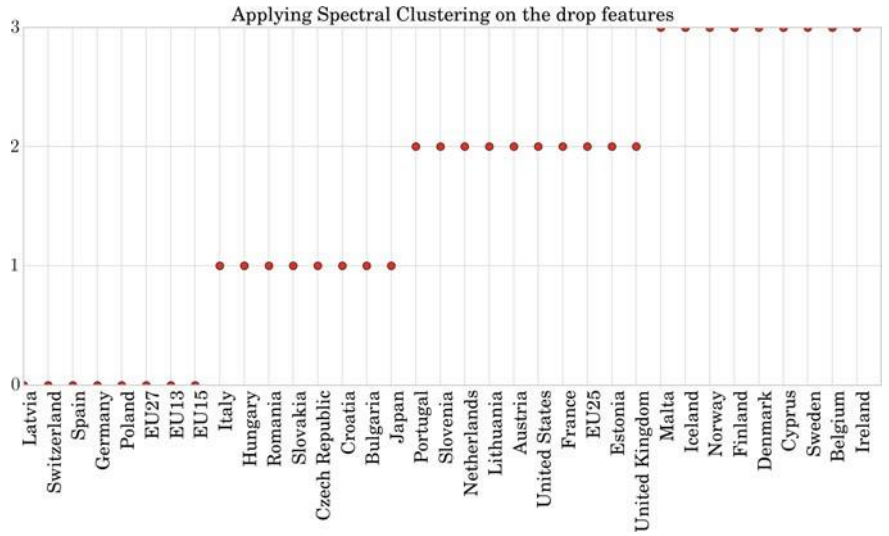


Fig. 7.13 Spectral clustering applied to the European countries according to their expenditure on education

In [11]:

```
X = StandardScaler().fit_transform(edudrop.values)
distances = euclidean_distances(edudrop.values)
spectral = cluster.SpectralClustering(
    n_clusters = 4, affinity = "nearest_neighbors")
spectral.fit(edudrop.values)
y_pred = spectral.labels_.astype(np.int)
```

The result of this spectral clustering is shown in Fig. 7.13. Note that in general, the aim of spectral clustering is to obtain more balanced clusters. In this way, the predicted cluster 1 merges clusters 2 and 3 of the K-means clustering, cluster 2 corresponds to cluster 1 of the K-means clustering, cluster 0 mainly shifts to cluster2, and cluster 3 corresponds to cluster 0 of the K-means.

Applying agglomerative clustering, not only we do obtain different clusters, but also we can see how different clusters are obtained. Thus, in some way it is giving us information on which the most similar pairs of countries and clusters are. The corresponding code that applies the agglomerative clustering will be as follows:

In [12]:

```
from scipy.cluster.hierarchy import linkage, dendrogram
from scipy.spatial.distance import pdist

X_train = edudrop.values
dist = pdist(X_train, 'euclidean')
linkage_matrix = linkage(dist, method = 'complete');

plt.figure(figsize = (11.3, 11.3))
dendrogram(linkage_matrix, orientation="right",
           color_threshold = 3,
           labels = wrk_countries_names,
           leaf_font_size = 20);
plt.tight_layout()
```

In Scikit-learn, the parameter *color_threshold* of the command dendro- gram() colors all the descendent links below a cluster node k the same color if k is the first node below the color_threshold. All links connecting nodes with distances greater than or equal to the threshold are colored blue. Hence, using *color_threshold*
= 3, the clusters obtained are as follows:

- Cluster 0: ('Cyprus', 'Denmark', 'Iceland')
- Cluster 1: ('Bulgaria', 'Croatia', 'Czech Republic', 'Italy', 'Japan', 'Romania', 'Slovakia')
- Cluster 2: ('Belgium', 'Finland', 'Ireland', 'Malta', 'Norway', 'Sweden')
- Cluster 3: ('Austria', 'Estonia', 'EU13', 'EU15', 'EU25', 'EU27', 'France', 'Germany', 'Hungary', 'Latvia', 'Lithuania', 'Netherlands', 'Poland', 'Portugal', 'Slovenia', 'Spain', 'Switzerland', 'United Kingdom', 'United States')
  Note that, to a high degree, they correspond to the clusters obtained by the K-means (except for permutation of cluster labels, which is irrelevant).

Figure 7.14 shows the construction of the clusters using complete linkage agglomerative clustering. Different cuts at different levels of the dendrogram allow us to obtain different numbers of clusters.

To summarize, we can compare the results of the three clustering approaches. We cannot expect the results to coincide, since the different approaches are based on different criteria for constructing clusters. Nonetheless, we can still observe that in this case, K-means and the agglomerative approaches gave the same results (up to apermutation of the number of cluster, which is irrelevant); while spectral clustering gave more evenly distributed clusters. This later approach fused clusters 0 and 2 ofthe agglomerative clustering in cluster 1, and split cluster 3 of the agglomerative clustering into its clusters 0 and 3. Note that these results could change when usingdifferent distances among data.

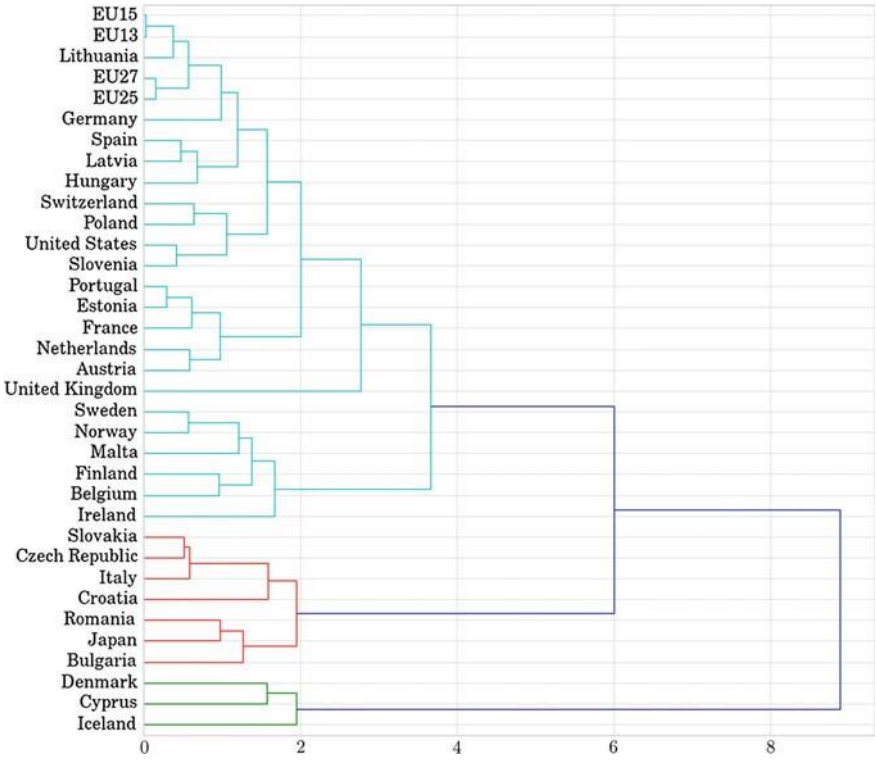


Fig. 7.14 Agglomerative clustering applied to cluster European countries according to their expenditure on education

UNIT-5

Network Analysis, Graphs, Social Networks, centrality, drawing centrality of Graphs, PageRank, Ego-Networks, community Detection

Network Analysis

Introduction

Network data are generated when we consider relationships between two or more entities in the data, like the highways connecting cities, friendships between peo- ple or their phone calls. In recent years, a huge number of network data are being generated and analyzed in different fields. For instance, in sociology there is inter-est in analyzing blog networks, which can be built based on their citations, to look for divisions in their structures between political orientations. Another example is infectious disease transmission networks, which are built in epidemiological studies to find the best way to prevent infection of people in a territory, by isolating cer- tain areas. Other examples studied in the field of technology include interconnected computer networks or power grids, which are analyzed to optimize their functioning. We also find examples in academia, where we can build co-authorship networks and citation networks to analyze collaborations among Universities.

Structuring data as networks can facilitate the study of the data for different goals;for example, to discover the weaknesses of a structure. That could be the objective of a biologist studying a community of plants and trying to establish which of its properties promote quick transmission of a disease. A contrasting objective would be to find and exploit structures that work efficiently for the transmission of messages across the network. This may be the goal of an advertising agent trying to find the best strategy for spreading publicity.

How to analyze networks and extract the features we want to study are someof the issues we consider in this chapter. In particular, we introduce some basic concepts related with networks, such as connected components, centrality measures, ego-networks, and PageRank. We present some useful Python tools for the analysisof networks and discuss some of the visualization options. In order to motivate andillustrate the concepts, we perform social network analysis using real data. We present a practical case based on a public dataset which consists of a set of interconnected

Facebook friendship networks. We formulate multiple questions at different levels:the local/member level, the community level, and the global level.

In general, some of the questions we try to solve are the following:

- What type of network are we dealing with?
- Which is the most representative member of the network in terms of being the most connected to the rest of the members?
- Which is the most representative member of the network in terms of being the most circulated on the paths between the rest of the members?
- Which is the most representative member of the network in terms of proximity tothe rest of the members?
- Which is the most representative member of the network in terms of being the most accessible from any location in the network?
- There are many ways of calculating the representativeness or importance of a member, each one with a different meaning, so: how can we illustrate them and compare them?
- Are there different communities in the network? If so, how many?
- Does any member of the network belong to more than one community? That is, is there any overlap between the communities? How much overlap? How can weillustrate this overlap?
- Which is the largest community in the network?
- Which is the most dense community (in terms of connections)?
- How can we automatically detect the communities in the network?
- Is there any difference between automatically detected communities and real ones(manually labeled by users)?

Basic Definitions in Graphs

Graph is the mathematical term used to refer to a network. Thus, the field that studies networks is called *graph theory* and it provides the tools necessary to analyze networks. Leonhard Euler defined the first graph in 1735, as an abstraction of one ofthe problems posed by mathematicians of the time regarding Konigsberg, a city withtwo islands created by the River Pregel, which was crossed by seven bridges. The problem was: is it possible to walk through the town of Konigsberg crossing each bridge once and only once? Euler represented the land areas as nodes and the bridges connecting them as edges of a graph and proved that the walk was not possible forthis particular graph.

A graph is defined as a set of *nodes*, which are an abstraction of any entities (parts of a city, persons, etc.), and the connecting links between pairs of nodes called *edges* or relationships. The edge between two nodes can be *directed* or *undirected*.A directed edge means that the edge points from one node to the other and not the otherway round. An example of a directed relationship is “a person knows another person”. An edge has a direction when person A knows person B, and not the reverse

direction

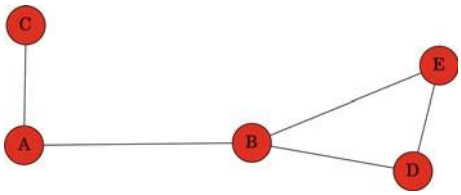


Fig. 8.1 Simple undirected labeled graph with 5 nodesand5 edges

if B does not know A (which is usual for many fans and celebrities). An undirected edge means that there is a symmetric relationship. An example is "a person shook hands with another person"; in this case, the relationship, unavoidably, involves both persons and there is no directionality. Depending on whether the edges of a graph are directed or undirected, the graph is called a *directed graph* or an *undirected graph*,respectively.

The *degree* of a node is the number of edges that connect to it. Figure 8.1 showsan example of an undirected graph with 5 nodes and 5 edges. The degree of node Cis 1, while the degree of nodes A, D and E is 2 and for node B it is 3. If a network isdirected, then nodes have two different degrees, the *in-degree*, which is the number of incoming edges, and the *out-degree*, which is the number of outgoing edges.

In some cases, there is information we would like to add to graphs to model properties of the entities that the nodes represent or their relationships. We could add *strengths* or *weights* to the links between the nodes, to represent some real-world measure. For instance, the length of the highways connecting the cities in a network.In this case, the graph is called a *weighted graph*.

Some other elementary concepts that are useful in graph analysis are those weexplain in what follows. We define a *path* in a network to be a sequence of nodesconnected by edges. Moreover, many applications of graphs require *shortest paths*to be computed. The shortest path problem is the problem of finding a path betweentwo nodes in a graph such that the length of the path or the sum of the weights ofedges in the path is minimized. In the example in Fig. 8.1, the paths (C, A, B, E) and(C, A, B, D, E) are those between nodes C and E. This graph is unweighted, so theshortest path between C and E is the one that follows the fewer edges: (C, A, B, E).A graph is said to be *connected* if for every pair of nodes, there is a path between them. A graph is *fully connected* or *complete* if each pair of nodes is connected byan edge. A *connected component* or simply a *component* of a graph is a subset of itsnodes such that every node in the subset has a path to every other one. In the exampleof Fig. 8.1, the graph has one connected component. A *subgraph* is a subset of thenodes of a graph and all the edges linking those nodes. Any group of nodes can form a subgraph.

Social Network Analysis

Social network analysis processes social data structured in graphs. It involves the extraction of several characteristics and graphics to describe the main properties of the network. Some general properties of networks, such as the shape of the network degree distribution (defined bellow) or the average path length, determine the type of network, such as a *small-world* network or a *scale-free* network. A small-world network is a type of graph in which most nodes are not neighbors of one another, butmost nodes can be reached from every other node in a small number of steps. This is the so-called *small-world phenomenon* which can be interpreted by the fact that strangers are linked by a short chain of acquaintances. In a small-world network, people usually form communities or small groups where everyone knows every- one else. Such communities can be seen as complete graphs. In addition, most the community members have a few relationships with people outside that community. However, some people are connected to a large number of communities. These maybe celebrities and such people are considered as the *hubs* that are responsible for the small-world phenomenon. Many small-world networks are also scale-free net- works. In a scale-free network the node degree distribution follows a power law (a relationship function between two quantities x and y defined as $y \quad x^n$, where n isa constant). The name *scale-free* comes from the fact that power laws have the same functional form at all scales, i.e., their shape does not change on multiplication by a scale factor. Thus, by definition, a scale-free network has many nodes with a very few connections and a small number of nodes with many connections. This structure is typical of the World Wide Web and other social networks. In the following sections, we illustrate this and other graph properties that are useful in social network analysis.

In [1]:

Basics in NetworkX

NetworkX[1] is a Python toolbox for the creation, manipulation and study of the struc- ture, dynamics and functions of complex networks. After importing the toolbox, wecan create an undirected graph with 5 nodes by adding the edges, as is done in the following code. The output is the graph in Fig. 8.1.

```
import networkx as nx
G = nx.Graph()
G.add_edge('A', 'B');
G.add_edge('A', 'C');
G.add_edge('B', 'D');
G.add_edge('B', 'E');
G.add_edge('D', 'E');
nx.draw_networkx(G)
```

To create a directed graph we would use nx.DiGraph().

Practical Case: Facebook Dataset

For our practical case we consider data from the Facebook network. In particular, we use the data *Social circles: Facebook*[2] from the Stanford Large Network Dataset[3] (SNAP) collection. The SNAP collection has links to a great variety of networks such as Facebook-style social networks, citation networks, Twitter networks or open communities like Live Journal. The Facebook dataset consists of a network repre- senting friendship between Facebook users. The Facebook data was anonymized by replacing the internal Facebook identifiers for each user with a new value.

The network corresponds to an undirected and unweighted graph that contains users of Facebook (nodes) and their friendship relations (edges). The Facebook dataset is defined by an edge list in a plain text file with one edge per line.

In[2]: Let us load the Facebook network and start extracting the basic information from the graph, including the numbers of nodes and edges, and the average

```
fb = nx.read_edgelist("files/ch08/facebook_combined.txt")
fb_n, fb_k = fb.order(), fb.size()
fb_avg_deg = fb_k / fb_n
print 'Nodes: ', fb_n
print 'Edges: ', fb_k
print 'Average degree: ', fb_avg_deg
```

Out[2]:
```
Nodes: 4039
Edges: 88234
Average degree: 21
```

The Facebook dataset has a total of 4,039 users and 88,234 friendship connections, with an average degree of 21. In order to better understand the graph, let us compute the degree distribution of the graph. If the graph were directed, we would need to generate two distributions: one for the in-degree and another for the out-degree. A way to illustrate the degree distribution is by computing the histogram of degrees and plotting it, as the following code does with the output shown in Fig. 8.2:

In[3]:
```
degrees = fb.degree().values()
degree_hist = plt.hist(degrees, 100)
```

The graph in Fig. 8.2 is a power-law distribution. Thus, we can say that the Face- book network is a *scale-free network*.

Next, let us find out if the Facebook dataset contains more than one connectedcomponent (previously defined in Sect. 8.2):

In[4]:
```
print '# connected components of Facebook network: ',
      nx.number_connected_components(fb)
```

Out[4]: # connected components of Facebook network: 1

As it can be seen, there is only one connected component in the Facebook network. Thus, the Facebook network is a connected graph (see definition in Sect. 8.2). We can try to divide the graph into different connected components, which can be potentialcommunities (see Sect. 8.6). To do that, we can remove one node from the graph (this operation also involves removing the edges linking the node) and see if the number of connected components of the graph changes. In the following code, we prune the graph by removing node '0' (arbitrarily selected) and compute the number of connected components of the pruned version of the graph:

In[5]:

```
fb_prun = nx.read_edgelist(
    "files/ch08/facebook_combined.txt")
fb_prun.remove_node('0')
print 'Remaining nodes:', fb_prun.number_of_nodes()
print 'New # connected components:',
      nx.number_connected_components(fb_prun)
```

Out[5]: Remaining nodes: 4038
New # connected components: 19

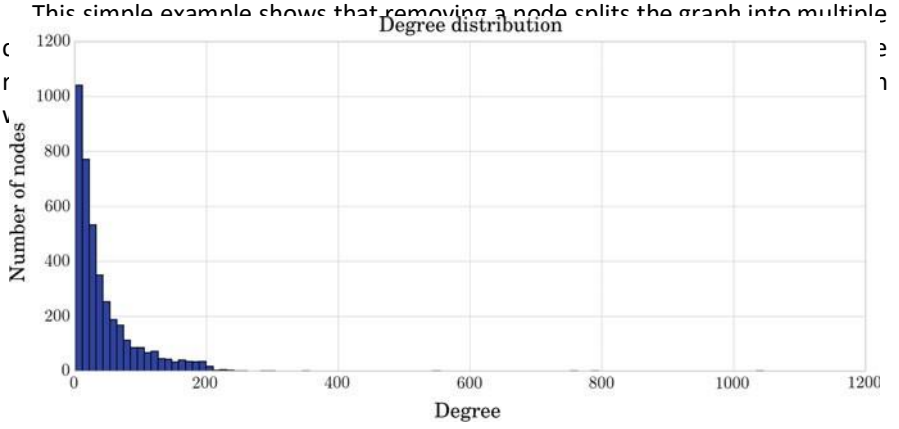
Now there are 19 connected components, but let us see how big the biggest is andhow small the smallest is:

In[6]:

```
fb_components = nx.connected_components(fb_prun)
print 'Sizes of the connected components',
      [len(c) for c in fb_components]
```

Out[6]: Sizes of the connected components [4015, 1, 3, 2, 2, 1, 1, 1,
1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1]

This simple example shows that removing a node splits the graph into multiple
c e
r n
v



connected to node '0' in the original graph and when that node was removed they were converted into connected components of size 1. These nodes, only connected to one neighbor, are probably not important nodes in the structure of the graph. We can generalize the analysis by studying the *centrality* of the nodes. The next section is devoted to explore this concept.

Centrality

The centrality of a node measures its relative importance within the graph. In this section we focus on undirected graphs. Centrality concepts were first developed in social network analysis. The first studies indicated that central nodes are probably more influential, have greater access to information, and can communicate their opinions to others more efficiently [1]. Thus, the applications of centrality concepts in a social network include identifying the most influential people, the most informed people, or the most communicative people. In practice, what centrality means will depend on the application and the meaning of the entities represented as nodes in the data and the connections between those nodes. Various measures of the centrality of a node have been proposed. We present four of the best-known measures: *degree centrality*, *betweenness centrality*, *closeness centrality*, and *eigenvector centrality*.

Degree centrality is defined as the number of edges of the node. So the more ties a node has, the more central the node is. To achieve a normalized degree centrality of a node, the measure is divided by the total number of graph nodes (n) without counting this particular one (n 1). The normalized measure provides proportions and allows us to compare it among graphs. Degree centrality is related to the capacity of a node to capture any information that is floating through the network. In social networks, connections are associated with positive aspects such as knowledge or friendship.

Betweenness centrality quantifies the number of times a node is crossed along the shortest path/s between any other pair of nodes. For the normalized measure this number is divided by the total number of shortest paths for every pair of nodes. Intuitively, if we think of a public bus transportation network, the bus stop (node) with the highest betweenness has the most traffic. In social networks, a person with high betweenness has more power in the sense that more people depend on him/her to make connections with other people or to access information from other people. Comparing this measure with degree centrality, we can say that degree centrality depends only on the node's neighbors; thus, it is more local than the betweenness centrality, which depends on the connection properties of every pair of nodes in the graph, except pairs with the node in question itself. The equivalent measure exists for edges. The betweenness centrality of an edge is the proportion of the shortest paths between all node pairs which pass through it.

Closeness centrality tries to quantify the position a node occupies in the

networkbased on a distance calculation. The distance metric used between a pair of nodes is defined by the length of its shortest path. The closeness of a node is inversely proportional to the length of the average shortest path between that node and all the

other nodes in the graph. In this case, we interpret a central node as being close to,and able to communicate quickly with, the other nodes in a social network.

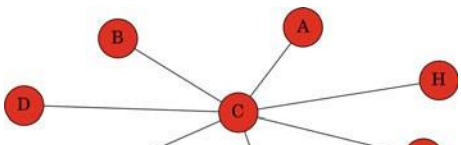
Eigenvector centrality defines a relative score for a node based on its connections and considering that connections from high centrality nodes contribute more to the score of the node than connections from low centrality nodes. It is a measure of the influence of a node in a network, in the following sense: it measures the extent to which a node is connected to influential nodes. Accordingly, an important node is connected to important neighbors.

Let us illustrate the centrality measures with an example. In Fig. 8.3, we showan undirected star graph with $n = 8$ nodes. Node C is obviously important, since it can exchange information with more nodes than the others. The degree centrality measures this idea. In this star network, node C has a degree centrality of 7 or 1 if we consider the normalized measure, whereas all other nodes have a degree of 1or 1/7 if we consider the normalized measure. Another reason why node C is moreimportant than the others in this star network is that it lies between each of the otherpairs of nodes, and no other node lies between C and any other node. If node C wants to contact F, C can do it directly; whereas if node F wants to contact B, it must go through C. This gives node C the capacity to broke/prevent contact amongother nodes and to isolate nodes from information. The betweenness centrality is underneath this idea. In this example, the betweenness centrality of the node C is 28, computed as $(n-1)(n-2)/2$, while the rest of nodes have a betweenness of 0. Thefinal reason why we can say node C is superior in the star network is because C is closer to more nodes than any other node is. In the example, node C is at a distanceof 1 from all other 7 nodes and each other node is at a distance 2 from all other nodes, except C. So, node C has closeness centrality of 1/7, while the rest of nodes have a closeness of 1/13. The normalized measures, computed by dividing by $n-1$, are 1 for C and 7/13 for the other nodes.

An important concept in social network analysis is that of a *hub* node, which is defined as a node with high degree centrality and betweenness centrality. When a hub governs a very centralized network, the network can be easily fragmented by removing that hub.

Coming back to the Facebook example, let us compute the degree centrality of Facebook graph nodes. In the code below we show the user identifier of the 10 mostcentral nodes together with their normalized degree centrality measure. We also show the degree histogram to extract some more information from the shape of the distribution. It might be useful to represent distributions using logarithmic scale. We

Fig. 8.3 Star graph example



do that with the matplotlib.loglog()function. Figure 8.4 shows the degree centrality histogram in linear and logarithmic scales as computed in the box

In[7]:
```
degree_cent_fb = nx.degree_centrality(fb)
print 'Facebook degree centrality: ',
      sorted(degree_cent_fb.items(),
             key = lambda x: x[1],
             reverse = True)[:10]
degree_hist = plt.hist(list(degree_cent_fb.values()), 100)
plt.loglog(degree_hist[1][1:],
           degree_hist[0], 'b', marker = 'o')
```

Out[7]: Facebook degree centrality: [(u'107', 0.258791480931154),
(u'1684', 0.1961367013372957), (u'1912', 0.18697374938088163),
(u'3437', 0.13546310054482416), (u'0', 0.08593363051015354),
(u'2543', 0.07280832095096582), (u'2347', 0.07206537890044576),
(u'1888', 0.0629024269440317), (u'1800', 0.06067360079247152),
(u'1663', 0.058197127290737984)]

The previous plots show us that there is an interesting (large) set of nodes which corresponds to low degrees. The representation using a logarithmic scale (right-hand graphic in Fig. 8.4) is useful to distinguish the members of this set of nodes, whichare clearly visible as a straight line at low values for the x-axis (upper left-hand part of the logarithmic plot). We can conclude that most of the nodes in the graph have low degree centrality; only a few of them have high degree centrality. These latter nodes can be properly seen as the points in the bottom right-hand part of the logarithmic plot.

The next code computes the betweenness, closeness, and eigenvector centrality and prints the top 10 central nodes for each measure.

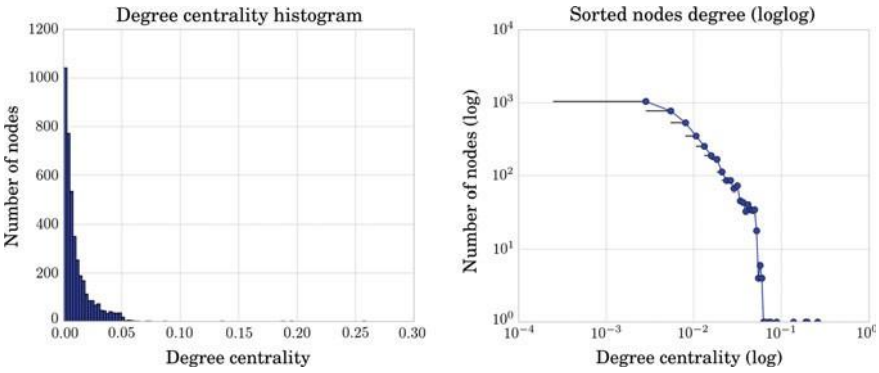


Fig. 8.4 Degree centrality histogram shown using a linear scale (*left*) and a log scale for both the x- and y-axis (*right*)

In [8]:

```
betweenness_fb = nx.betweenness_centrality(fb)
closeness_fb = nx.closeness_centrality(fb)
eigencentrality_fb = nx.eigenvector_centrality(fb)
print 'Facebook betweenness centrality:',
      sorted(betweenness_fb.items(),
             key = lambda x: x[1],
             reverse = True)[:10]
print 'Facebook closeness centrality:',
      sorted(closeness_fb.items(),
             key = lambda x: x[1],
             reverse = True)[:10]
print 'Facebook eigenvector centrality:',
      sorted(eigencentrality_fb.items(),
             key = lambda x: x[1],
             reverse = True)[:10]
```

Out[8]: Facebook betweenness centrality: [(u'107', 0.4805180785560141),(u'1684', 0.33779744973019843), (u'3437', 0.23611535735892616), (u'1912', 0.2292953395868727), (u'1085', 0.1490150921166526), (u'0', 0.1463059214744276), (u'698', 0.11533045020560861), (u'567', 0.09631033121856114), (u'58', 0.08436020590796521), (u'428', 0.06430906239323908)]

Out[8]: Facebook closeness centrality: [(u'107', 0.45969945355191255),(u'58', 0.3974018305284913), (u'428', 0.3948371956585509), (u'563', 0.3939127889961955), (u'1684', 0.39360561458231796), (u'171', 0.37049270575282134), (u'348', 0.36991572004397216), (u'483', 0.3698479575013739), (u'414', 0.3695433330282786), (u'376', 0.36655773420479304)]
Facebook eigenvector centrality: [(u'1912', 0.09540688873596524),(u'2266', 0.08698328226321951), (u'2206', 0.08605240174265624), (u'2233', 0.08517341350597836), (u'2464', 0.08427878364685948), (u'2142', 0.08419312450068105), (u'2218', 0.08415574433673866), (u'2078', 0.08413617905810111), (u'2123', 0.08367142125897363), (u'1993', 0.08353243711860482)]

As can be seen in the previous results, each measure gives a different ordering of the nodes. The node '107' is the most central node for degree (see box Out [7]),betweenness, and closeness centrality, while it is not among the 10 most central nodes for eigenvector centrality. The second most central node is different for closeness and eigenvector centralities; while the third most central node is different for all fourcentrality measures.

Another interesting measure is the *current flow betweenness centrality*, also called *random walk betweenness centrality*, of a node. It can be defined as the probability of passing through the node in question on a random walk starting and ending at some node. In this way, the betweenness is not computed as a function of shortest paths, but of all paths. This makes sense for some social networks where messages may get to their final destination not by the shortest path, but by a random path, as in the case of gossip floating through a social network for example.

Computing the current flow betweenness centrality can take a while, so we

will work with a trimmed Facebook network instead of the original one. In fact, we can

pose the question: What happen if we only consider the graph nodes with more than the average degree of the network (21)? We can trim the graph using degree centrality values. To do this, in the next code, we define a function to trim the graph based on the degree centrality of the graph nodes. We set the threshold to

In[9]:
```
def trim_degree_centrality(graph, degree = 0.01):
    g = graph.copy()
    d = nx.degree_centrality(g)
    for n in g.nodes():
        if d[n] <= degree:
            g.remove_node(n)
    return g
thr = 21.0/(fb.order() - 1.0)

print 'Degree centrality threshold:', thr

fb_trimmed = trim_degree_centrality(fb, degree = thr)
print 'Remaining # nodes:', len(fb_trimmed)
```

Out[9]: Degree centrality threshold: 0.00520059435364Remaining # nodes: 2226

The new graph is much smaller; we have removed almost half of the nodes (we have moved from 4,039 to 2,226 nodes).

The current flow betweenness centrality measure needs connected graphs, as does any betweenness centrality measure, so we should first extract a connected compo- nent from the trimmed Facebook network and then compute the

In[10]:
```
fb_subgraph = list(nx.connected_component_subgraphs(
    fb_trimed))
print '# subgraphs found:', size(fb_subgraph)
print '# nodes in the first subgraph:',
      len(fb_subgraph[0])
betweenness = nx.betweenness_centrality(fb_subgraph[0])
print 'Trimmed FB betweenness: ',
      sorted(betweenness.items(), key = lambda x: x[1],
             reverse = True)[:10]
current_flow = nx.current_flow_betweenness_centrality(
    fb_subgraph[0])
print 'Trimmed FB current flow betweenness:',
      sorted(current_flow.items(), key = lambda x: x[1],
      reverse = True)[:10]
```

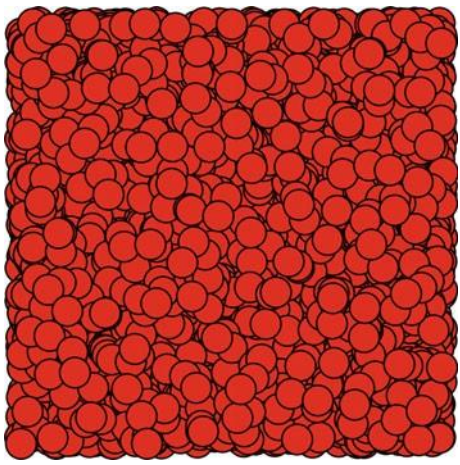


Fig. 8.5 The Facebook network with a randomlayout

Out[10]:# subgraphs found: 2
nodes in the first subgraph: 2225
Trimmed FB betweenness: [(u'107', 0.5469164906683255),
(u'1684', 0.3133966633778371), (u'1912', 0.19965597457246995),
(u'3437', 0.13002843874261014), (u'1577', 0.1274607407928195),
(u'1085', 0.11517250980098293), (u'1718', 0.08916631761105698),
(u'428', 0.0638271827912378), (u'1465', 0.057995900747731755),
(u'567', 0.05414376521577943)]
Trimmed FB current flow betweenness: [(u'107',
0.2858892136334576), (u'1718', 0.2678396761785764), (u'1684',
0.1585162194931393), (u'1085', 0.1572155780323929), (u'1405',
0.1253563113363113), (u'3437', 0.10482568101478178), (u'1912',
0.09369897700970155), (u'1577', 0.08897207040045449), (u'136',
0.07052866082249776), (u'1505', 0.06152347046861114)]

As can be seen, there are similarities in the 10 most central nodes for the between- ness and current flow betweenness centralities. In particular, seven up to ten are thesame nodes, even if they are differently ordered.

Drawing Centrality in Graphs

In this section we focus on graph visualization, which can help in the network dataunderstanding and usability.

The visualization of a network with a large amount of nodes is a complex task. Different layouts can be used to try to build a proper visualization. For instance, we can draw the Facebook graph using the random layout (nx.random_layout), but this is a bad option, as can be seen in Fig. 8.5. Other alternatives can be more useful. In the box below, we use the *Spring* layout, as it is used in the default function (nx.draw), but with more iterations. The function nx.spring_layout returns the position of the nodes using the Fruchterman–Reingold force-directed algorithm.

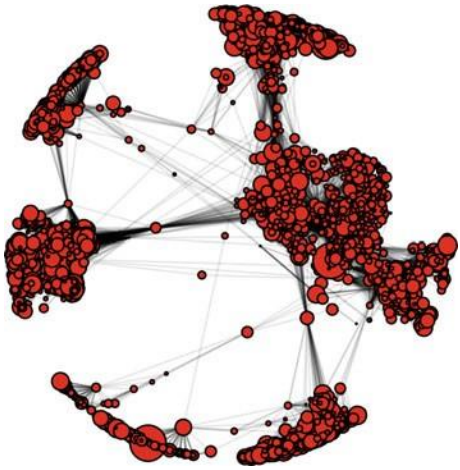


Fig. 8.6 The Facebook network drawn using the Spring layout and degree centrality to define the node size

This algorithm distributes the graph nodes in such a way that all the edges are more or less equally long and they cross themselves as few times as possible. Moreover, we can change the size of the nodes to that defined by their degree centrality. As can be seen in the code, the degree centrality is normalized to values between 0 and 1, and multiplied by a constant to make the sizes appropriate for the format of the figure:

In [11]:
```
pos_fb = nx.spring_layout(fb,iterations = 1000)

nsize = np.array([v for v in degree_cent_fb.values()])

nsize = 500*(nsize - min(nsize))/(max(nsize) - min(nsize))

nodes = nx.draw_networkx_nodes(fb, pos = pos_fb,
                               node_size = nsize)
edges = nx.draw_networkx_edges(fb, pos = pos_fb,
                               alpha = .1)
```

The resulting graph visualization is shown in Fig. 8.6. This illustration allows us to understand the network better. Now we can distinguish several groups of nodes or "communities" clearly in the graph. Moreover, the larger nodes are the more centralnodes, which are highly connected of the Facebook graph.

We can also use the betweenness centrality to define the size of the nodes. In this way, we obtain a new illustration stressing the nodes with higher betweenness, which are those with a large influence on the transfer of information through the network. The new graph is shown in Fig. 8.7. As expected, the central nodes are now those connecting the different communities.

Generally different centrality metrics will be positively correlated, but when they are not, there is probably something interesting about the network nodes. For instance, if you can spot nodes with high betweenness but relatively low degree, these are thenodes with few links but which are crucial for network flow. We can also look for

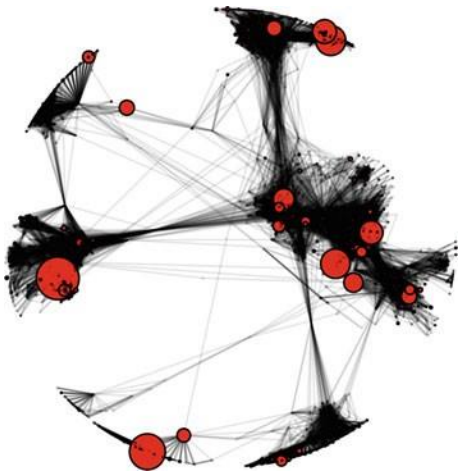


Fig. 8.7 The Facebook network drawn using theSpring layout and betweenness centrality todefine the node size

the opposite effect: nodes with high degree but relatively low betweenness. These nodes are those with redundant communication.

Changing the centrality measure to closeness and eigenvector, we obtain the graphs in Figs. 8.8 and 8.9, respectively. As can be seen, the central nodes arealso different for these measures. With this or other visualizations you will be able to discern different types of nodes. You can probably see nodes with high closeness centrality but low degree; these are essential nodes linked to a few important or active nodes. If the opposite occurs, if there are nodes with high degree centrality but lowcloseness, these can be interpreted as nodes embedded in a community that is far removed from the rest of the network.

In other examples of social networks, you could find nodes with high closeness centrality but low betweenness; these are nodes near many people, but since there may be multiple paths in the network, they are not the only ones to be near many people. Finally, it is usually difficult to find nodes with high betweenness but low closeness, since this would mean that the node in question monopolized the links from a small number of people to many others.

PageRank

PageRank is an algorithm related to the concept of eigenvector centrality in directed graphs. It is used to rate webpages objectively and effectively measure the attention devoted to them. PageRank was invented by Larry Page and Sergey Brin, and becamea Google trademark in 1998 [2].

Assigning the importance of a webpage is a subjective task, which depends on the interests and knowledge of the persons that browse the webpages. However, there are ways to objectively rank the relative importance of webpages.

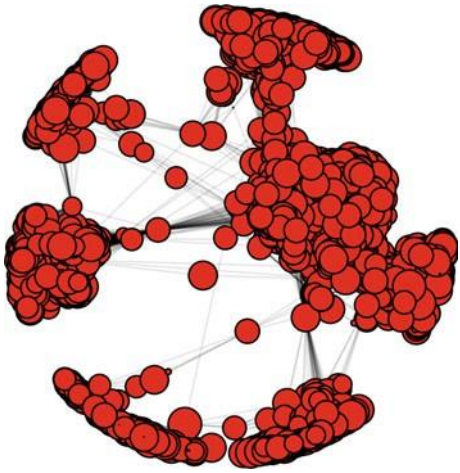


Fig. 8.8 The Facebook network drawn using the Spring layout and closeness centrality to define the nodesize

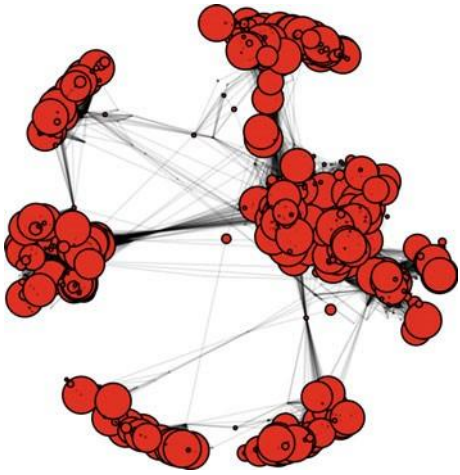


Fig. 8.9 The Facebook network drawn using theSpring layout and eigenvector centrality todefine the node size

We consider the directed graph formed by nodes corresponding to the webpagesand edges corresponding to the hyperlinks. Intuitively, a hyperlink to a page countsas a vote of support and a page has a high rank if the sum of the ranks of its incoming edges is high. This considers both cases when a page has many incoming links and when a page has a few highly ranked incoming links. Nowadays, a variant of the algorithm is used by Google. It does not only use information on the number of edges pointing into and out of a website, but uses many more variables.

We can describe the PageRank algorithm from a probabilistic point of view. The rank of page P_i is the probability that a surfer on the Internet who starts visiting a random page and follows links, visits the page P_i. With more details, we consider

that the weights assigned to the edges of a network by its transition matrix, M, are the probabilities that the surfer goes from one webpage to another. We can understand the

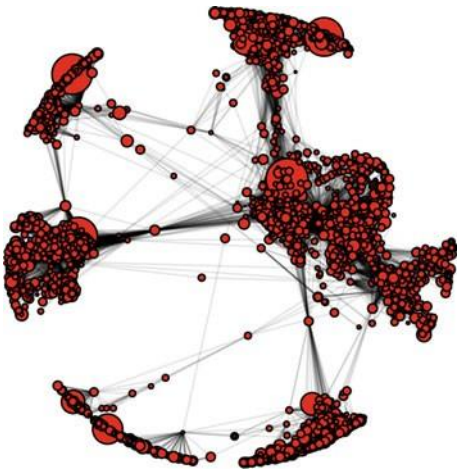


Fig. 8.10 The Facebook network drawn using the Spring layout and PageRank to define the node size

rank computation as a random walk through the network. We start with an initial equal probability for each page: $v_0 = (\frac{1}{n}, \ldots, \frac{1}{n})$, where n is the number of nodes. Then we can compute the probability that each page is visited after one step by applying the transition matrix: $v_1 = Mv$. The probability that each page will be visited after k steps is given by $v_k = M^k a$. After several steps, the sequence converges to a unique probabilistic vector a^* which is the *PageRank vector*. The i-th element of this vector is the probability that at each moment the surfer visits page P_i. We need a nonambiguous definition of the rank of a page for any directed web graph. However,
in the Internet, we can expect to find pages that do not contain outgoing links and this configuration can lead to certain problems to the explained procedure. In order to overcome this problem, the algorithm fixes a positive constant p between 0 and 1 (a typical value for p is 0.85) and redefines the transition matrix of the graph by
$R = (1 - p) M + p B$, where $B = \frac{1}{n} I$, and I is the identity matrix. Therefore, a node with no outgoing edges has probability $\frac{I}{n}$ of moving to any other node.

Let us compute the PageRank vector of the Facebook network and use it to define the size of the nodes, as was done in box In [11].

In [12]:

```
pr = nx.pagerank(fb, alpha = 0.85)
nsize = np.array([v for v in pr.values()])
nsize = 500*(nsize - min(nsize))/(max(nsize) - min(nsize))
nodes = nx.draw_networkx_nodes(fb,
                               pos = pos_fb,
                               node_size = nsize)
edges = nx.draw_networkx_edges(fb,
                               pos = pos_fb,
                               alpha = .1)
```

The code above outputs the graph in Fig. 8.10, that emphasizes some of the nodeswith high PageRank. Looking the graph carefully one can realize that there is one large node per community.

8.5 Ego-Networks

Ego-networks are subnetworks of neighbors that are centered on a certain node. InFacebook and LinkedIn, these are described as “your network". Every person in anego-network has her/his own ego-network and can only access the nodes in it. All ego-networks interlock to form the whole social network. The ego-network definition depends on the network distance considered. In the basic case, a distance of 1, a link means that person A is a friends of person B, a distance of 2 means that a person, C, is a friend of a friend of A, and a distance of 3 means that another person, D, is a friendof a friend of a friend of A. Knowing the size of an ego-network is important when it comes to understanding the reach of the information that a person can transmit or have access to. Figure 8.11 shows an example of an ego-network. The blue node isthe *ego*, while the rest of the nodes are red.

Our Facebook network was manually labeled by users into a set of 10 ego-networks. The public dataset includes the information of these 10 manually definedego-networks. In particular, we have available the list of the 10 ego nodes: ‘0’, ‘107’,‘348’, ‘414’, ‘686’, ‘1684’, ‘1912’, ‘3437’, ‘3980’ and their connections. These ego-networks are interconnected to form the fully connected graph we have been analyzing in previous sections.

In Sect. 8.4 we saw that node ‘107’ is the most central node of the Facebook network for three of the four centrality measures computed. So, let us extract the ego-networks of the popular node ‘107’ with a distance of 1 and 2, and compute their sizes. NetworkX has a function devoted to this task:

In [13]:

```
ego_107 = nx.ego_graph(fb, '107')
print '# nodes of ego graph 107:',
      len(ego_107)
print '# nodes of ego graph 107 with radius up to 2:',
      len(nx.ego_graph(fb, '107', radius = 2))
```

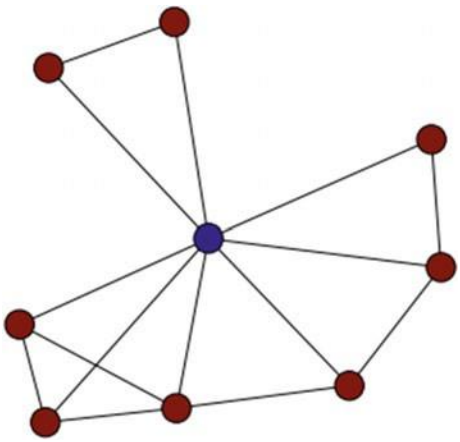


Fig. 8.11 Example of an ego-network. The *blue* nodeis the ego

Out[13]:# nodes of ego graph 107: 1046
nodes of ego graph 107 with radius up to 2: 2687

The ego-network size is 1,046 with a distance of 1, but when we expand the distance to 2, node '107' is able to reach up to 2,687 nodes. That is quite a large ego-network, containing more than half of the total number of nodes.

Since the dataset also provides the previously labeled ego-networks, we can com-pute the actual size of the ego-network following the user labeling. We can access the ego-networks by simply importing os.path and reading the edge list corre-sponding, for instance, to node '107', as in the following code:

In [14]:
```
import os.path
ego_id = 107
G_107 = nx.read_edgelist(
    os.path.join('files/ch08/facebook',
                 '{0}.edges'.format(ego_id)),
    nodetype = int)
print 'Nodes of the ego graph 107:', len(G_107)
```

Out[14]:Nodes of the ego graph 107: 1034

As can be seen, the size of the previously defined ego-network of node '107' is slightly different from the ego-network automatically computed using NetworkX. This is due to the fact that the manual labeling is not necessarily referred to the subgraph of neighbors at a distance of 1.

We can now answer some other questions about the structure of the Facebook network and compare the 10 different ego-networks among them. First, we can compute which the most densely connected ego-network is from the total of 10. Todo that, in the code below, we compute the number of edges in every ego-network and select the network with the maximum number:

In [15]:

```
ego_ids = ( 0, 107, 348,
            414, 686, 698,
            1684, 1912, 3437, 3980)
ego_sizes = zeros((10, 1))
i = 0
# Fill the 'ego_sizes' vector with the size (# edges) of the
    10 ego-networks in egoids
for id in ego_ids :
    G = nx.read_edgelist(
        os.path.join('files/ch08/facebook',
                     '{0}.edges'.format(id)),
        nodetype = int)
    ego_sizes[i] = G.size()
    i = i + 1
[i_max,j] = (ego_sizes == ego_sizes.max()).nonzero()
ego_max = ego_ids[i_max]
print ' The most densely connected ego - network is \
      that of node:', ego_max

G = nx.read_edgelist(
    os.path.join('files/ch08/facebook',
                 '{0}.edges'.format(ego_max)),
    nodetype = int)
print 'Nodes: ', G.order()
print 'Edges: ', G.size()
print 'Average degree: ', G_k / G_n
```

Out[15]:The most densely connected ego-network is that of node: 1912Nodes: 747
Edges: 30025
Average degree: 40

The most densely connected ego-network is that of node '1912', which has an average degree of 40. We can also compute which is the largest (in number of nodes) ego-network, changing the measure of sizes from G.size() by G.order(). In this case, we obtain that the largest ego-network is that of node '107', which has 1,034 nodes and an average degree of 25.

Next let us work out how much intersection exists between the ego-networks inthe Facebook network. To do this, in the code below, we add a field 'egonet' for every node and store an array with the ego-networks the node belongs to. Then, having thelength of these arrays, we compute the number of nodes that belong to 1, 2, 3, 4 andmore than 4 ego-networks:

In [16]:

```
# Add a field 'egonet' to the nodes of the whole facebook
    network.
# Default value egonet = [], meaning that this node does not
     belong to any ego-netowrk
for i in fb.nodes() :
    fb.node[str(i)]['egonet'] = []

# Fill the 'egonet' field with one of the 10 ego values in
    ego_ids:
for id in ego_ids :
    G = nx.read_edgelist(
        os.path.join('files/ch08/facebook',
                     '{0}.edges'.format(id)),
        nodetype = int)
    print id
    for n in G.nodes() :
        if (fb.node[str(n)]['egonet'] == []) :
            fb.node[str(n)]['egonet'] = [id]
        else :
            fb.node[str(n)]['egonet'].append(id)

# Compute the intersections:
S = [len(x['egonet']) for x in fb.node.values()]


print '# nodes into 0 ego-network: ', sum(equal(S, 0))
print '# nodes into 1 ego-network: ', sum(equal(S, 1))
print '# nodes into 2 ego-network: ', sum(equal(S, 2))
print '# nodes into 3 ego-network: ', sum(equal(S, 3))
print '# nodes into 4 ego-network: ', sum(equal(S, 4))
print '# nodes into more than 4 ego-network: ', sum(greater(S, 4))
```

Out[16]:
```
# nodes into 0 ego-network: 80
# nodes into 1 ego-network: 3844
# nodes into 2 ego-network: 102
# nodes into 3 ego-network: 11
# nodes into 4 ego-network: 2
# nodes into more than 4 ego-network: 0
```

As can be seen, there is an intersection between the ego-networks in the Facebook network, since some of the nodes belong to more than 1 and up to 4 ego-networks simultaneously.

We can also try to visualize the different ego-networks. In the following code, we draw the ego-networks using different colors on the whole Facebook network and we obtain the graph in Fig. 8.12. As can be seen, the ego-networks clearly formgroups of nodes that can be seen as communities.

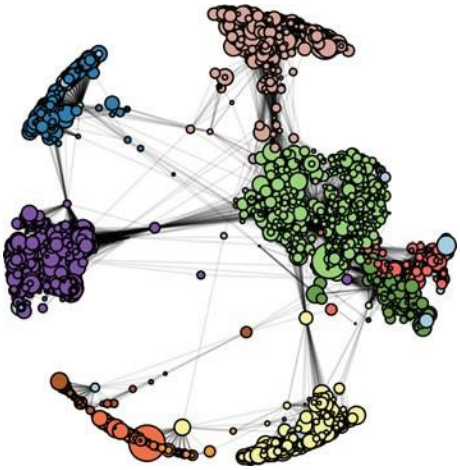


Fig. 8.12 The Facebook network drawn using the Spring layout and different colors to separate the ego-networks

In [17]:

```
# Add a field 'egocolor' to the nodes of the whole facebooknetwork.
# Default value egocolor r =0, meaning that this node
does not belong to any ego-netowrk for i in fb.nodes() :fb.node[str(i)]['egocolor']
    = 0

# Fill the 'egocolor' field with a different color numberfor each ego-network in
    ego_ids:
idColor = 1
for id in ego_ids :
    G = nx.read_edgelist(
        os.path.join('files/ch08/facebook',
                     '{0}.edges'.format(id)),nodetype = int)
    for n in G.nodes() :
        fb.node[str(n)]['egocolor'] = idColoridColor += 1

colors = [x['egocolor'] for x in fb.node.values()]

nsize = np.array([v for v in degree_cent_fb.values()])

nsize =500*(nsize - min(nsize))/(max(nsize)- min(nsize))nodes = nx.

draw_networkx_nodes(
    fb, pos = pos_fb,
    cmap = plt.get_cmap('Paired'),node_color = colors,
    node_size = nsize,
    with_labels = False)
edges=nx.draw_networkx_edges(fb, pos = pos_fb, alpha = .1)
```

However, the graph in Fig. 8.12 does not illustrate how much overlap is there between the ego-networks. To do that, we can visualize the intersection between ego-networks using a *Venn* or an *Euler* diagram. Both diagrams are useful in order to see how networks are related. Figure 8.13 shows the Venn diagram of the Facebook network. This powerful and complex graph cannot be easily built in Python tool-

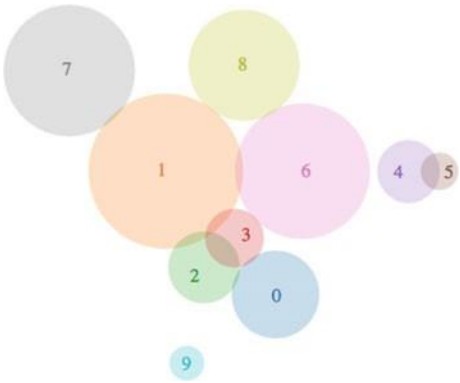


Fig. 8.13 Venn diagram. The area is weighted according to the number of friends in each ego-network and the intersection betweenego-networks is related to the number of common users

boxes like NetworkX or Matplotlib. In order to create it, we have used a JavaScript visualization library called D3.JS.[4]

Community Detection

A community in a network can be seen as a set of nodes of the network that is densely connected internally. The detection of communities in a network is a difficult task since the number and sizes of communities are usually unknown [3].

Several methods for community detection have been developed. Here, we apply one of the methods to automatically extract communities from the Facebook network. We import the Community toolbox[5] which implements the Louvain method for community detection. In the code below, we compute the best partition and plot theresulting communities in the whole Facebook network with different colors, as we did in box In [17]. The resulting graph is shown in Fig. 8.14.

In [18]:
```
import community partition = community.best_partition(fb)
    print "#
communities found:", max(partition.values()) colors2 =
[partition.get(node) for node in fb.nodes()] nsize = np.
    array([v
for v in degree_cent_fb.values()]) nsize = 500*(nsize -
min(nsize))/(max(nsize)- min(nsize)) nodes =
nx.draw_networkx_nodes(
    fb, pos = pos_fb,
    cmap = plt.get_cmap('Paired'),
    node_color = colors2,
    node_size = nsize,
    with_labels = False)
edges = nx.draw_networkx_edges(fb, pos = pos_fb, alpha = .1)
```

.

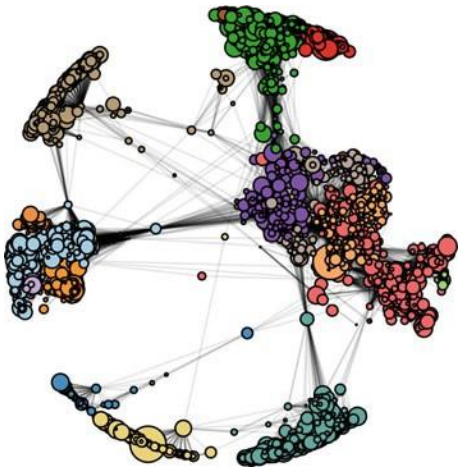


Fig. 8.14 The Facebook network drawn using the Spring layout and different colors to separate the communities found

Out[18]:# communities found: 15

As can be seen, the 15 communities found automatically are similar to the 10 ego-networks loaded from the dataset (Fig. 8.12). However, some of the 10 ego-networks are subdivided into several communities now. This discrepancy can be due to the fact that the ego-networks are manually annotated based on more properties of the nodes, whereas communities are extracted based only on the graph information.